Apna College Bihar

Unit 1.0 : Introduction

Basic Terminologies: Elementary Data Organizations

Introduction :

A data structure is a systematic way of organizing and storing data in computer memory so that it can be accessed, modified and processed efficiently. Choosing an appropriate data structure helps reduce execution time and memory usage.

Data organization refers to arranging data in a meaningful way so that algorithms can perform operations like searching, insertion, deletion, and sorting effectively.

1. Data :

- Data is a collection of raw facts and figures.
- It has no meaning until it is processed.
- Example :
  - Student Roll Number
  - Name
  - Marks
  - Age

Example :

Roll	Name	Marks
101	Rahul	90
102	Priya	85

2. Information :

Information is processed and meaningful data.

Example :

- Rahul scored 90 marks.
- Average marks of the class are 82.

Difference between Data and Information :

Basis	Data	Information
Meaning	Raw facts	Processed data
Nature	No meaning	Meaningful
Stage	Input	Output

3. Data Item :

A Data Item is a single unit of data.

Examples :

- Name
- Roll Number
- Age
- Salary

4. Group Item :

A Group Item is a collection of related data items.

Example : Student Record

→ Roll Number
→ Name
→ Branch
→ Semester
→ Marks

5. Elementary Item :

An Elementary Item cannot be divided into smaller meaningful parts.

Examples :

- Age
- Gender
- Roll Number

Apna College Bihar

6. Entity :

- An Entity is any real-world object about which data is stored.
- Examples : Student, Teacher, Employee, Book, College, Course.

7. Attribute :

An Attribute describes the properties of an entity.

- Example : For entity "Student" attributes are Roll Number, Name, Branch, Age, Marks, etc.

8. Field :

- A Field stores one attribute value.
- A field is also called a data item in a record.

Example :

Roll	Name	Age
101	Rahul	19

Here,
Roll = Field
Name = Field
Age = Field

9. Record :

- A Record is a collection of related fields representing one entity.
- All fields in a record together describe one entity.

Example :

Roll	Name	Branch	Age
101	Rahul	CSE	19

The entire row represents one student record.

10. File :

- A File is a collection of related records.
- All records in a file are of the same type.

Example : Student File

Roll	Name	Branch	Age
101	Rahul	CSE	19
102	Priya	ECE	20
103	Amit	ME	21

The complete table is called a file.

11. Database :

- A Database is a collection of multiple related files organized together.
- It is used to store a large amount of data and maintain relationships between them.

Example : A College Database may contain many files such as :

→ Student File
→ Faculty File
→ Library File
→ Hostel File
→ Examination File

12. Key :

- A Key is an attribute or set of attributes that uniquely identifies a record in a file.
- It helps in searching, sorting and maintaining data integrity.

Example :

Roll	Name
101	Rahul
102	Priya
103	Amit

Roll Number is the key because each student has a unique roll number.

Apna College Bihar

13. Data Structure :

- A Data Structure is a method of organizing and storing data in memory so that operations can be performed efficiently.
- It is used to reduce time complexity and memory usage.

Examples : Array, Linked List, Stack, Queue, Tree, Graph.

14. Classification of Data Structures :

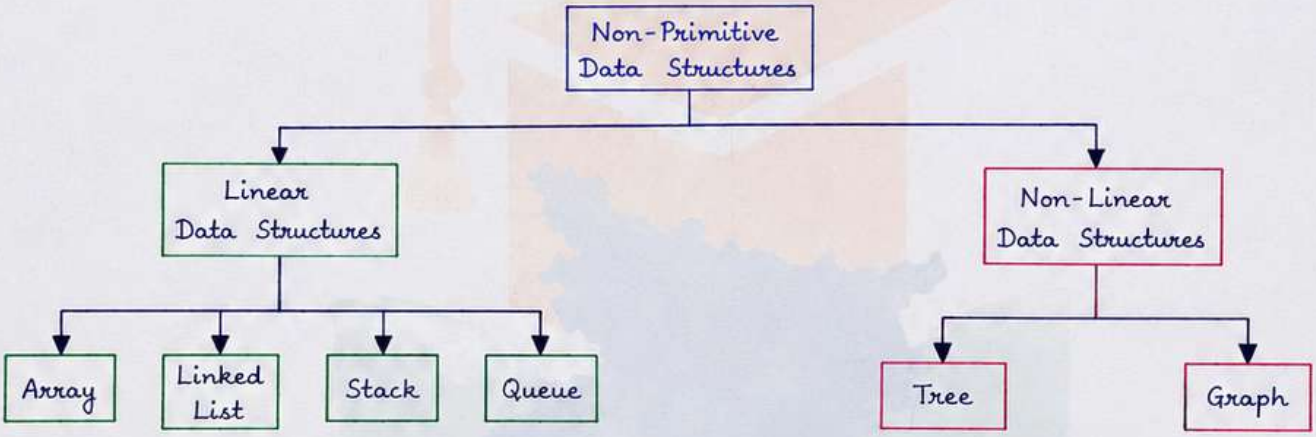
- Data Structures are of two types :
  (A) Primitive Data Structures
  (B) Non-Primitive Data Structures

(A) Primitive Data Structures :

- These store single values.
- Examples : int, float, char, double, bool, etc.

(B) Non-Primitive Data Structures :

- These can store a collection of values.
- They are further divided into two types :



- Elements are arranged in a sequential manner.
- Each element (except first and last) has a unique predecessor and successor.
- Examples : Array, Linked List, Stack, Queue.

- Elements are not arranged in sequence.
- One element can be connected to multiple elements.
- Examples : Tree, Graph.

Summary Table :

	Category	Nature	Example
1.	Primitive Data Structure	Stores single values	int, float, char, double, bool
2.	Non-Primitive Data Structure	Stores collection of values	Array, Linked List, Stack, Queue, Tree, Graph
2.a	Linear Data Structure	Sequential arrangement	Array, Linked List, Stack, Queue
2.b	Non-Linear Data Structure	Non-sequential arrangement	Tree, Graph

Apna College Bihar

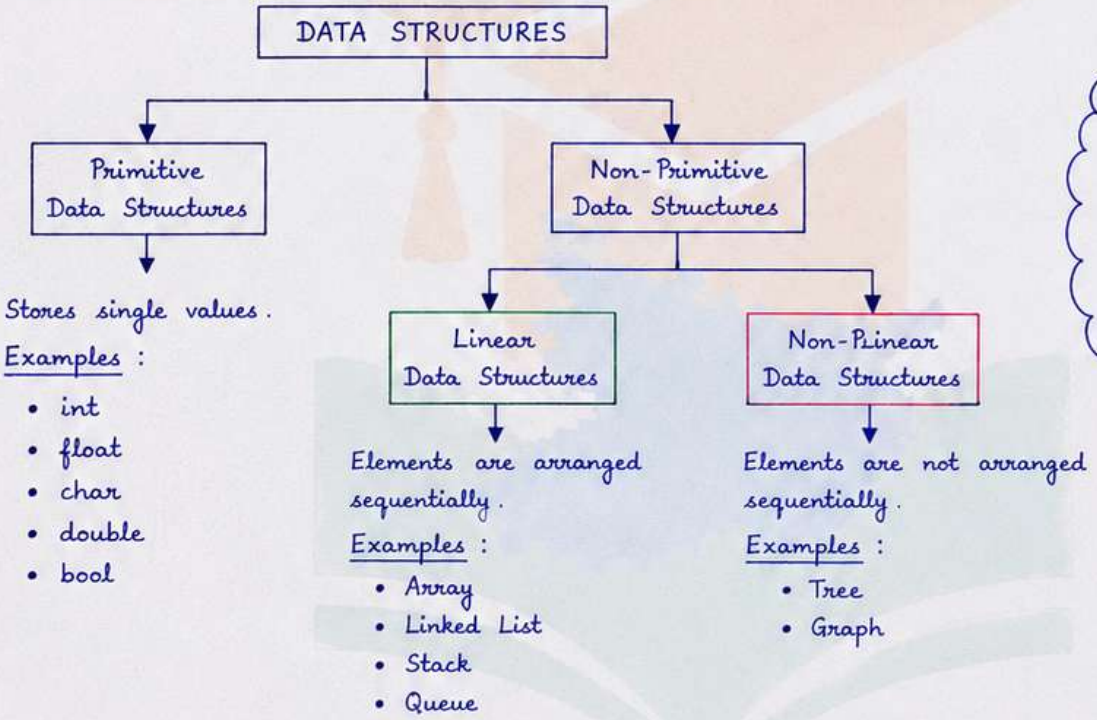
15. Characteristics of a Good Data Structure :

- Efficient memory utilization.
- Fast insertion of data.
- Fast deletion of data.
- Fast searching of data.
- Easy traversal of data.
- Easy implementation.
- Scalable for large datasets.
- Reducing time complexity of operations.
- Improves performance of algorithms.

16. Applications of Data Structures :

- Operating Systems
- Database Management Systems (DBMS)
- Compiler Design
- Artificial Intelligence
- Machine Learning
- Computer Networks
- Web Applications
- Mobile Applications
- Search Engines
- Social Media Platforms

17. Data Structures Overview (Diagram) :



Note :

Choosing the right data structure is very important for writing efficient and optimized algorithms.

18. Exam Points (Important Summary) :

- → Data is raw facts ; Information is processed data.
- → A field stores one attribute value.
- → A record is a collection of fields.
- → A file is a collection of records.
- → A database is a collection of files.
- → A key uniquely identifies a record.
- → Data structure organizes data efficiently.
- → Primitive data structures store single values.
- → Non-primitive data structures are divided into linear and non-linear structures.
- → Arrays, Linked Lists, Stacks and Queues are linear structures.
- → Trees and Graphs are non-linear structures.
- → Good data structure improves performance and reduces time and memory complexity.

Apna College Bihar

Data Structure Operations: Insertion, Deletion, Traversal, and Searching

Introduction :

After storing data in a data structure, different operations are performed to manage and manipulate the data. These operations are called Data Structure Operations. Every data structure (Array, Linked List, Stack, Queue, Tree, Graph, etc.) supports these operations, although the implementation may differ.

The basic operations are :

1. Insertion
2. Deletion
3. Traversal
4. Searching
5. Updating
6. Sorting
7. Merging

---

1. Insertion :

- Insertion is the process of adding a new element into a data structure at a specified position.
- Purpose : Store new information, increase the size of the data structure and maintain updated records.

Example :

Original Array :

10	20	40	50
1	2	3	4

(Index)

→

Insert 30 at position 3 :

10	20	30	40	50
1	2	3	4	5

(Index)

Steps of Insertion :

1. Find the required position.
2. Shift elements (if necessary).
3. Insert the new element.
4. Update the size.

Insertion Process (Insert 30 at position 3)

Step 1 : Original Array

10	20	40	50
1	2	3	4

→ Step 2 : Shift elements

10	20	40	40
1	2	3	4

↙

Step 3 : Insert 30

10	20	30	40	50
1	2	3	4	5

→ Step 4 : Update size

10	20	30	40	50
1	2	3	4	5

Applications :

- Student registration
- Employee records
- Online shopping cart
- Database entry

Apna College Bihar

2. Deletion :

- Deletion is the process of removing an existing element from a data structure.
- Purpose : Remove unwanted data, reduce memory usage, and keep records updated.

Example :

Original Array :

10	20	30	40	50

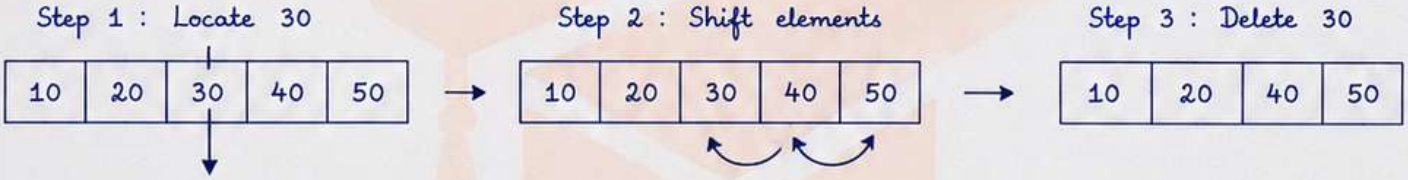
Delete 30 :

10	20	40	50

Steps of Deletion :

1. Locate the element to be deleted.
2. Remove the element.
3. Shift remaining elements (if required).
4. Update the size.

Deletion Process (Delete 30)



Applications :

- Deleting files.
- Removing products from a cart.
- Cancelling student admission.
- Removing employee records.
- Deleting messages or emails.

3. Traversal :

- Traversal means visiting every element of a data structure exactly once in a systematic order.
- Purpose : Display data, count elements, calculate sum or average, and process every record.

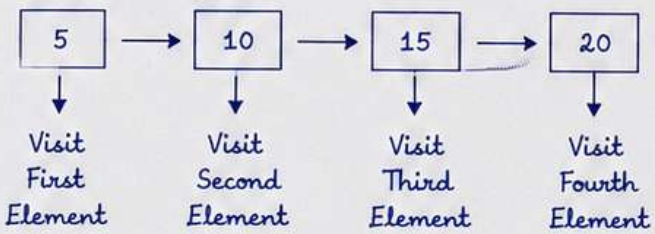
Example :

Array :

5	10	15	20

Traversal Order : 5 → 10 → 15 → 20

Traversal Process :



Applications :

- Printing all records.
- Generating reports.
- Finding maximum and minimum values.
- Data processing and analysis.

Apna College Bihar

4. Searching :

- Searching is the process of finding a particular element in a data structure.
- **Purpose** : Locate required information quickly and check whether data is available.

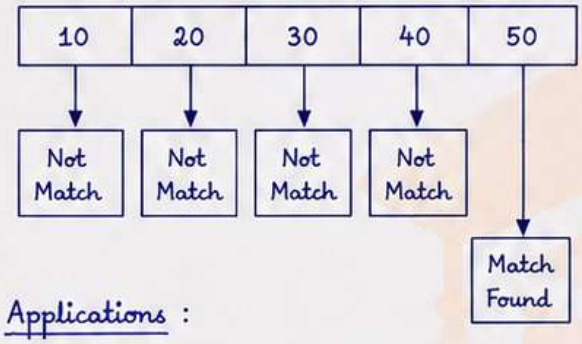
Example :

Array :

10	20	30	40	50
0	1	2	3	4

(Index)

Searching Process (Linear Search for 40)



Search 40 :

- $10 \neq 40$
- $20 \neq 40$
- $30 \neq 40$
- $40 = 40$ ⟶ Element Found at Index 3

Search 60 :

- $10 \neq 60$
- $20 \neq 60$
- $30 \neq 60$
- $40 \neq 60$
- $50 \neq 60$ ⟶ Element Not Found

Applications :

- Search student roll number.
- Search employee ID.
- Search contact number.
- Search product in an online store.

Note :

- Linear Search works in any data structure.
- Binary Search works only on sorted data.

---

5. Updating :

- Updating means modifying the value of an existing element.
- **Purpose** : Keep records current and correct.

Example :

Before Update :

Roll No.	Name	Mobile
101	Rahul	9123456780

⟶

After Update (Mobile No. changed)

Roll No.	Name	Mobile
101	Rahul	9876543210

Update Process :

Locate the record ⟶ Identify the field to be updated ⟶ Modify the value ⟶ Save the changes ⟶ Record is updated

Applications :

- Updating marks.
- Changing address.
- Editing profile details.
- Updating stock quantity.

Important Points :

★ Search is used to find data.
★ Update is used to modify existing data.
★ Both operations are essential for maintaining accurate information.

Apna College Bihar

6. Types of Searching :

① Linear Search : Elements are checked one by one from start to end.

② Binary Search : Elements are checked by dividing the data into two halves.

Linear Search (Example) :

Array :

10	20	30	40	50
0	1	2	3	4

Search 30 :

Compare 10 → Not match → Compare 20 → Not match → Compare 30 → Match → Element Found at Index 2

Binary Search (Example) :

Sorted Array :

10	20	30	40	50
0	1	2	3	4

Search 30 :

Step 1 : Mid = 2 (Element = 30) → Match Found
(10, 20, [30], 40, 50)

Step 2 : If not match, compare with mid
(If less → left half, If greater → right half)

7. Updating :

- Updating means changing the value of an existing element.
- Purpose : Modify old data and keep records correct.

Example :

Before Update :

Roll No.	Name	Marks
101	Aditi	78

→ After Update (Marks changed)

Roll No.	Name	Marks
101	Aditi	92

Applications :

- Updating marks in result.
- Updating address or mobile number.
- Editing customer information.

8. Sorting :

- Sorting means arranging data in a particular order.
- Purpose : Easy searching and better data analysis.

Example :

Unsorted Array :

40	10	30	20

Ascending Order :

10	20	30	40

Descending Order :

40	30	20	10

Sorting Algorithms

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort

9. Merging :

- Merging means combining two or more data structures into one.
- Purpose : Combine datasets and process them as one.

Example :

Array A :

10	20	30

Array B :

40	50	60

→ Merged Array :

10	20	30	40	50	60

Merge Process :

Step 1 : Start

A :		20	30		
B :	40	50	60		
M :					

Step 2 : 10

A :	20	30			
B :	40	50	60		
M :	10				

Step 3 : 20

A :	30				
B :	40	50	60		
M :	10	20			

Step 4 : 30

A :					
B :	40	50	60		
M :	10	20	30		

Step 5 : Copy remaining

A :						
B :						
M :	10	20	30	40	50	60

Applications :

- Merging student lists of two classes.
- Combining two sorted arrays.
- Merging two databases.

Important Points :

★ Linear Search is simple but slow for large data.
★ Binary Search is fast but works only on sorted data.
★ Updating keeps data accurate and up-to-date.
★ Sorting helps in fast searching and better performance.
★ Merging combines data from multiple sources.

Apna College Bihar

Analysis of an Algorithm

1. Introduction :

An Algorithm is a finite sequence of well-defined instructions used to solve a problem or perform a task. However, every algorithm does not perform equally. Some algorithms execute faster and use less memory than others. Therefore, before implementing an algorithm, its efficiency must be analyzed.

Analysis of an Algorithm is the process of evaluating the performance of an algorithm in terms of execution time and memory usage as the input size increases.

The main objective is to select the most efficient algorithm for solving a problem.

2. What is Algorithm Analysis ?

Algorithm Analysis is the study of an algorithm to determine :

- How much time it takes to execute.
- How much memory it requires.
- How it performs for small and large inputs.
- Whether it is efficient or not.

3. Objectives :

- Compare different algorithms.
- Select the best algorithm.
- Reduce execution time.
- Minimize memory usage.
- Improve overall system performance.

4. Need for Algorithm Analysis :

- Different algorithms solve the same problem with different efficiencies.
- Large datasets require faster algorithms.
- Efficient algorithms reduce CPU time.
- Better algorithms improve user experience.
- They optimize hardware resource utilization.

Apna College Bihar

5. Types of Analysis :

There are mainly two types of analysis.

- (A) Time Complexity
- (B) Space Complexity

(A) Time Complexity :

Definition : Time Complexity is the amount of time an algorithm takes to complete its execution as the input size (n) increases. It measures the growth of execution time rather than the actual time in seconds.

Factors Affecting Time Complexity :

- Number of input elements
- Number of loops
- Nested loops
- Recursive calls
- Number of comparisons

Example :

```
for(i = 1; i <= n; i++)
{
    print(i);
}
```

The loop runs n times.

Time Complexity = O(n)

(B) Space Complexity :

Definition : Space Complexity is the total amount of memory required by an algorithm during execution.

It includes :

- Variables
- Arrays
- Function calls
- Recursion stack
- Temporary memory

Example :

```
int a[100];
. .
.
. . .
a[99] = 0;
```

The array occupies memory for 100 integers.

6. Comparison Table :

Basis	Time Complexity	Space Complexity
Meaning	Execution time	Memory usage
Measures	Speed of algorithm	Amount of memory used
Depends on	Number of operations	Memory allocated
Goal	Faster execution	Less memory usage
Examples	O(1), O(n), O(n^2), O(log n)	O(1), O(n), O(n^2), etc.
Measured in	Number of steps (operations)	Memory units (bytes, KB, MB)

7. Examples of Time and Space Complexity :

(i) Constant Time :

```
x = a + b;
```

Time Complexity = O(1)
Space Complexity = O(1)

(ii) Linear Time :

```
for(i = 1 to n)
  sum = sum + a[i];
```

Time Complexity = O(n)
Space Complexity = O(1)

(iii) Quadratic Time :

```
for(i = 1 to n)
  for(j = 1 to n)
    c[i][j] = 0;
```

Time Complexity = O(n^2)
Space Complexity = O(n^2)

(iv) Logarithmic Time :

```
while (n > 1)
  n = n / 2;
```

Time Complexity = O(log n)
Space Complexity = O(1)

Apna College Bihar

8. Cases of Algorithm Analysis :

① **Best Case** :
It is the minimum time required by an algorithm to complete its execution.
Example : Searching the first element in Linear Search.

Array :

10	20	30	40	50
0	1	2	3	4

Index : 0 2 3 4

Element found at first position.

② **Average Case** :
It is the expected execution time for a random input.
Example : Searching an element in the middle of the array.

10	20	30	40	50
0	1	2	3	4

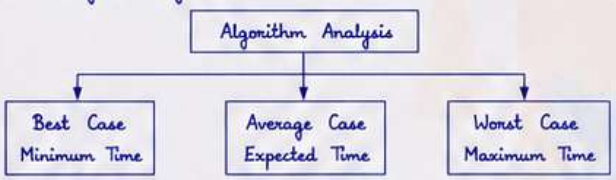
Element found at middle position.

③ **Worst Case** :
It is the maximum time required by an algorithm to complete its execution.
Example : Searching the last element in Linear Search.

10	20	30	40	50
0	1	2	3	4

Element found at last position.

Case Analysis Diagram :



★ Generally, Worst Case is considered while designing algorithms.

9. Performance Measurement :

The performance of an algorithm depends on the following factors :

- Execution Time
- Memory Usage
- Input Size
- CPU Speed
- Compiler Optimization
- Hardware and System Configuration.

10. Characteristics of a Good Algorithm :

① Correctness – Gives the correct output.
② Simplicity – Easy to understand and implement.
③ Efficiency – Takes less time and memory.
④ Finiteness – Must end after a finite number of steps.
⑤ Well-defined Steps – Each step must be clear and unambiguous.
⑥ Generality – Works for all valid inputs.
⑦ Low Memory Usage – Uses minimum memory.
⑧ Fast Execution – Completes in less time.

11. Case Analysis Example : Linear Search

Linear Search Algorithm :

```
for(i = 0; i < n; i++)
{
    if(a[i] == key)
        return i;
}
return -1;
```

- Best Case : key = a[0] → 1 comparison.
- Average Case : key in middle → $\frac{(n+1)}{1+1/2}$ comparisons
- Worst Case : key = a[n-1] or not present → n comparisons

Case	Condition	Number of Comparisons	Time Complexity
Best Case	Key at first position	1	O(1)
Average Case	Key in middle	(n+1)/2	O(n)
Worst Case	Key at last / not found	n	O(n)

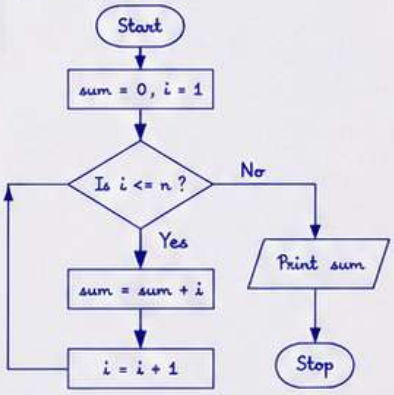
12. Algorithm Execution Flow (Example)

Example : Find Sum of first n natural numbers

Algorithm :

```
sum = 0;
for(i = 1; i <= n; i++)
{
    sum = sum + i;
}
print(sum);
```

Flow Diagram :



Key Points to Remember :

★ Algorithm Analysis helps to evaluate efficiency.
★ Two main parameters – Time and Space Complexity.
★ Time → Number of operations.
★ Space → Amount of memory used.
★ Worst Case is important for algorithm design.
★ A good algorithm is correct, efficient and simple.
★ Analysis helps in selecting the best algorithm.
★ It reduces time, saves memory and improves overall performance.

Apna College Bihar

Asymptotic Notations

1. Introduction :

When comparing algorithms, the exact execution time depends on factors like CPU speed, compiler, and hardware. Therefore, instead of measuring actual time, we use Asymptotic Notations to describe how an algorithm grows as the input size (n) increases.

Asymptotic Notations provide a mathematical way to analyze and compare the efficiency of algorithms.

They help determine the performance of an algorithm for large input sizes without depending on the machine or programming language.

2. What are Asymptotic Notations ?

Definition : Asymptotic Notations are mathematical tools used to describe the growth rate of an algorithm's time complexity or space complexity as the input size increases.

Objectives :

- Compare algorithms.
- Measure efficiency.
- Predict performance for large inputs.
- Ignore machine-dependent factors.
- Choose the best algorithm.

3. Need for Asymptotic Notations :

- Different computers execute programs at different speeds.
- Exact execution time changes with hardware.
- They provide a standard method for comparison.
- They simplify complexity analysis.
- They are useful for designing efficient algorithms.

4. Types of Asymptotic Notations :

There are three main types :

1. Big O Notation (O)
2. Big Omega Notation (Ω)
3. Big Theta Notation (Θ)

Why these notations are important ?

- ★ They help compare algorithms.
- ★ They focus on large input behavior.
- ★ They are independent of machine, programming language and compiler.
- ★ They help us select the most efficient algorithm.



5. Types of Asymptotic Notations :

There are three main types :

1. Big O Notation (O)
2. Big Omega Notation (Ω)
3. Big Theta Notation (Θ)

6. Big O Notation (O) :

- Big O represents the upper bound or Worst Case performance of an algorithm.
- It tells the maximum amount of time an algorithm may take.

Example :

```
for(i = 1; i <= n; i++)
{
    print(i);
}
```

→ The loop runs n times.

Time Complexity = O(n)

More Examples :

- O(1) → Constant Time
- O(log n) → Logarithmic Time
- O(n) → Linear Time
- O(n log n) → Linear Logarithmic Time
- O(n²) → Quadratic Time

7. Common Time Complexities :

S.No.	Complexity	Name	Performance
①	O(1)	Constant	Excellent
②	O(log n)	Logarithmic	Very Fast
③	O(n)	Linear	Good
④	O(n log n)	Linear Logarithmic	Efficient
⑤	O(n²)	Quadratic	Slow
⑥	O(n³)	Cubic	Very Slow
⑦	O(2ⁿ)	Exponential	Extremely Slow
⑧	O(n!)	Factorial	Worst

Observations :

★ Lower complexity means better efficiency.

★ O(1) is the best and O(n!) is the worst.

8. Big Omega Notation (Ω) :

- Big Omega represents the lower bound or Best Case performance of an algorithm.
- It tells the minimum time required by an algorithm.

Example :

Linear Search

If the required element is found at the first position.

Time Complexity = Ω(1)

9. Big Theta Notation (Θ) :

- Big Theta represents the tight bound of an algorithm.
- It indicates that the algorithm performs approximately
- the same in both upper and lower bounds.

Example :

```
for(i = 1; i <= n; i++)
{
    print(i);
}
```

Best Case = Ω(n)
Worst Case = O(n)

Therefore,
Time Complexity = Θ(n)

10. Summary of Three Notations :

Notation	Represents	Case
Big O (O)	Upper Bound	Worst Case
Big Omega (Ω)	Lower Bound	Best Case
Big Theta (Θ)	Tight Bound	Average / Exact Growth

Mathematically,

$$\Omega(n) \leq \Theta(f(n)) \leq O(g(n))$$

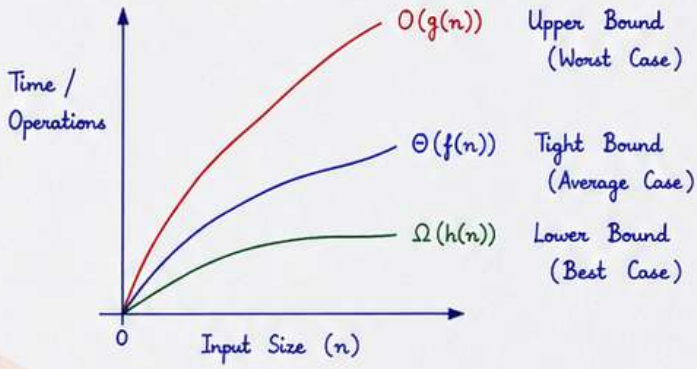
for large values of n.

Apna College Bihar

11. Comparsion of Asymptotic Notations :

Notation	Represents	Case	Meaning
Big O (O)	Upper Bound	Worst Case	Maximum time an algorithm may take.
Big Omega (Ω)	Lower Bound	Best Case	Minimum time an algorithm may take.
Big Theta (Θ)	Tight Bound	Average / Exact Growth	Algorithm grows nearly the same in best and worst case.

12. Three Notation Comparison Diagram :



13. Example Using All Three Notations :

Example : Linear Search

Searching an element in an array of size n.

- Best Case : Element found at first position → Ω(1)
- Worst Case : Element found at last position → O(n)
- Average Case : Element found in middle → Θ(n)

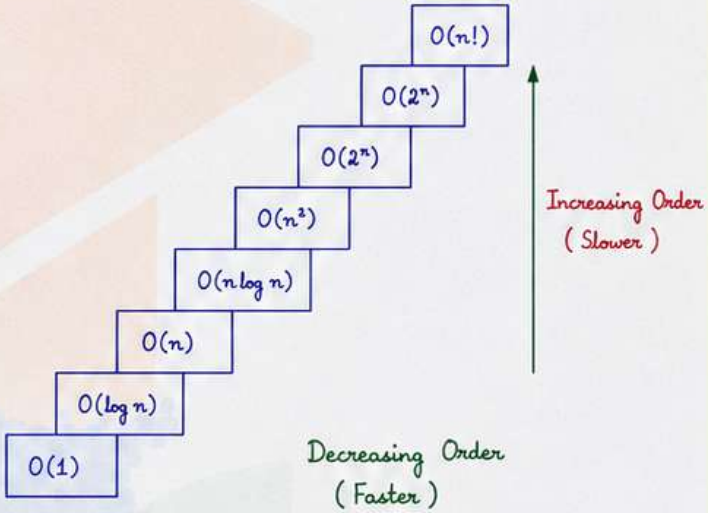
```
for(i = 1; i<= n; i++)
{
    if(a[i] == key)
        return i;
}
return -1;
```

→ Best Case : Ω(1)

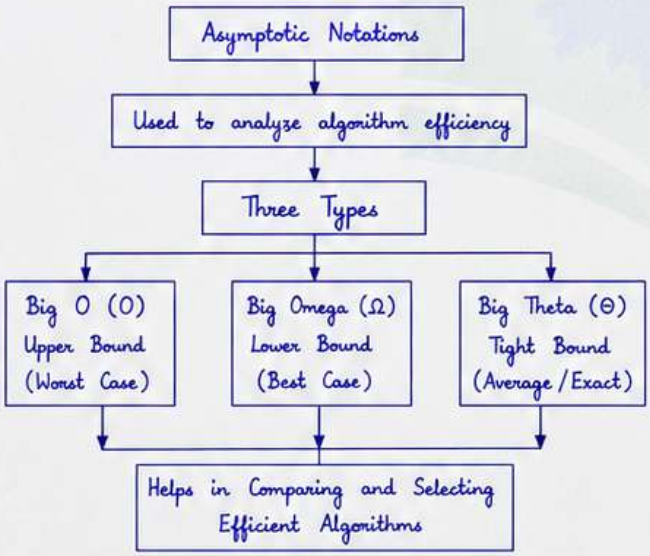
→ Worst Case : O(n)

→ Average Case : Θ(n)

14. Order of Growth (Increasing Order) :



15. Asymptotic Notations Flowchart :



16. Applications :

- Algorithm comparison.
- Software optimization.
- Competitive programming.
- Database query optimization.
- Artificial Intelligence.
- Machine Learning.
- Operating Systems.
- Compiler design.
- Network routing algorithms.
- Cryptography.

Exam Points (Very Important)

★ Asymptotic Notations measure growth rate for large input sizes.

★ Big O (O) gives Upper Bound (Worst Case).

★ Big Omega (Ω) gives Lower Bound (Best Case).

★ Big Theta (Θ) gives Tight Bound (Average / Exact Growth).

★ They are independent of machine, language and compiler.

★ Lower complexity means better efficiency.

★ Used to compare and select the best algorithm.

Apna College Bihar

Time - Space Trade-off

1. Introduction :

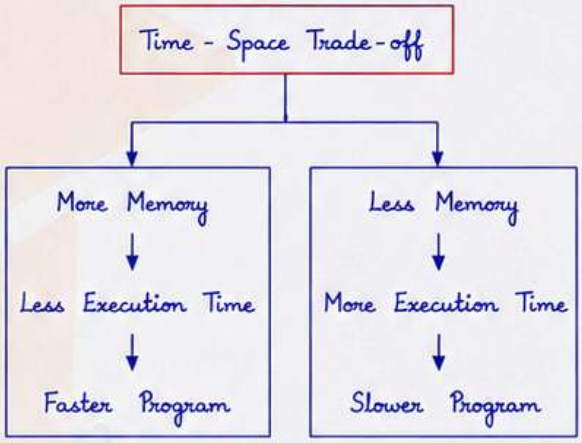
In computer science, an algorithm should use both minimum execution time and minimum memory. However, it is often impossible to optimize both simultaneously. Sometimes we use more memory to reduce execution time, while in other situations we use less memory but accept longer execution time. This concept is known as the Time - Space Trade-off.

A programmer chooses the appropriate balance between time and space according to the application's requirements.

2. What is Time - Space Trade-off ?

Definition :

The Time - Space Trade-off is a technique in which an algorithm sacrifices memory to improve speed or sacrifices speed to reduce memory usage.



In simple words :

More Memory	→	Less Execution Time
Less Memory	→	More Execution Time

3. Need for Time - Space Trade-off :

- Computer memory is limited.
- Fast execution is important in real-time systems.
- Large datasets require efficient resource management.
- Different applications have different requirements.
- It helps optimize overall system performance.

4. Time - Space Relationship :

(A) More Space, Less Time

- Extra memory is used.
- Repeated calculations are avoided.
- Execution becomes faster.

Examples :

1. Hash Tables
2. Caching
3. Dynamic Programming
4. Lookup Tables

(B) Less Space, More Time

- Very little extra memory is used.
- The algorithm performs repeated calculations.
- Execution takes more time.

Examples :

1. Linear Search
2. Recursive algorithms without memoization
4. Simple iterative algorithms

Note :

There is a balance between time and space. We have to choose what is more important for a particular problem.

Apna College Bihar

5. More Space, Less Time :

- Extra memory is used.
- Repeated calculations are avoided.
- Execution becomes faster.

Examples :

1. Hash Tables
2. Caching
3. Dynamic Programming
4. Lookup Tables

6. Less Space, More Time :

- Very little extra memory is used.
- The algorithm performs repeated calculations.
- Execution takes more time.

Examples :

1. Linear Search
2. Recursive algorithms without memoization
3. Simple iterative algorithms

7. Advantages :

1. Faster execution when required.
2. Better utilization of available resources.
3. Flexible algorithm design.
4. Improved performance in many applications.
5. Useful for handling large datasets.

8. Disadvantages :

1. Higher memory consumption.
2. Increased implementation complexity.
3. More expensive hardware may be required.
4. Memory wastage in some cases.

9. Comparison Table :

Basis	More Space, Less Time	Less Space, More Time
Memory Usage	High	Low
Execution Time	Fast	Slow
Repeated Calculations	No	Yes
Performance	Better	Lower
Example	Hash Table	Linear Search

10. Real-life Examples :

(A) Cache Memory :



- More memory is used.
- CPU execution becomes faster.

(B) Browser Cache :



- More disk space is used.
- Websites load quickly.

(C) Google Maps :



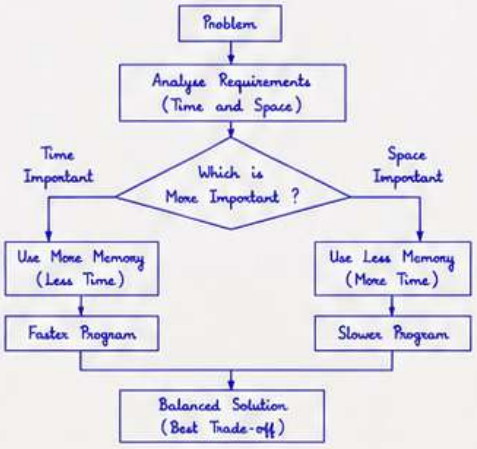
- Routes and locations are stored.
- More storage is required.
- Navigation becomes faster.

(D) Database Indexing :

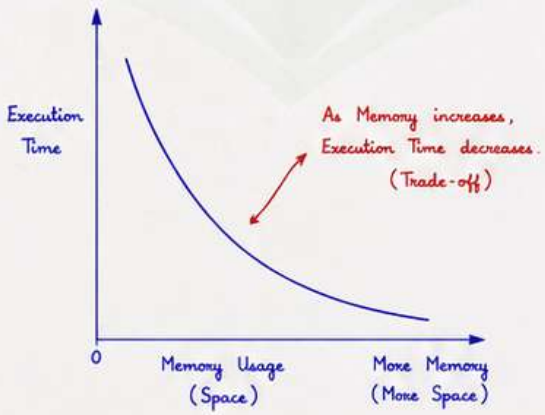


- More storage is required for indexes.
- Searching becomes much faster.

11. Time - Space Trade-off Flowchart :



12. Memory vs Execution Time Graph :



13. Exam Points (Summary)

- ★ Time - Space Trade-off balances execution time and memory usage.
- ★ More memory generally reduces execution time.
- ★ Less memory generally increases execution time.
- ★ Cache memory, hashing and dynamic programming are examples of using more memory for better speed.
- ★ It helps in selecting the most suitable algorithm based on system requirements.

Apna College Bihar

Topic 6 : Resource Allocation in Algorithms

1. Introduction :

An algorithm needs resources to execute. These resources include CPU time, memory, I/O devices, bandwidth, storage and other system facilities. Efficient allocation of these resources is important to achieve good performance.

2. What is Resource Allocation ?

Resource Allocation is the process of distributing available system resources among different tasks or processes to maximize efficiency and minimize waiting time.

In simple words :

Right resource	→	Right process
Right time	→	Better performance

3. Types of Resources :

- CPU (Processor Time)
- Memory (Primary and Secondary)
- I/O Devices (Printer, Disk, Scanner)
- Network Bandwidth
- Storage Space
- File and Data Resources

4. Objectives of Resource Allocation :

1. Maximize CPU utilization.
2. Minimize waiting time.
3. Maximize throughput.
4. Ensure fairness among processes.
5. Avoid deadlock and starvation.
6. Improve overall system performance.

5. Need for Resource Allocation :

- System resources are limited.
- Many processes compete for resources.
- Proper allocation increases system efficiency.
- Prevents resource wastage.
- Helps in achieving better performance.

6. Resource Allocation Process :



7. Resource Allocation Strategies :

- Contiguous Allocation
- Non-contiguous Allocation
- Preemptive Allocation
- Non-preemptive Allocation
- Dynamic Allocation
- Static Allocation

Choice of strategy depends on the type of system and requirements.

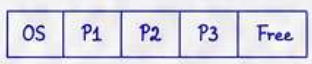
8. Examples :

(A) CPU Scheduling



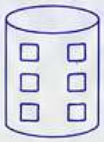
CPU time is allocated to processes according to scheduling algorithm.

(B) Memory Allocation



Memory is divided and allocated to processes.

(C) Disk Allocation



Disk blocks are allocated to files or processes.

(D) Bandwidth Allocation

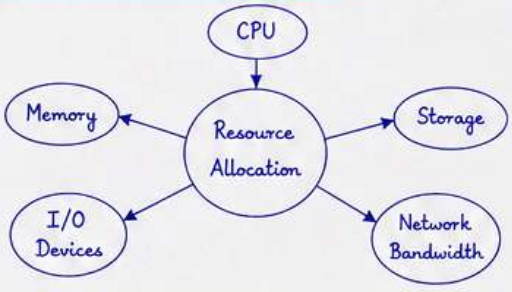


Network bandwidth is shared among users.

9. Resource Allocation vs Poor Allocation :

Basis	Good Resource Allocation	Poor Resource Allocation
CPU Utilization	High	Low
Waiting Time	Low	High
Throughput	High	Low
Resource Usage	Efficient	Wastage
Performance	Better	Poor
Example	CPU Scheduling	Improper Scheduling

10. Resource Allocation Diagram :



Key Point :

Efficient resource allocation ensures better performance, fairness and optimum use of system resources.

Apna College Bihar

11. Real-Life Examples :

(A) Cache Memory :



- More memory is used.
- CPU execution becomes faster.

(B) Browser Cache :



- More disk space is used.
- Websites load quickly.

(C) Google Maps :



- Routes and locations are stored.
- More storage is required.
- Navigation becomes faster.

(D) Database Indexing :

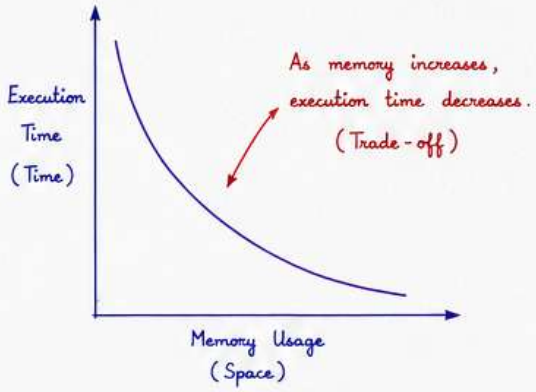


- More storage is required for indexes.
- Searching becomes much faster.

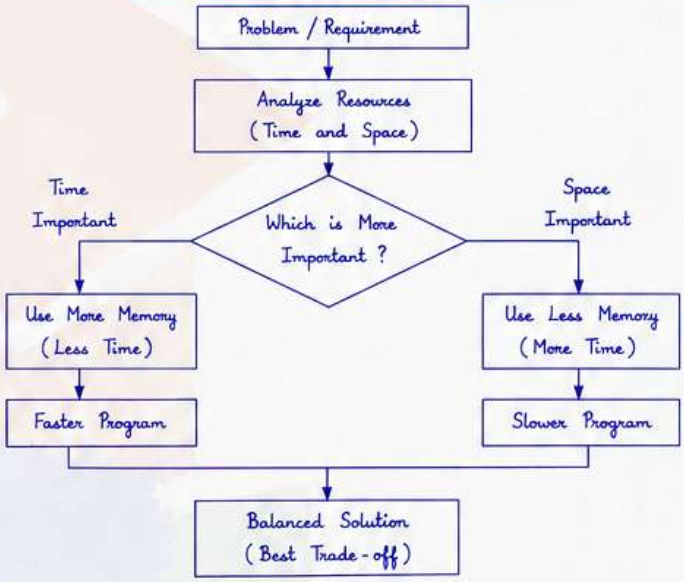
12. Applications of Resource Allocation :

1. Operating Systems
2. Database Management Systems
3. Artificial Intelligence
4. Machine Learning
5. Search Engines
6. Computer Networks
7. Web Browsers
8. Mobile Applications
9. Data Compression
10. Cloud Computing

13. Memory vs Execution Time Graph :



14. Complete Time-Space Trade-off Flowchart :



15. Memory vs Time Trade-off Table :

Aspect	More Space, Less Time	Less Space, More Time
Memory Usage	High	Low
Execution Time	Less (Fast)	More (Slow)
Repeated Calculations	Avoided	Performed
Implementation	Complex	Simple
Hardware Cost	High	Low
Performance	Better	Lower
Example	Hash Table, Caching, Dynamic Programming	Linear Search, Recursive (without memoization)

16. Resource Allocation Pyramid :



17. Time-Space Trade-off Summary :

- ★ More memory generally reduces execution time.
- ★ Less memory generally increases execution time.
- ★ Cache memory, hashing and dynamic programming are examples of using more memory for better speed.
- ★ Time-Space Trade-off helps in selecting the most suitable algorithm based on system requirements.

18. Exam Points (Very Important)

1. Time-Space Trade-off balances execution time and memory usage.
2. Using more memory generally reduces execution time.
3. Using less memory generally increases execution time.
4. Cache memory, hashing, lookup tables and dynamic programming are examples.
5. It helps in achieving better efficiency and performance.
6. Proper allocation prevents resource wastage.
7. Choice of strategy depends on problem type and system needs.

Apna College Bihar

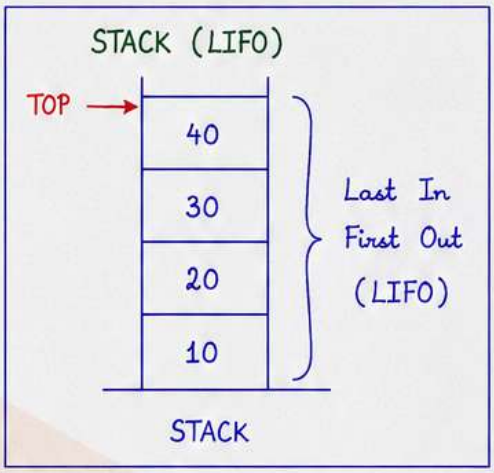
Unit 2.0 : Stacks and Queues

Topic 1 : ADT Stack and its Operations : Algorithms and Their Complexity Analysis

1. Introduction :

A Stack is a linear data structure that follows the LIFO (Last In, First Out) principle. It means the element inserted last is removed first. A Stack is used in many computer science applications such as function calls, expression evaluation, recursion, browser history, and undo operations.

An Abstract Data Type (ADT) defines the logical behavior of a data structure without specifying how it is implemented. Thus, a Stack ADT specifies the operations that can be performed on a stack.



2. What is Stack ADT ?

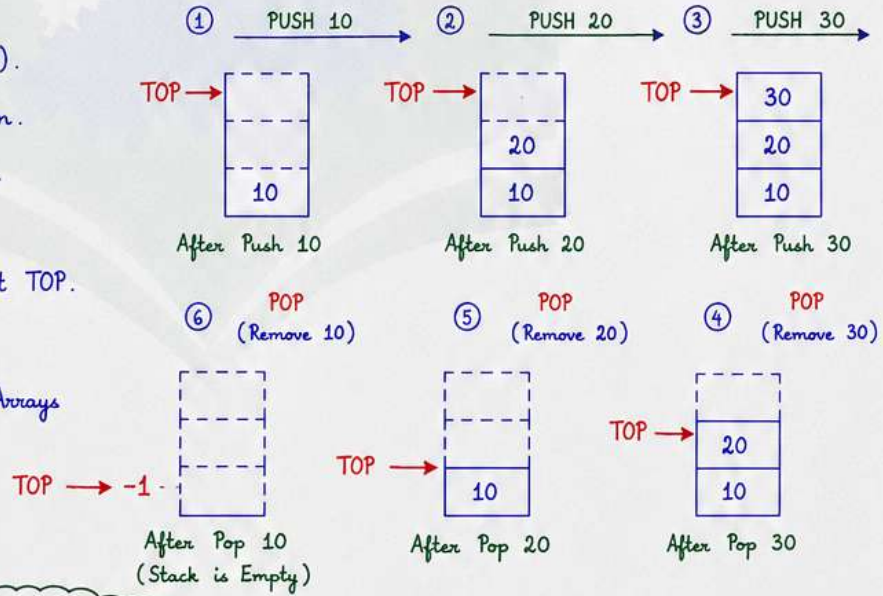
A Stack ADT is a linear abstract data type in which insertion and deletion take place only from one end called the TOP.

> **In simple words :**
> We can insert and delete elements only from one end of the stack called TOP.

3. Characteristics of Stack :

- ★ Follows LIFO (Last In, First Out).
- ★ Insertion is called PUSH operation.
- ★ Deletion is called POP operation.
- ★ Only one end (TOP) is used.
- ★ New element is always added at TOP.
- ★ Element is removed from TOP.
- ★ It can be implemented using Arrays or Linked Lists.

LIFO Illustration :



> **Note :**
> LIFO means "Last In, First Out".
> Element 30 is inserted last, so it is removed first.
> Then 20 is removed, and finally 10 is removed.

Apna College Bihar

4. Basic Stack Operations :

The main operations performed on a Stack ADT are :

1. PUSH
2. POP
3. PEEK (TOP)
4. isEmpty()
5. isFull() (In Array Implementation)
6. SIZE()

5. Push Operation :

The PUSH operation inserts a new element at the TOP of the stack.

Push Algorithm :

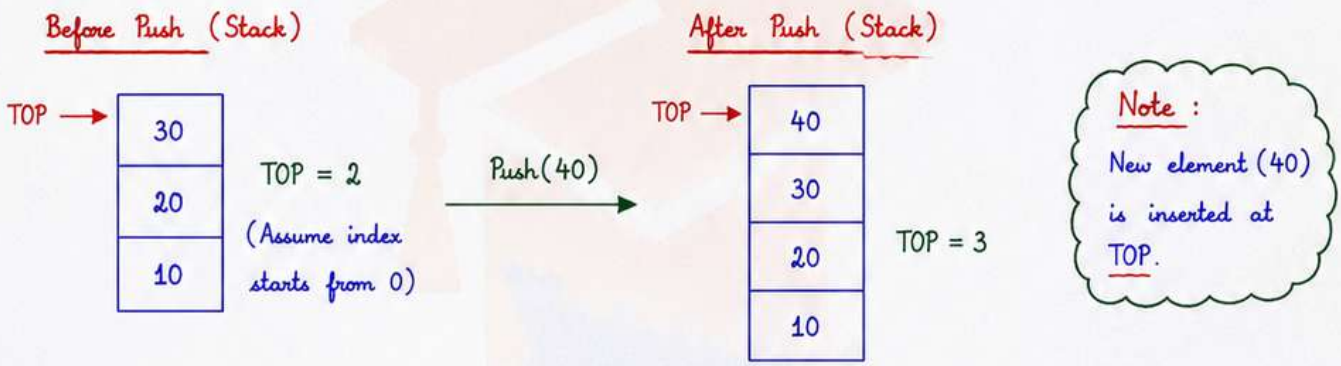
Step 1 : Check if the stack is full.

Step 2 : If full, display "Stack Overflow".

Step 3 : Otherwise, increment TOP.

Step 4 : Insert the element at TOP.

(A) Example of Push Operation :

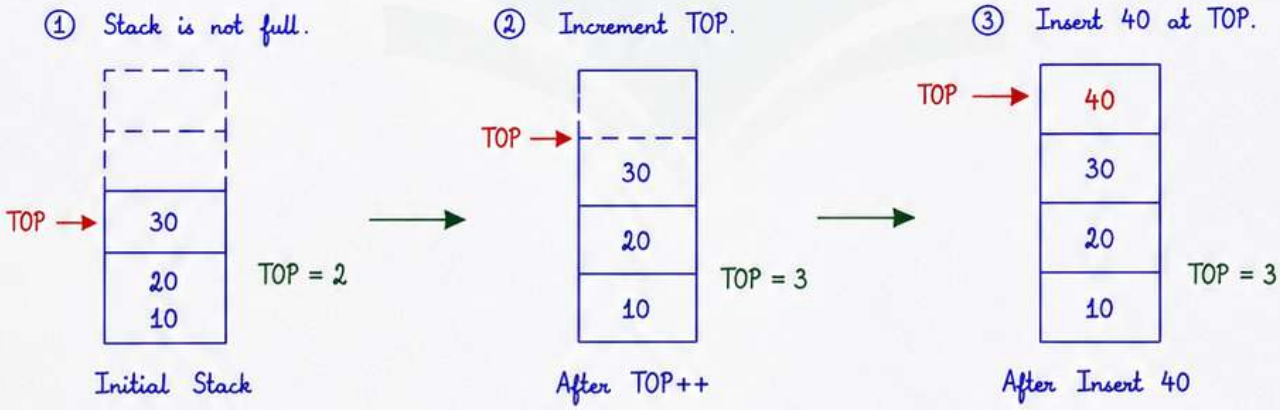


Note : New element (40) is inserted at TOP.

(B) Time Complexity :

Time Complexity of PUSH = O(1)

6. Push Operation – Detailed Illustration :



Important Point : PUSH always happens at TOP and increases the size of the stack by one (until the stack is full).

Apna College Bihar

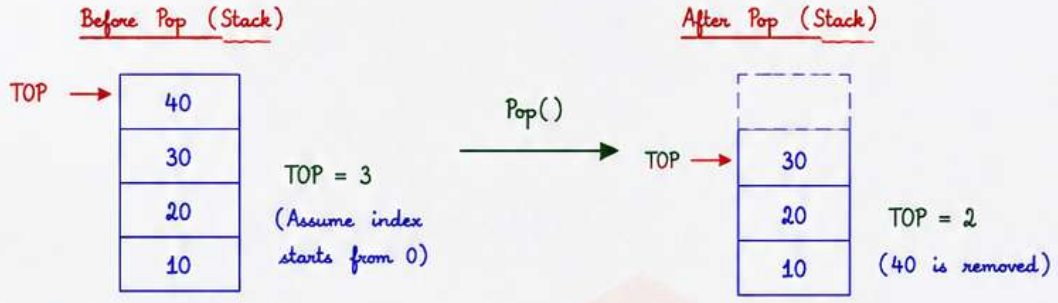
7. Pop Operation :

The POP operation removes the TOP element from the stack.

Pop Algorithm :

Step 1 : Check if the stack is empty.
Step 2 : If empty, display "Stack Underflow".
Step 3 : Otherwise, remove the top element.
Step 4 : Decrement TOP.

(A) Example of Pop Operation :



Note : Deleted element is always the element at TOP.

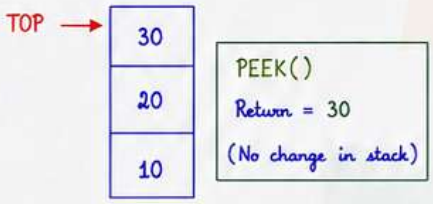
(B) Time Complexity :

Time Complexity of POP = O(1)

8. Peek (TOP) Operation :

The PEEK operation returns the topmost element without removing it.

Example :



Time Complexity :

O(1)

9. isEmpty() :

This operation checks whether the stack is empty or not.

Algorithm :

If TOP = -1
  Stack is Empty
Else
  Stack is Not Empty

Example :

If TOP = -1 ⇒ Empty
If TOP ≥ 0 ⇒ Not Empty

Time Complexity :

O(1)

10. isFull() (Array Implementation) :

This operation checks whether the stack is full or not.

Algorithm :

If TOP = MAX - 1
  Stack is Full
Else
  Stack is Not Full

Example :

If TOP = MAX - 1 ⇒ Full
If TOP < MAX - 1 ⇒ Not Full

Time Complexity :

O(1)

11. Size Operation :

This operation returns the number of elements currently present in the stack.

For Array Implementation :
Size = TOP + 1

Example :

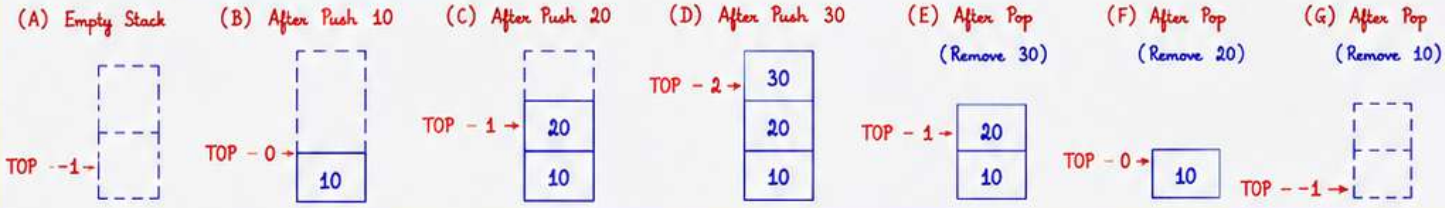


TOP = 2
Size = TOP + 1
= 2 + 1
= 3

Summary of this page :

- ★ Pop removes the top element.
- ★ Peek returns top element without removing.
- ★ isEmpty() checks empty condition.
- ★ isFull() checks full condition.
- ★ Size gives the number of elements.

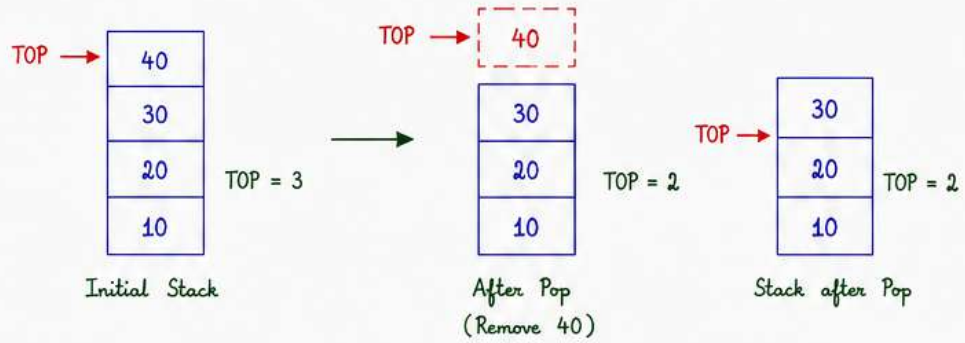
Small Stack Illustrations (Using TOP Pointer) :



Apna College Bihar

12. Pop Operation - Detailed Illustration :

① Stack is not empty.
② Remove the element at TOP.
③ Decrement TOP.
③ Decrement TOP.

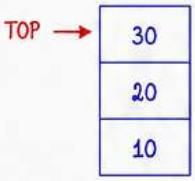


Initial Stack | After Pop (Remove 40) | Stack after Pop

Note :
Pop removes only the top element and decreases the size of the stack by one.

13. Peek (TOP) Operation :

The PEEK operation returns the topmost element without removing it.



PEEK()
Return = 30
(No change in stack)

Time Complexity : O(1)

14. isEmpty() Operation :

This operation checks whether the stack is empty or not.

Algorithm :
If TOP = -1
  Stack is Empty
Else
  Stack is Not Empty

Example :
If TOP = -1 ⇒ Empty
If TOP = 0 ⇒ Not Empty
If TOP = 2 ⇒ Not Empty

Time Complexity : O(1)

15. isFull() Operation

This operation checks whether the stack is full or not.

Algorithm (Array Implementation):
If TOP = MAX - 1
  Stack is Full
Else
  Stack is Not Full

Example :
If TOP = MAX - 1 ⇒ Full
If TOP < MAX - 1 ⇒ Not Full

Time Complexity : O(1)

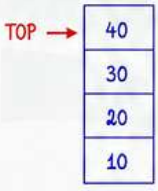
16. Size Operation :

This operation returns the number of elements currently present in the stack.

For Array Implementation :
Size = TOP + 1

Time Complexity : O(1)

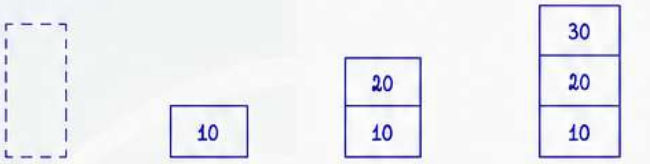
Example :



TOP = 3
Size = TOP + 1
= 3 + 1
= 4

17. Small Stack State Examples :

(A) Empty Stack (B) One Element (C) Two Elements (D) Three Elements



TOP = -1 | TOP = 0 | TOP = 1 | TOP = 2

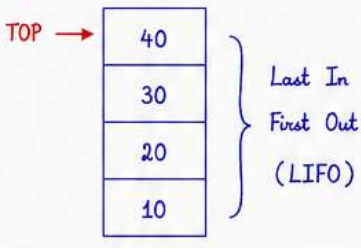
18. Complexity Analysis of Stack Operations :

Operation	Time Complexity
Push	O(1)
Pop	O(1)
Peek (Top)	O(1)
isEmpty()	O(1)
isFull()	O(1)
Size	O(1)

19. Applications of Stack :

- Function call management (Call Stack).
- Recursion.
- Expression evaluation (Infix, Prefix, Postfix).
- Parentheses matching.
- Browser back button.
- Undo / Redo operations.
- Syntax checking in compilers.
- Depth First Search (DFS).

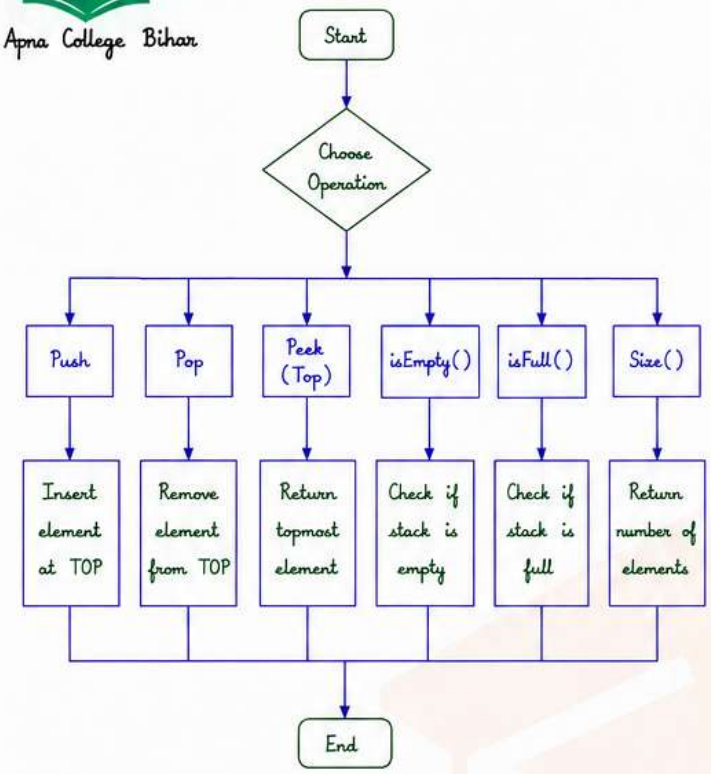
Complete Stack (LIFO) Diagram :



STACK

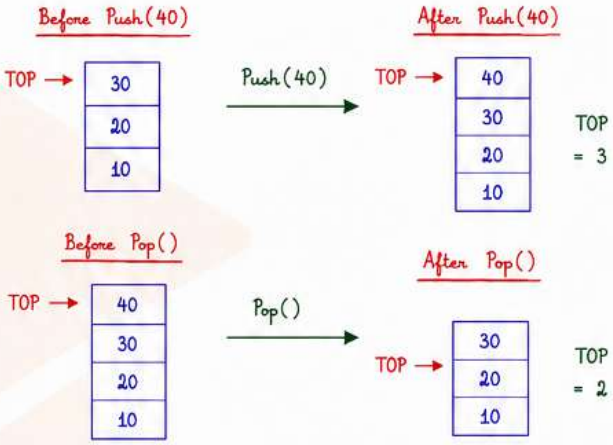
Apna College Bihar

20. Stack Operations - Flowchart :



21. Push vs Pop Comparison :

Feature	Push Operation	Pop Operation
Meaning	Insert element at TOP	Remove element from TOP
Type	Insertion	Deletion
Position	At TOP	At TOP
Effect on TOP	TOP is incremented	TOP is decremented
Check Condition	Check for Stack Overflow	Check for Stack Underflow
Time Complexity	O(1)	O(1)
Space Complexity	O(n)	O(n)
Example	Push(40)	Pop()



22. Important Points (Summary) :

- ★ Stack follows LIFO (Last In, First Out) principle.
- ★ Insertion operation is called PUSH.
- ★ Deletion operation is called POP.
- ★ All operations are performed at TOP.
- ★ TOP always points to the topmost element.
- ★ Push and Pop operations take O(1) time.
- ★ Stack can be implemented using Arrays or Linked Lists.
- ★ Stack Overflow occurs when stack is full.
- ★ Stack Underflow occurs when stack is empty.
- ★ Stack is widely used in function calls, recursion, expression evaluation, and browser history.

23. Common Errors in Stack Operations :

- ★ Pushing element when stack is full → Overflow.
- ★ Popping element when stack is empty → Underflow.
- ★ Accessing TOP when stack is empty.
- ★ Not updating TOP correctly after Push or Pop.
- ★ Not checking boundary conditions.
- ★ Incorrect implementation of isEmpty() and isFull().

24. Quick Revision Table :

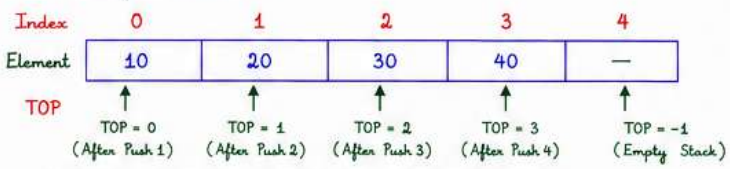
Operation	What it Does	Condition Check	Time Complexity
Push	Insert element at TOP	isFull()	O(1)
Pop	Remove element from TOP	isEmpty()	O(1)
Peek (Top)	Return topmost element	isEmpty()	O(1)
isEmpty()	Check if stack is empty	TOP = -1	O(1)
isFull()	Check if stack is full	TOP = MAX - 1	O(1)
Size()	Return number of elements	Size = TOP + 1	O(1)

25. Exam Points (Very Important) :

- ✓ Stack is a linear data structure.
- ✓ It follows LIFO (Last In, First Out).
- ✓ TOP is the only end used for all operations.
- ✓ Push inserts element at TOP.
- ✓ Pop removes element from TOP.
- ✓ Peek returns TOP element without removing.
- ✓ All basic operations take O(1) time.
- ✓ Stack can be implemented using Arrays or Linked Lists.
- ✓ Stack is used in recursion, expression evaluation, browser history, and syntax checking.

26. Memory View of Stack (Array Implementation) :

Example : MAX = 5



Note : When stack is empty, TOP = -1.
When stack is full, TOP = MAX - 1.

Apna College Bihar

Topic 2 : Applications of Stacks : Expression Conversion and Evaluation

1. Introduction :

A Stack is one of the most widely used data structure in computer science. One of its most important applications is Expression Conversion and Expression Evaluation. Mathematical expressions written in different notations can be converted and evaluated efficiently using a stack.

The three common expression notations are :

- ★ Infix
- ★ Prefix
- ★ Postfix

Note :
Stack simplifies the conversion between these notations and helps evaluate expressions without ambiguity.

2. Expression :

Definition : An expression is a combination of operands (variables or constants) and operators that represents a computation.

Examples :

- A + B
- (A + B) × C
- X − Y + Z

Components of an Expression :

- Operands : A, B, X, Y, Numbers
- Operators : +, −, ×, ÷, %, ^

3. Types of Expressions :

There are three types of expressions :

① Infix Expression ② Prefix Expression ③ Postfix Expression

4. Infix Expression :

Definition : In an infix expression, the operator is placed between two operands.

Examples :

- A + B
- A × B
- (A + B) × C

Characteristics :

- ★ Easy for humans to understand.
- ★ Parentheses are often required.
- ★ Most programming languages use infix notation.

Infix Representation



Operator is placed between two operands.

5. More Examples of Infix Expression :

Infix Expression	Meaning
A + B	A plus B
A − B + C	A minus B plus C
(A + B) × (C − D)	(A plus B) multiply (C minus D)
A + B × C	A plus (B multiply C) [due to precedence]

Important Point :
In infix expression, operator comes in between two operands. So, it is less suitable for computer evaluation.

Apna College Bihar

6. Prefix Expression (Polish Notation) :

In a prefix expression, the operator is written before its operands.

Examples :

① + A B

② * + A B C

Equivalent Infix :

+ A B → A + B

* + A B C → (A + B) × C

Advantages :

★ No parentheses required.
★ Faster evaluation using a stack.
★ Eliminates ambiguity.
★ Easy for computer processing.

Posfix Representation

Operator Operand Operand



Operator is placed before its operands.

7. Postfix Expression (Reverse Polish Notation) :

In a postfix expression, the operator is written after its operands.

Examples :

① A B +

② A B + C *

Equivalent Infix :

A B + → A + B

A B + C * → (A + B) × C

Advantages :

★ Parentheses are not needed.
★ Very easy for computers to evaluate.
★ Used in stack based calculators.
★ Suitable for compilers and interpreters.

Postfix Representation

Operand Operand Operator



Operator is placed after its operands.

8. Comparison of Expression Types :

Feature	Infix	Prefix	Postfix
Operator Position	Between operands	Before operands	After operands
Parentheses Required	Yes	No	No
Human Readability	Easy	Moderate	Moderate
Computer Evaluation	Difficult	Easy	Easy
Example	(A + B) × C	* + A B C	A B + C *

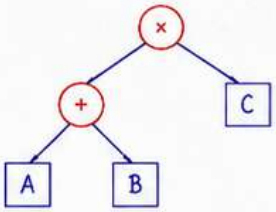
Expression Notations

✓ Infix → Operator between operands.
✓ Prefix → Operator before operands.
✓ Postfix → Operator after operands.

9. Visual Representation of the Three Notations :

1. Infix Expression

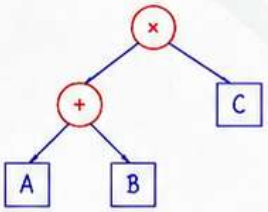
(A + B) × C



Operator between operands.

Prefix Expression

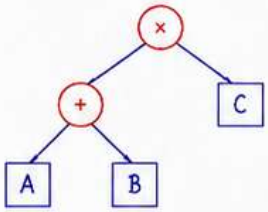
* + A B C



Operator before operands.

Postfix Expression

A B + C *



Operator after operands.

Legend :

- → Operator
- → Operand

Key Point : Prefix and Postfix expressions do not require parentheses, so they are more suitable for computer evaluation and are widely used in compilers and calculators.

Apna College Bihar

10. Expression Conversion :

A stack is used to convert expressions from one notation to another. This is very useful in compilers and expression parsers.

Note :
Among all conversions, Infix → Postfix is most commonly used because it is easy to evaluate.

Types of Conversion :

① Infix → Postfix
② Infix → Prefix
③ Prefix → Infix
④ Postfix → Infix
⑤ Prefix → Postfix
⑥ Postfix → Prefix

11. Infix to Postfix Conversion Algorithm :

① Scan the infix expression from left to right.
② If the symbol is an operand, print it to output.
③ If the symbol is '(', push it onto the stack.
④ If the symbol is ')', pop from stack and print until '(' is found. Discard both parentheses.
⑤ If the symbol is an operator, compare it with the top of the stack.
- If stack is empty or top is '(', push the operator.
- If precedence of incoming operator is greater than top operator, push it.
- If precedence is less than or equal to top operator, pop from stack and print. Repeat until above condition becomes true. Then push the operator.

⑥ After scanning complete expression, pop all remaining operators from stack and print.

Precedence and Associativity :

Operator	Precedence	Associativity
(	0	Left to Right
+ , -	1	Left to Right
* , / , %	2	Left to Right
^	3	Right to Left

Example : Convert (A + B) * C - D / E to Postfix

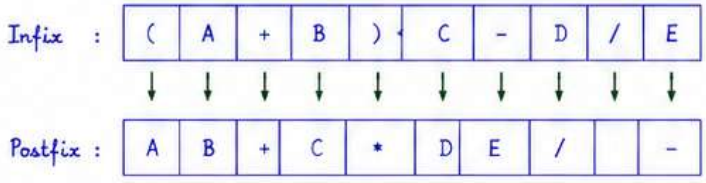
Infix : (A + B) * C - D / E
Postfix : A B + C * D E / -

Step-by-Step Conversion Using Stack :

Step	Scanned Symbol	Action	Stack (Top → Right)	Output
1	(	Push '('	(	
2	A	Operand → Print	(	A
3	+	Push '+'	(+	A
4	B	Operand → Print	(+	A B
5	)	Pop until '('		A B +
6	*	Push '*'	*	A B +
7	C	Operand → Print	*	A B + C
8	-	Pop '*' (because * > -) then Push '-'	-	A B + C *
9	D	Operand → Print	-	A B + C * D
10	/	Push '/' (because / > -)	- /	A B + C * D
11	E	Operand → Print	- /	A B + C * D E
12	End	Pop remaining operators		A B + C * D E / -

Important Points :
★ Always prefer Postfix for evaluation.
★ Stack helps in managing operators.
★ Parentheses control the order of operations.
★ Precedence and associativity are key to conversion.

Infix → Postfix Conversion Illustration :



12. Postfix Expression Evaluation using Stack :

A postfix expression can be evaluated easily using a stack.

Apna College Bihar

Algorithm :

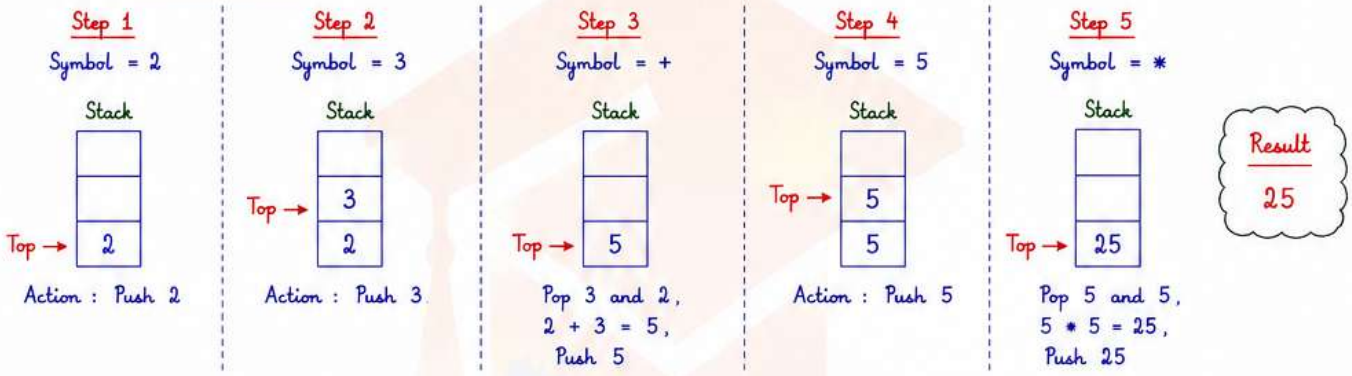
1. Scan the postfix expression from left to right.
2. If the symbol is an operand (operand), push it onto the stack.
3. If the symbol is an operator,
   - Pop the top two operands.
   - Perform the operation (first pop is Right operand, second pop is Left operand).
   - Push the result back onto the stack.
4. Repeat steps 2 and 3 until the expression is fully scanned.
5. The final value in the stack is the result.

Example : Evaluate the postfix expression `23+5*`

Step	Scanned Symbol	Stack (Bottom → Top)	Action / Explanation
1	2	[2]	Push operand 2
2	3	[2 , 3]	Push operand 3
3	+	[5]	Pop 3 and 2, 2+3 = 5, Push 5
4	5	[5 , 5]	Push operand 5
5	*	[25]	Pop 5 and 5, 5*5 = 25, Push 25

Final Result : 25

Detailed Stack Illustration :



13. Infix to Prefix Conversion Algorithm :

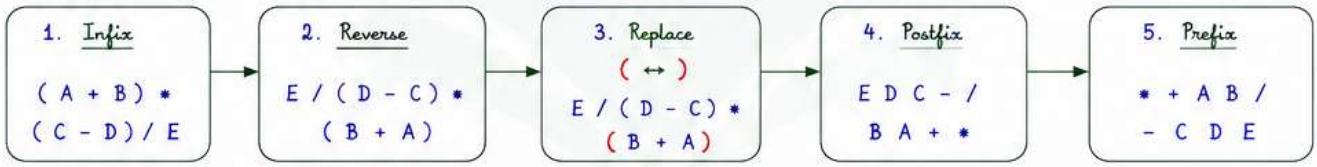
Algorithm :

1. Reverse the given infix expression.
2. Replace '(' with ')' and ')' with '('.
3. Convert the above expression to postfix.
4. Reverse the obtained postfix expression.
5. The resulting expression is the prefix expression.

Example : Convert `(A + B) * (C - D) / E` to Prefix.

Step	Expression
1. Infix Expression	(A + B) * (C - D) / E
2. Reverse the Expression	E / (D - C) * (B + A)
3. Replace '(' with ')' and ')' with '('	E / (D - C) * (B + A)
4. Convert to Postfix	E D C - / B A + *
5. Reverse Postfix (Prefix)	* + A B / - C D E

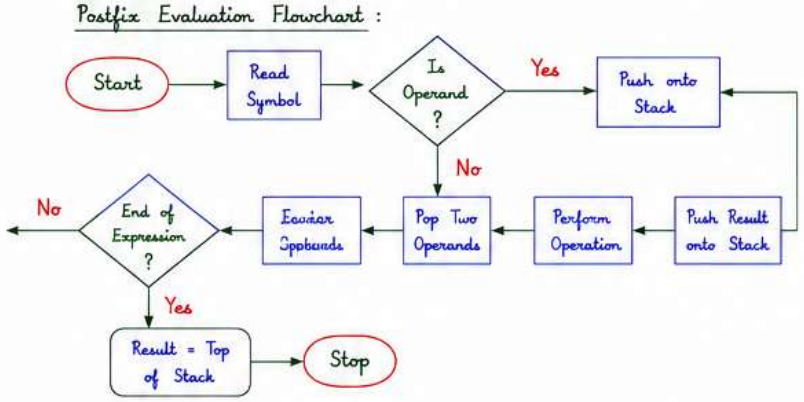
Infix → Prefix Conversion Illustration :



Important Points :

- ★ Postfix evaluation is easy and efficient.
- ★ Stack stores operands and intermediate results.
- ★ First pop is Right operand, Second pop is Left operand.
- ★ Infix to Prefix conversion uses reverse + replace + postfix + reverse.
- ★ Stack is widely used in compilers and calculators.

Postfix Evaluation Flowchart :



Apna College Bihar

14. Postfix Expression Evaluation Algorithm :

Algorithm :

1. Scan the postfix expression from left to right.
2. If the symbol is an operand, push it onto the stack.
3. If the symbol is an operator, pop the top two operands.
   - First pop is Right operand (let it be B).
   - Second pop is Left operand (let it be A).
   - Perform the operation A operator B.
   - Push the result back onto the stack.
4. Repeat steps 2 and 3 until the expression is fully scanned.
5. The final value in the stack is the result of the expression.

Example : Evaluate the postfix expression 23+5*

Expression : 2 3 + 5 *

Step-by-Step Evaluation Using Stack :

Step	Scanned Symbol	Stack (Bottom → Top)	Action / Explanation
1	2	[2]	Push 2
2	3	[2 , 3]	Push 3
3	+	[5]	Pop 3 and 2, 2+3 = 5 Push 5
4	5	[5 , 5]	Push 5
5	*	[25]	Pop 5 and 5, 5*5 = 25 Push 25

Final Result (Top of Stack) : 25

Detailed Stack Illustration :

Step 1
Symbol = 2
Stack
Top → 2
Action : Push 2

Step 2
Symbol = 3
Stack
Top → 3
2
Action : Push 3

Step 3
Symbol = +
Stack
5
Action : Pop 3 and 2, 2 + 3 = 5, Push 5

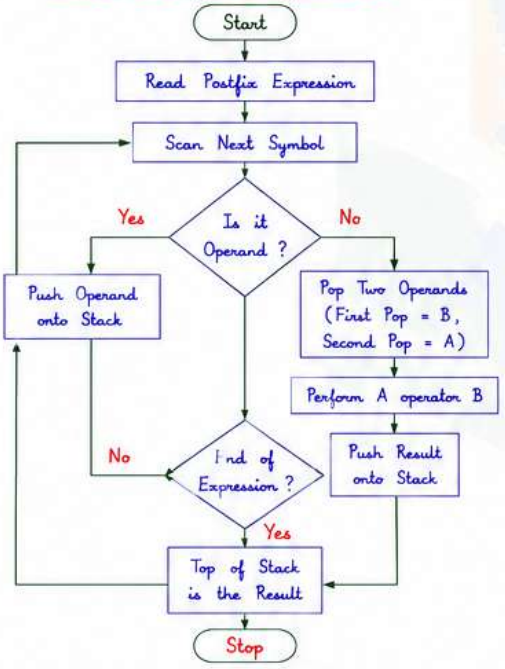
Step 4
Symbol = 5
Stack
Top → 5
5
Action : Push 5

Step 5
Symbol = *
Stack
Top → 25
Action : Pop 5 and 5, 5 * 5 = 25, Push 25

Important Note :

- ★ In postfix evaluation, operands are pushed and operators are applied.
- ★ Order of popping is important.
- ★ First pop is Right operand.
- ★ Second pop is Left operand.

15. Postfix Evaluation Flowchart :



16. Applications of Expression Evaluation :

- ★ Used in calculators to evaluate arithmetic expressions.
- ★ Used in compilers for expression parsing and code generation.
- ★ Used in mathematical software and scientific calculators.
- ★ Used in interpreters and assemblers.
- ★ Used in data processing and formula evaluation.
- ★ Used in postfix based languages and scripting.

17. Complete Expression Conversion & Evaluation Flow :

Mathematical Expression (Infix) → Convert to Postfix using Stack → Evaluate Postfix using Stack → Result (Top of Stack)

18. Quick Revision Table :

Notation	Operator Position	Parentheses Required	Easy for Humans	Easy for Computers	Used For
Infix	Between Operands	Yes	Yes	No	Writing Expressions
Prefix	Before Operands	No	Moderate	Yes	Compilers, Prefix Evaluation
Postfix	After Operands	No	Moderate	Yes	Calculators, Stack Evaluation

19. Exam Points (Most Important) :

- ★ Stack is widely used for expression conversion and evaluation.
- ★ Three notations : Infix, Prefix, Postfix.
- ★ Infix → operator between operands.
- ★ Prefix → operator before operands.
- ★ Postfix → operator after operands.
- ★ Prefix and Postfix do not require parentheses.
- ★ Infix is easy for humans but difficult for computers.
- ★ Stack is used for conversion between notations.
- ★ Infix → Postfix is the most commonly used conversion.
- ★ Postfix evaluation uses stack : push operands, apply operators.
- ★ In evaluation, first pop is Right operand, second pop is Left.
- ★ Final result of postfix evaluation is at the top of stack.
- ★ Expression conversion and evaluation are important applications of stacks in real world.

Apna College Bihar

Topic 3 : Corresponding Algorithms and Complexity Analysis (Stack)

1. Introduction :

A Stack is an Abstract Data Type (ADT) that follows the LIFO (Last In, First Out) principle. Various algorithms are used to perform stack operations efficiently. Algorithm analysis helps us determine the execution time (Time Complexity) and memory usage (Space Complexity) of these operations.

Since all stack operations are performed only at the TOP, most of them require constant time $O(1)$.

Note :

All basic stack operations are performed only at the TOP. Therefore, these operations are very fast and efficient.

2. What is an Algorithm ?

An Algorithm is a finite sequence of well-defined steps used to solve a problem or perform a specific task.

3. Characteristics of an Algorithm :

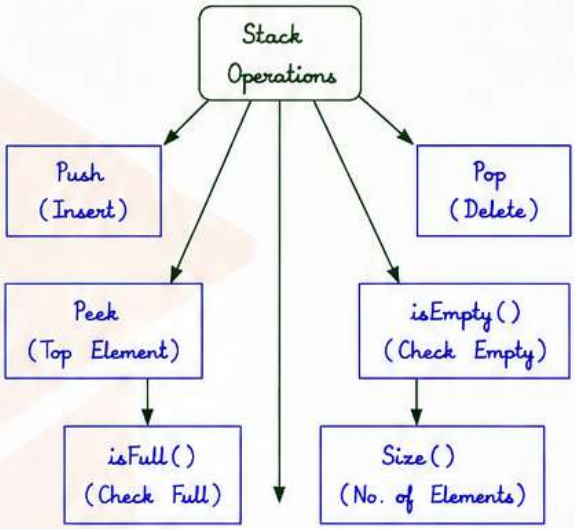
1. Well-defined steps
2. Finite number of instructions
3. Unambiguous
4. Efficient
5. Produces correct output

4. Stack Algorithms :

The main stack algorithms are :

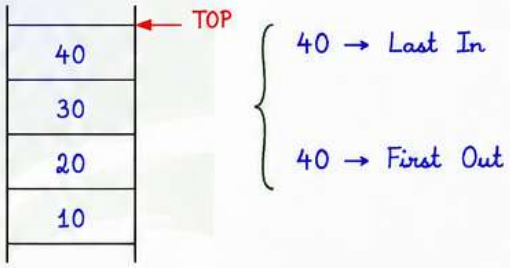
1. Push Algorithm
2. Pop Algorithm
3. Peek Algorithm
4. isEmpty() Algorithm
5. isFull() Algorithm
6. Size() Algorithm

5. Stack Operations Overview :



6. Stack (LIFO) Concept :

Stack follows the LIFO (Last In, First Out) principle.



Elements are inserted and removed only from the TOP.

Key Idea :

- ★ All stack operations are performed only at the TOP.
- ★ Most stack operations take constant time $O(1)$.
- ★ Stack algorithms are simple, fast and efficient.
- ★ These algorithms are widely used in many applications.

Apna College Bihar

7. isEmpty() Algorithm :

Algorithm :

If TOP == -1
    Stack is Empty
Else
    Stack is Not Empty.

Time Complexity → O(1)
Space Complexity → O(1)

8. isFull() Algorithm (Array Implementation)

Algorithm :

If TOP == MAX - 1
    Stack is Full
Else
    Stack is Not Full.

Time Complexity → O(1)
Space Complexity → O(1)

9. Size() Algorithm :

Algorithm :

Size = TOP + 1
Return Size.

Time Complexity → O(1)
Space Complexity → O(1)

10. Time Complexity Analysis :

Operation	Best Case	Average Case	Worst Case
Push	O(1)	O(1)	O(1)
Pop	O(1)	O(1)	O(1)
Peek	O(1)	O(1)	O(1)
isEmpty()	O(1)	O(1)	O(1)
isFull()	O(1)	O(1)	O(1)
Size()	O(1)	O(1)	O(1)

11. Space Complexity Analysis :

All basic stack operations require only a few temporary variables.

Therefore,

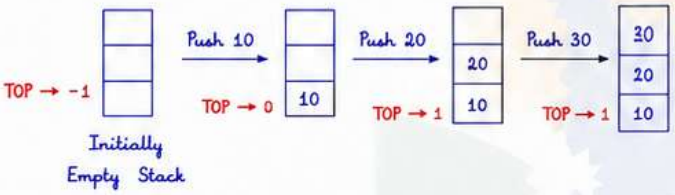
Space Complexity = O(1)

If the stack stores n elements,

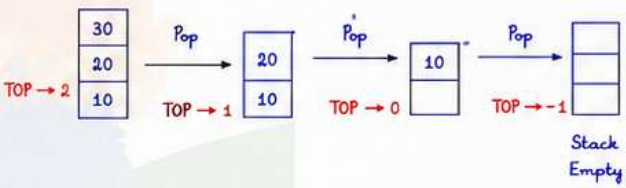
Overall Stack Memory = O(n)

12. Illustration of Stack Operations :

(a) Push Operation

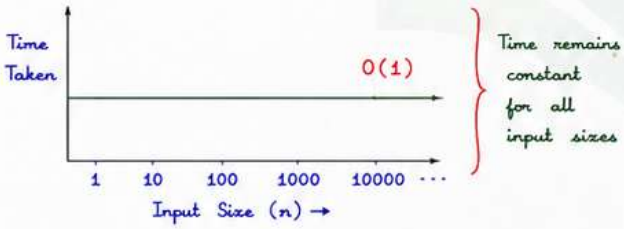


(b) Pop Operation



13. Concept of O(1) (Constant Time)

In O(1) time complexity, the time taken by an operation remains constant, no matter how large the input size (n) is.



14. Advantages :

- ★ Simple implementation.
- ★ Operations take constant time O(1).
- ★ Efficient memory access.
- ★ Fast insertion and deletion.
- ★ Useful in recursion.
- ★ Easy expression evaluation.
- ★ Used in compiler design.
- ★ Supports Depth First Search (DFS).

15. Limitations :

- ★ Fixed size in array implementation.
- ★ Stack Overflow may occur.
- ★ Stack Underflow may occur.
- ★ Only the top element is directly accessible.
- ★ Random access is not possible.

16. Applications :

- ★ Function Call Stack
- ★ Recursion
- ★ Expression Evaluation
- ★ Expression Conversion
- ★ Browser Back Button
- ★ Undo / Redo Operations
- ★ Parentheses Matching
- ★ Compiler Design
- ★ Depth First Search (DFS)
- ★ Backtracking Algorithms

Important Points (Summary)

- ★ Stack follows LIFO (Last In, First Out).
- ★ All basic operations take O(1) time.
- ★ Stack algorithms are simple and efficient.
- ★ These algorithms are widely used in many real-life applications.

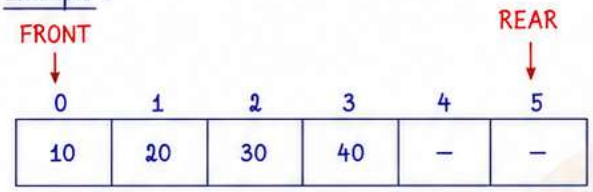
Apna College Bihar

ADT Queue, Types of Queue : Simple Queue, Circular Queue, Priority Queue

4. Queue Representation :

A queue is represented using an array with two pointers FRONT and REAR.

Example :



FIFO Order :
Insert (Enqueue) → 10 → 20 → 30 → 40
Remove (Dequeue) ← 10 ← 20 ← 30 ← 40

5. Simple Queue :

A simple queue is the basic form of queue in which insertion is performed at REAR and deletion at FRONT.

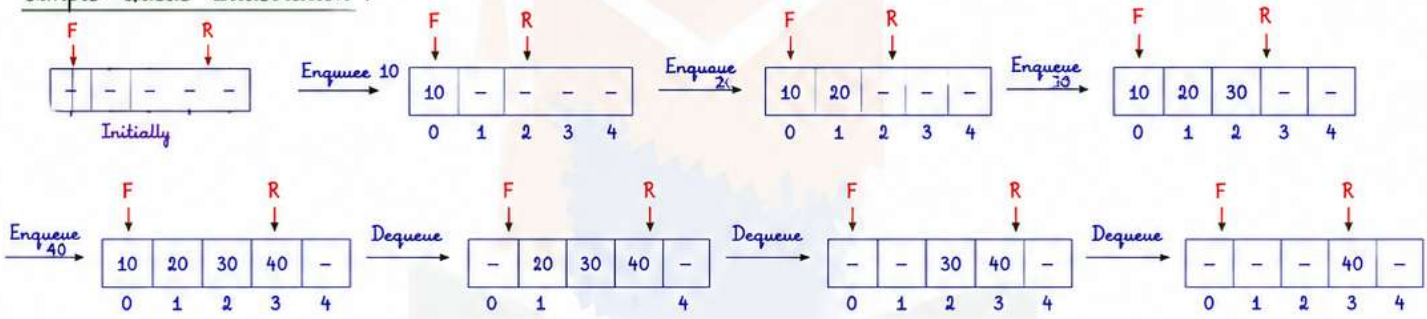
Advantages :

- ★ Easy to understand and implement.
- ★ Efficient insertion and deletion.
- ★ Suitable for small applications.

Limitations :

- ★ Wastage of memory after repeated deletions.
- ★ Queue may become full even when empty spaces exist at the beginning.

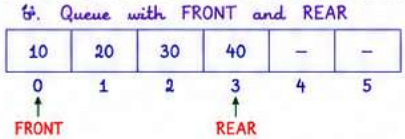
Simple Queue Illustration :



6. Queue Terminology :

- ★ FRONT : The pointer which points to the front element (deletion takes place here).
- ★ REAR : The pointer which points to the last element (insertion takes place here).
- ★ Empty Queue : When FRONT = -1.
- ★ Full Queue : When REAR = MAX - 1. (in array implementation).

Q. Queue with FRONT and REAR



Elements are removed from FRONT
Elements are inserted at REAR

7. Basic Queue Operations :

1. Enqueue (Insert)
2. Dequeue (Delete)
3. Peek / Front (View Front Element)
4. isEmpty() (Check Empty)
5. isFull() (Check Full)
6. Size() (Number of Elements)

Key Points :

- ★ Queue follows FIFO (First In, First Out).
- ★ Enqueue is done at REAR.
- ★ Dequeue is done from FRONT.
- ★ FRONT and REAR are the two main pointers.
- ★ Queue can be implemented using Array or Linked List.

Apna College Bihar

ADT Queue, Types of Queue : Simple Queue, Circular Queue, Priority Queue

6. Circular Queue :

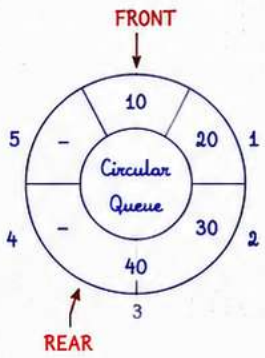
A circular queue is a modified form of queue in which the last position is connected to the first position, forming a circle.

When REAR reaches the last position, it wraps around to the first empty position.

Advantages :

- ★ Better memory utilization.
- ★ Eliminates memory wastage.
- ★ Efficient insertion and deletion.
- ★ Suitable for buffering and real time applications.

Circular Queue Representation :



Here : FRONT = 0, REAR = 3

Working of Circular Queue :

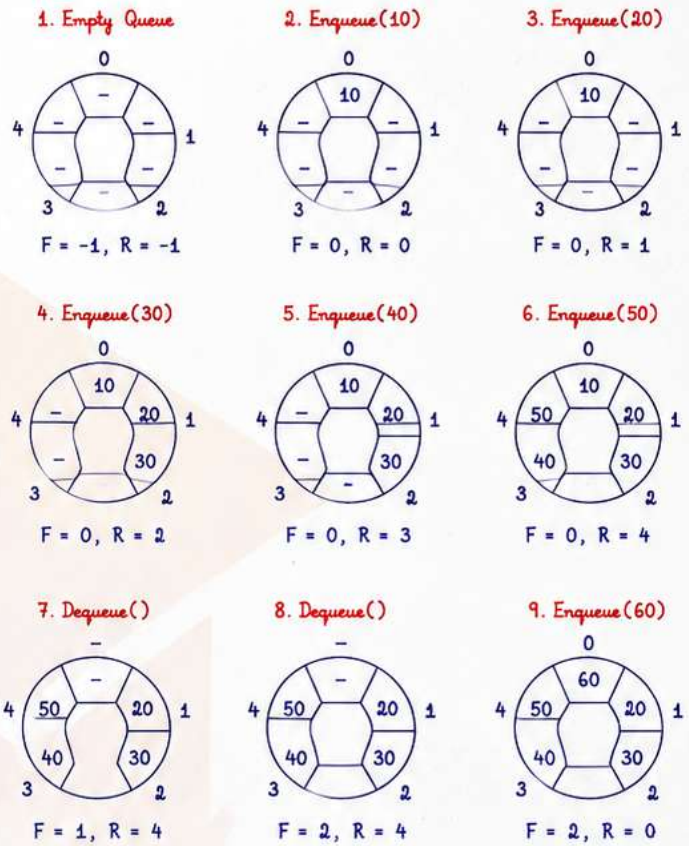
① Initially, FRONT = -1, REAR = -1 (Empty Queue)
② When first element is inserted : FRONT = REAR = 0
③ When elements are inserted, REAR moves forward.
④ When REAR reaches last position (MAX -1), it moves to 0.
⑤ When elements are deleted, FRONT moves forward.
⑥ When FRONT > REAR, still queue is not empty until all elements are deleted.

Condition for Circular Queue (Array Implementation)

Queue is Full : (REAR + 1) % MAX == FRONT

Queue is Empty : FRONT == -1

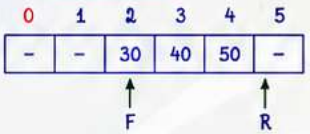
Circular Queue Example (MAX = 5)



Note :
When REAR is at last position and we want to insert more elements, it comes to the first position (0). This is called Wrap-around.

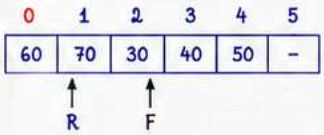
Memory Utilization Comparison :

Simple Queue



Here, positions 0 and 1 are empty but cannot be used. (Wastage)

Circular Queue



Here, empty spaces are utilized by wrap-around. (No Wastage)

Important Points :

- ★ Circular queue reduces memory wastage.
- ★ It is mostly used in buffering, real-time systems, CPU scheduling.
- ★ In circular queue, the positions are used in circular manner.
- ★ It requires both FRONT and REAR pointers.



Apna College Bihar

7. Priority Queue :

A priority queue is a queue in which each element has a priority. The element with higher priority is deleted first. If two elements have same priority, then FIFO order is followed.

Types :

- ★ Ascending Priority Queue (Smaller number = Higher priority)
- ★ Descending Priority Queue (Larger number = Higher priority)

Example (Ascending Priority Queue) :

Element	Priority	Insertion Order
A	3	1st
B	1	2nd
C	2	3rd
D	2	4th

Removal Order

B(1) → C(2) → D(2) → A(3)

(Smaller number = Higher Priority)

Priority Queue Illustration :

P=3	P=1	P=2	P=2
A	B	C	D

Deletion → B → C → D → A

8. Comparison of Queue Types :

Feature	Simple Queue	Circular Queue	Priority Queue
Principle	FIFO	FIFO	Priority Based
Insertion	At REAR	At REAR	As per Priority
Deletion	From FRONT	From FRONT	Highest Priority First
Memory Utilization	Low	High	Medium
Memory Wastage	Yes	No	No
Priority Supported	No	No	Yes
Implementation	Easy	Moderate	Moderate
Best Use	Small Applications	Buffer, Real-time Systems	Scheduling, Processing

9. Priority Queue Types (By Order) :

① Ascending Priority Queue → Smaller number = Higher priority

② Descending Priority Queue → Larger number = Higher priority

Priority Queue Example (Descending Priority Queue) :

Element	Priority	Insertion Order
A	10	1st
B	40	2nd
C	30	3rd
D	20	4th

Removal Order

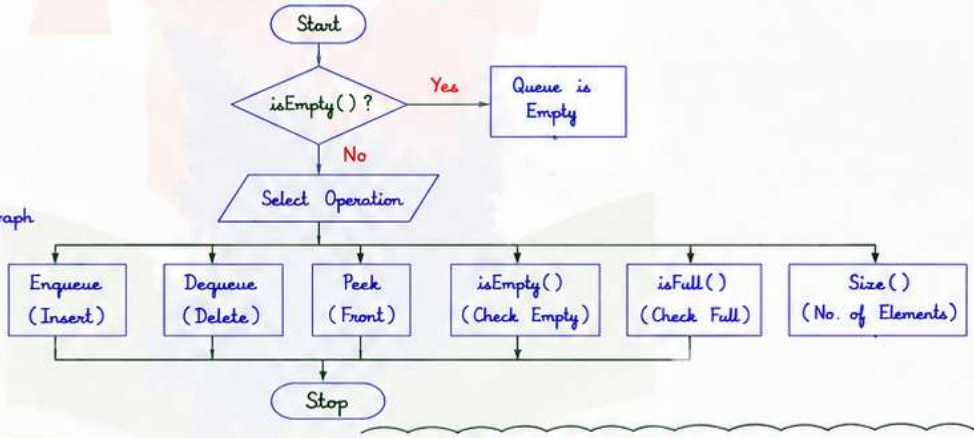
B(40) → C(30) → D(20) → A(10)

(Larger number = Higher Priority)

10. Applications of Queue :

- ★ CPU Scheduling (Ready Queue)
- ★ Printer Queue (Print Spooling)
- ★ Call Center Management
- ★ Ticket Reservation Systems
- ★ Network Packet Scheduling
- ★ Breadth First Search (BFS) in Graph
- ★ Keyboard Buffer
- ★ Message Queue in Systems
- ★ Shared Resource Management

11. Queue Operations Flowchart :



12. FIFO vs Priority Queue :

Feature	FIFO Queue	Priority Queue
Order	First In, First Out	Higher Priority First
Processing	By Arrival Time	By Priority
Priority	Not Considered	Considered
Use	Normal Scheduling	Priority Based Scheduling
Example	Printer Queue	CPU Priority Scheduling

Important Points (Summary) :

- ★ Queue follows FIFO (First In, First Out) principle.
- ★ Insertion is done at REAR, Deletion is done at FRONT.
- ★ Simple Queue may suffer from memory wastage.
- ★ Circular Queue overcomes memory wastage using wrap-around.
- ★ Priority Queue processes elements based on their priority.
- ★ Queue ADT is widely used in scheduling, buffering, and real-time systems.
- ★ Basic queue operations are Enqueue, Dequeue, Peek, isEmpty(), isFull(), Size().

Apna College Bihar

Operations on Each Type of Queue : Algorithms and Their Analysis

Introduction :

A Queue is a linear data structure that follows FIFO (First In, First Out) principle. Different types of queues perform similar operations, but their algorithms differ slightly.

Basic Operations on Queue :

1. Enqueue (Insertion)
2. Dequeue (Deletion)
3. Peek (Front)
4. isEmpty()
5. isFull()
6. Size()

Terminology :

- ★ FRONT : Points to the front element (deletion end).
- ★ REAR : Points to the last element (insertion end).
- ★ Empty Queue : FRONT = -1
- ★ Full Queue (Array) : REAR = MAX - 1

1. Enqueue Operation (Insertion) :

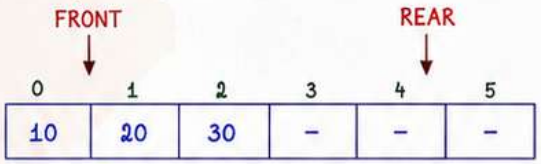
Definition : Enqueue inserts a new element at the REAR end of the queue.

Algorithm (Simple Queue) :

Step 1 : Check whether the queue is full.
Step 2 : If full, display "Queue Overflow".
Step 3 : If queue is empty, set FRONT = 0.
Step 4 : Increment REAR by 1.
Step 5 : Insert the new element at REAR.
Step 6 : Stop.

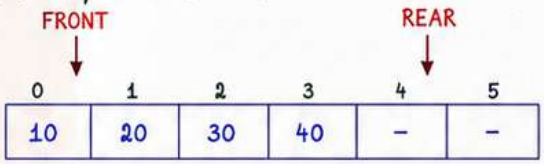
Example :

(a) Before Enqueue



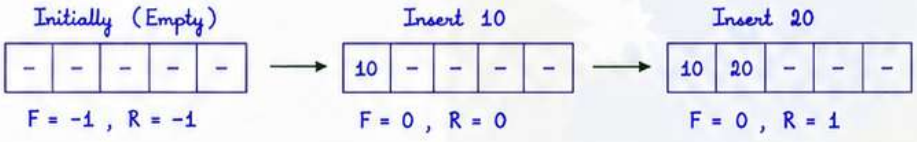
FRONT = 0 , REAR = 2

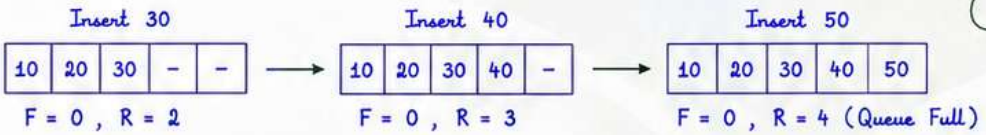
(b) Enqueue (40)



FRONT = 0 , REAR = 3

Insertion Process Illustration :





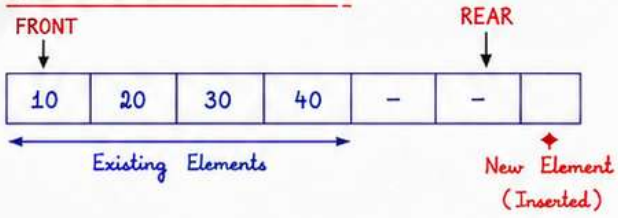
Time & Space Complexity (Enqueue) :

- ★ Time Complexity : O(1)
- ★ Space Complexity : O(1)

Note :

- ★ In simple queue, insertion is always done at REAR.
- ★ If the queue is full, no more elements can be inserted.
- ★ If queue is empty (FRONT = -1), then first element is inserted at index 0.

Queue Insertion at REAR :



Apna College Bihar

2. Dequeue Operation (Deletion) :

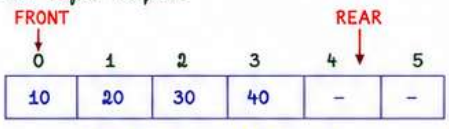
Definition : Dequeue removes the element from the FRONT end of the queue.

Algorithm (Simple Queue) :

Step 1 : Check whether the queue is empty.
Step 2 : If empty, display "Queue Underflow".
Step 3 : Remove the element at FRONT.
Step 4 : Increment FRONT by 1.
Step 5 : If FRONT > REAR,
reset both FRONT and REAR to -1.
Step 6 : Stop.

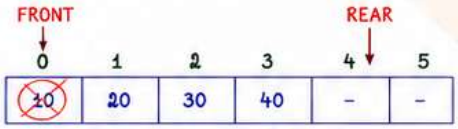
Example :

(a) Before Dequeue



FRONT = 0 , REAR = 3

(b) Dequeue (Remove 10)



FRONT = 1 , REAR = 3

(c) Dequeue (Remove 20)



FRONT = 2 , REAR = 3

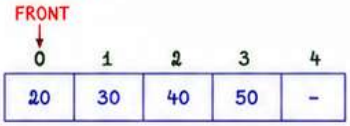
3. Peek (Front) Operation :

Definition : Peek returns the element at FRONT without removing it.

Algorithm :

Step 1 : Check whether queue is empty.
Step 2 : If empty, display "Queue Empty".
Step 3 : Return the element at FRONT.

Example :



Peek (Front) = 20

Time & Space Complexity :

★ Time : O(1)
★ Space : O(1)

4. isEmpty() Operation :

Algorithm :

```
If FRONT == -1
    Queue is Empty
Else
    Queue is Not Empty
```

Example :

- FRONT = -1 → Queue is Empty
- FRONT = 1 → Queue is Not Empty

Time Complexity : O(1)
Space Complexity : O(1)

5. isFull() Operation (Array Implementation) :

Algorithm :

```
If REAR == MAX - 1
    Queue is Full
Else
    Queue is Not Full
```

Example :

MAX = 6 (index 0 to 5)

If REAR = 5 → Queue is Full

If REAR < 5 → Queue is Not Full

Time Complexity : O(1)
Space Complexity : O(1)

6. Size() Operation :

Definition : Returns the total number of elements present in the queue.

Formula :

Size = REAR - FRONT + 1

Example :

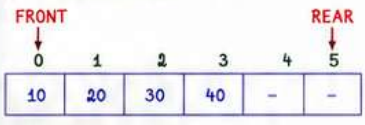
FRONT = 1 , REAR = 4

Size = 4 - 1 + 1
= 4

Time Complexity : O(1)
Space Complexity : O(1)

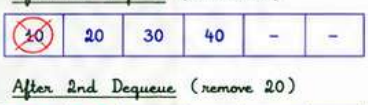
7. Dequeue Example (Multiple Removals) :

Initial Queue



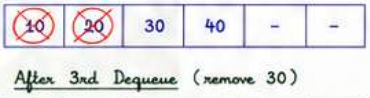
F = 0, R = 3

After 1st Dequeue (remove 10)



F = 1, R = 3

After 2nd Dequeue (remove 20)



F = 2, R = 3

After 3rd Dequeue (remove 30)



F = 3, R = 3

After 4th Dequeue (remove 40)



F = 4 > R = 3
Reset F = -1, R = -1
(Queue Empty)

Note :

★ In simple queue, after many dequeues, memory at front side is wasted.
★ This problem is solved in Circular Queue using wrap-around technique.
★ All basic operations (except priority queue insertion/deletion) take O(1) time.

Apna College Bihar

8. Circular Queue Operations :

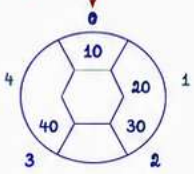
In circular queue, last position is connected to first position. After reaching the end, REAR moves to the beginning.

(A) Enqueue (Insertion) :

Algorithm :

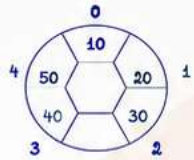
Step 1 : Check whether queue is full.
Step 2 : If full, display "Queue Overflow".
Step 3 : If empty, set FRONT = 0.
Step 4 : REAR = (REAR + 1) % MAX
Step 5 : Insert element at REAR.
Step 6 : Stop.

Example (MAX = 5)



FRONT = 0 , REAR = 3

Enqueue(50)

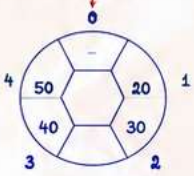


FRONT = 0 , REAR = 4

(B) Dequeue (Deletion) :

Step 1 : Check whether queue is empty.
Step 2 : If empty, display "Queue Underflow".
Step 3 : Remove element at FRONT.
Step 4 : If FRONT == REAR, reset both to -1.
Step 5 : Else FRONT = (FRONT + 1) % MAX
Step 6 : Stop.

Example (Remove 10)



FRONT = 1 , REAR = 4

9. Priority Queue Operations :

Elements are inserted according to their priority.
Higher priority element is deleted first.

Types :

★ Ascending Priority Queue (Lower number = Higher priority)
★ Descending Priority Queue (Higher number = Higher priority)

(A) Insertion :

★ Insert element in the queue at proper position according to priority.
★ Time Complexity : O(n)

(B) Deletion :

★ Remove element with highest priority.
★ Time Complexity : O(n)

Example (Descending Priority Queue)

Element	A	B	C	D	E
Priority	4	2	5	1	3

Insertion Order : A , B , C , D , E

Queue after each insertion (Highest priority first) :

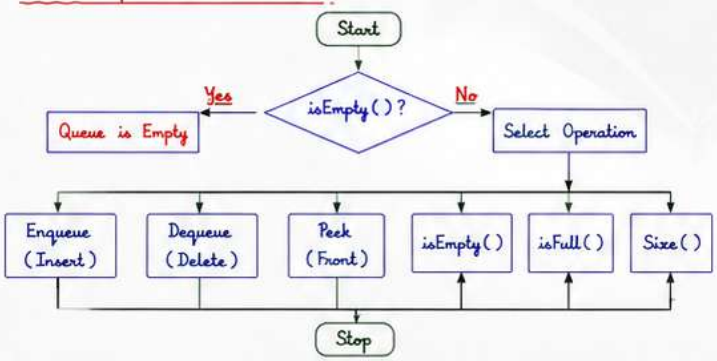
Insert A(4) : [A(4)]
Insert B(2) : [A(4) B(2)]
Insert C(5) : [C(5) A(4) B(2)]
Insert D(1) : [C(5) A(4) B(2) D(1)]
Insert E(3) : [C(5) A(4) E(3) B(2) D(1)]

Deletion Order : C(5) → A(4) → E(3) → B(2) → D(1)

10. Complexity Analysis Table :

Operation	Simple Queue	Circular Queue	Priority Queue
Enqueue (Insertion)	O(1)	O(1)	O(n)
Dequeue (Deletion)	O(1)	O(1)	O(n)
Peek (Front)	O(1)	O(1)	O(1)
isEmpty()	O(1)	O(1)	O(1)
isFull()	O(1)	O(1)	O(1)
Size()	O(1)	O(1)	O(1)

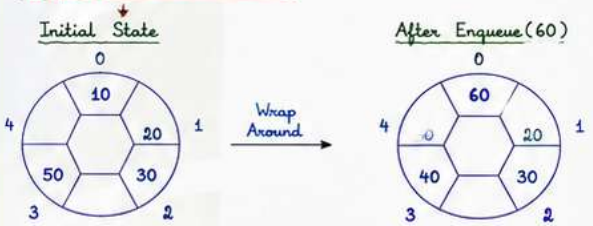
Queue Operations Flowchart :



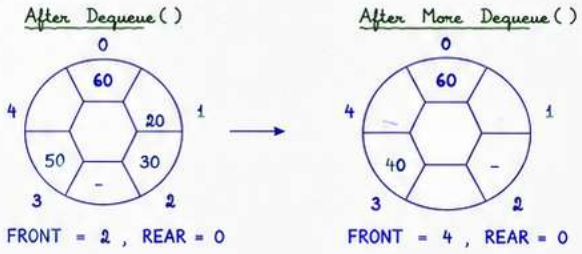
Circular Queue Conditions :

★ Empty Condition : FRONT == -1
★ Full Condition : (REAR + 1) % MAX == FRONT

Wrap-Around Illustration :



FRONT = 1
REAR = 4



FRONT = 2 , REAR = 0
FRONT = 4 , REAR = 0

Apna College Bihar

11. Priority Queue Operations :

In priority queue, elements are served based on their priority. Higher priority element is removed first.

(A) Insertion :

- ★ Insert the new element at the end of queue.
- ★ Compare its priority with previous elements and shift elements to maintain order.
- ★ Time Complexity : O(n)

Example :

(Priority ↑ means Higher number = Higher priority)

Insert	Priority	Queue After Insertion (Front → Rear)
10	2	10(2)
40	5	40(5) 10(2)
30	3	40(5) 30(3) 10(2)
50	1	40(5) 30(3) 10(2) 50(1)

(B) Deletion :

- ★ Remove the element at FRONT (highest priority).
- ★ Time Complexity : O(n)

Example :

Queue Before Deletion	Deleted Element	Queue After Deletion
40(5) 30(3) 10(2) 50(1)	40(5)	30(3) 10(2) 50(1)
30(3) 10(2) 50(1)	30(3)	10(2) 50(1)
10(2) 50(1)	10(2)	50(1)
50(1)	50(1)	Empty

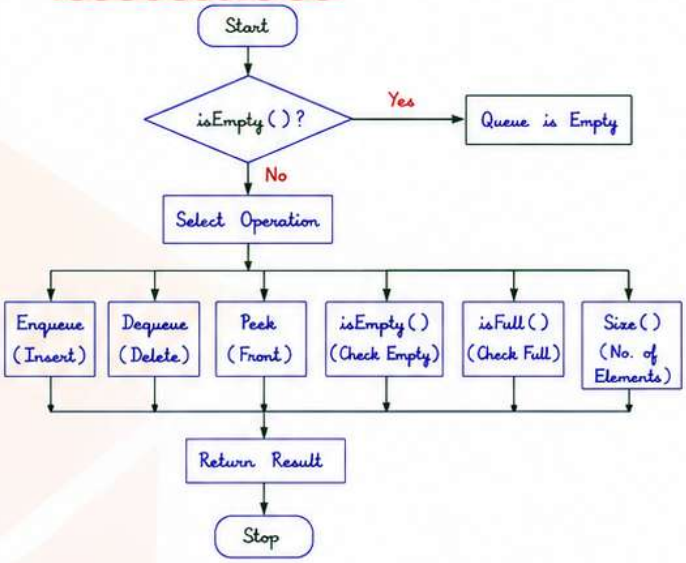
(C) Peek (Front) :

- ★ Return the highest priority element.
- ★ Time Complexity : O(1)

12. Complexity Analysis Table :

Operation	Simple Queue	Circular Queue	Priority Queue
Enqueue (Insertion)	O(1)	O(1)	O(n)
Dequeue (Deletion)	O(1)	O(1)	O(n)
Peek (Front)	O(1)	O(1)	O(1)
isEmpty( )	O(1)	O(1)	O(1)
isFull( )	O(1)	O(1)	O(1)
Size( )	O(1)	O(1)	O(1)

13. Queue Operations Flowchart :



14. Comparison of Queue Types :

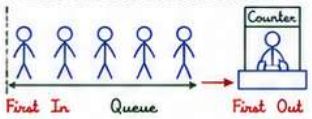
Feature	Simple Queue	Circular Queue	Priority Queue
Structure	Linear	Circular	Linear (Sorted)
Insertion	At REAR	At REAR (Circular)	At Proper Position (According to Priority)
Deletion	From FRONT	From FRONT (Circular)	Highest Priority Element (From FRONT)
Memory Utilization	Low (Wastage)	High (No Wastage)	Low to Medium
Implementation	Easy	Moderate	Difficult (Sorting Required)
Best Use	Simple Applications	Buffer, Real-time Systems	Scheduling, Processing with Priority

15. Applications of Queue :

- ★ CPU Scheduling (Ready Queue)
- ★ Printer Spooling System
- ★ Call Center Management
- ★ Breadth First Search (BFS) in Graph
- ★ Network Packet Scheduling
- ★ Message Queue in Systems
- ★ Shared Resource Management
- ★ Simulation of Real-life Scenarios (e.g., Traffic Systems, Banking Tokens)

Real-life Example :

In a ticket counter, the first person who comes (first inserted) gets the ticket first (first out), which is exactly how a queue works.



Important Points (Summary) :

- ★ Queue follows FIFO (First In, First Out) principle.
- ★ Enqueue inserts element at REAR.
- ★ Dequeue removes element from FRONT.
- ★ Simple Queue may waste memory space.
- ★ Circular Queue uses wrap-around to utilize all spaces efficiently.
- ★ Priority Queue serves elements based on priority.
- ★ All operations in Simple and Circular Queue are O(1).
- ★ Priority Queue insertion and deletion take O(n).
- ★ Queues are widely used in operating systems, networks, scheduling and buffering.

Apna College Bihar

Unit 3.0 : Linked Lists

Topic 1 : Singly Linked Lists : Representation in Memory

Introduction :

A Linked List is a dynamic linear data structure in which elements are stored in separate memory locations and connected using pointers (links). Unlike arrays, linked lists do not require contiguous memory allocation. The first and most basic type of linked list is the Singly Linked List (SLL).

A Singly Linked List consists of a sequence of nodes. Each node contains two parts :

① Data Field - stores the actual information.

② Link (Pointer) Field - stores the address of the next node.

The last node points to NULL, indicating the end of the list.

1. What is a Singly Linked List ?

Definition : A Singly Linked List is a linear data structure in which every node contains data and a pointer to the next node. Traversal is possible only in the forward direction.

Characteristics :

- ★ Dynamic size.
- ★ Non-contiguous memory allocation.
- ★ Efficient insertion and deletion.
- ★ Sequential access only.
- ★ Uses pointers for linking nodes.
- ★ Last node points to NULL.

2. Structure of a Node :

Each node has two fields :

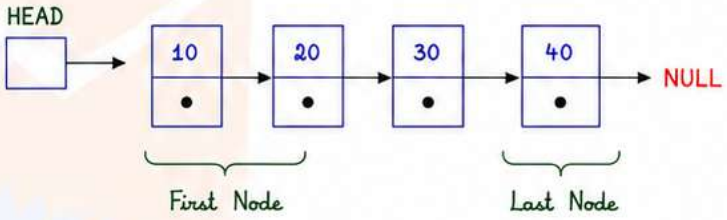
⇒ Data Field : Stores the value.

⇒ Link Field : Stores the address of the next node.

Representation :



3. Basic Singly Linked List :



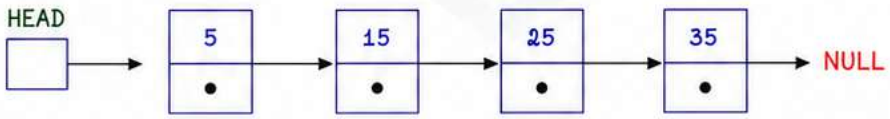
4. Definition :

A Singly Linked List is a collection of nodes where each node contains data and a pointer to the next node in the sequence. The last node contains NULL in its link part.

5. Characteristics (Summary) :

- ★ Dynamic in nature.
- ★ Memory is allocated as needed.
- ★ Insertion and deletion are easy.
- ★ Access is sequential.
- ★ Suitable for frequently changing data.
- ★ Final node points to NULL.

6. Small Singly Linked List Example :

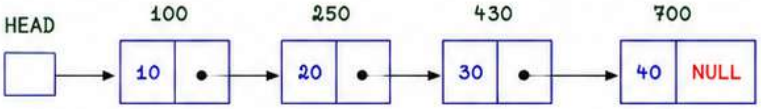


Apna College Bihar

7. Memory Representation :

Nodes are stored at different memory locations.

Example :



Memory Table :

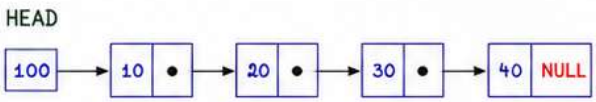
Address	Data	Next Address
100	10	250
250	20	430
430	30	700
700	40	NULL

Each node is stored at a separate memory location.

8. HEAD Pointer :

HEAD pointer stores the address of the first node of the linked list.

Example :



If HEAD = NULL, then the linked list is empty.

Empty Linked List :



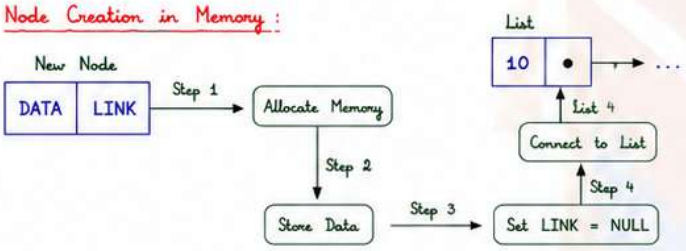
HEAD = NULL
No nodes in the list.

9. Node Creation (Steps) :

Steps to create a new node :

1. Allocate memory using malloc() or new.
2. Store the data in the data field.
3. Set the link field to NULL.
4. Connect the node to the linked list using pointers.

Node Creation in Memory :



10. Data and Link Field Explanation :



Stores the actual information or value. (Can be integer, float, character, structure, etc.)

Stores the address of the next node in the list. (Points to next node)

Note :
In the last node, LINK field contains NULL which indicates the end of the list.

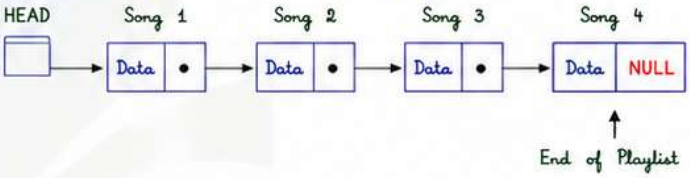
11. Types of Linked List :

- ★ Singly Linked List → Each node has one link.
- ★ Doubly Linked List → Each node has two links (next and previous).
- ★ Circular Linked List → Last node link points to the first node.
- ★ Circular Doubly Linked List → Combination of above two.

We are studying :
- ★ Singly Linked List (in this topic).

12. Real World Example :

Playlist of Songs (Singly Linked List)
Each song is a node. Link points to the next song.
The last song points to NULL (end of playlist).



Summary :

- ★ Singly Linked List is a dynamic linear data structure.
- ★ Each node contains data and address of next node.
- ★ Nodes are stored at different memory locations.
- ★ HEAD pointer points to the first node.
- ★ Last node link field contains NULL.
- ★ Traversal is possible in forward direction only.
- ★ Memory is allocated dynamically.
- ★ Widely used in many real-life applications.

Quick View :



Apna College Bihar

13. Advantages of Singly Linked List :

- ★ Dynamic size (can grow or shrink as needed).
- ★ Efficient insertion and deletion (no shifting).
- ★ Memory is allocated only when required.
- ★ No memory wastage due to fixed size.
- ★ Easy to implement and use.
- ★ Useful for applications where size is not known in advance.

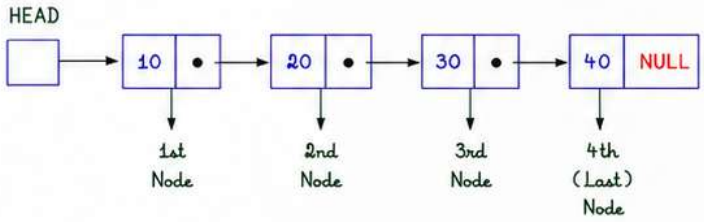
14. Disadvantages of Singly Linked List :

- ★ Sequential access only (no random access).
- ★ Extra memory required for pointers.
- ★ Reverse traversal is not possible.
- ★ Searching an element is slower than array.
- ★ Pointer manipulation increases complexity.

15. Applications of Singly Linked List :

- ★ Implementing Stack
- ★ Implementing Queue
- ★ Polynomial representation
- ★ Sparse matrix representation
- ★ Graph representation
- ★ Hash table chaining
- ★ Dynamic memory management
- ★ Music playlist
- ★ Browser history
- ★ Undo operations in editors
- ★ File system (linked allocation)
- ★ Simulation of real-world scenarios

16. Complete Singly Linked List Illustration :

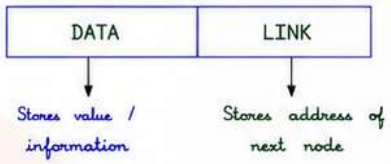


17. Data and Link Field Explanation :

Every node in a Singly Linked List contains two parts.

(1) Data Field ⟶ Stores the actual value or information.

(2) Link Field ⟶ Stores the address of the next node in the list.



Note :

In the last node, LINK field contains NULL.
It indicates the end of the list.

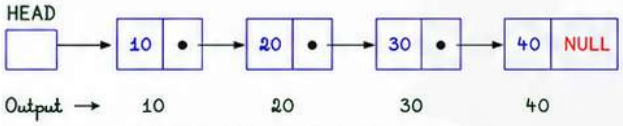
18. Traversal in Singly Linked List :

Traversal means visiting each node from beginning to end.

Algorithm :

```
Step 1 : START
Step 2 : Set a pointer "temp" = HEAD.
Step 3 : If temp == NULL, STOP (List is empty).
Step 4 : Print temp → DATA.
Step 5 : Move temp to next node (temp = temp → LINK).
Step 6 : Repeat Step 3 to Step 5 until temp == NULL.
Step 7 : STOP.
```

Example :



Output ⟶ 10 20 30 40

Time Complexity : O(n)
Space Complexity : O(1)

19. Empty Linked List :

If HEAD = NULL, then the list is empty.

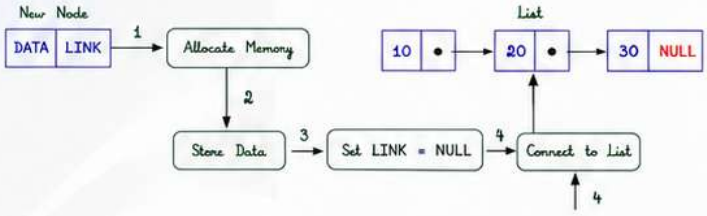
Example :



No nodes are present in the list.

20. Steps to Create a New Node :

① Allocate memory for a new node using malloc() or new.
② Store data in the DATA field.
③ Set the LINK field to NULL.
④ Connect the new node to the list using pointers.



Summary :

- ★ Singly Linked List is a linear data structure.
- ★ Each node has DATA and LINK fields.
- ★ Nodes are stored at different memory locations.
- ★ HEAD stores the address of first node.
- ★ Last node points to NULL.
- ★ It is dynamic in size and memory.
- ★ Traversal is in forward direction only.
- ★ Useful in many real-life applications.

Quick Revision :



Apna College Bihar

Topic 2 : Algorithms of Several Operations : Traversing , Searching , Insertion , Deletion

Introduction :

After creating a Singly Linked List , different operations are performed on it to manage the stored data. The four fundamental operations are :

- ★ Traversing
- ★ Searching
- ★ Insertion
- ★ Deletion

These operations are performed using pointers and help in accessing , modifying or removing nodes from the linked list.

1. Traversing :

Definition :

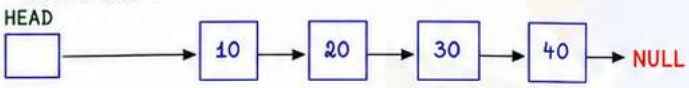
Traversing is the process of visiting each node of the linked list exactly once from the first node to the last node.

Algorithm :

Step 1 : Set TEMP = HEAD.
Step 2 : If HEAD = NULL , display "List is Empty".
Step 3 : Print DATA of TEMP.
Step 4 : Move TEMP = TEMP → LINK.
Step 5 : Repeat Steps 3 and 4 until TEMP = NULL.
Step 6 : Stop.

Example :

Linked List :

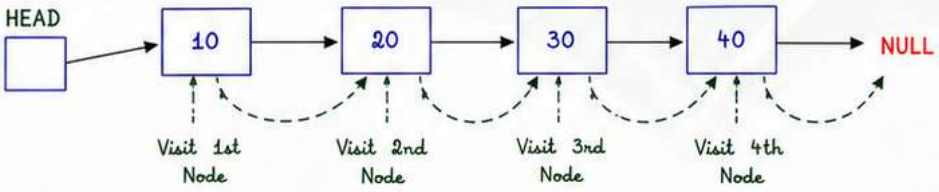


Traversal Output :

10 20 30 40 → Output

Time Complexity : O(n)

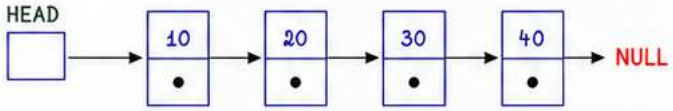
Linked List Traversal Diagram :



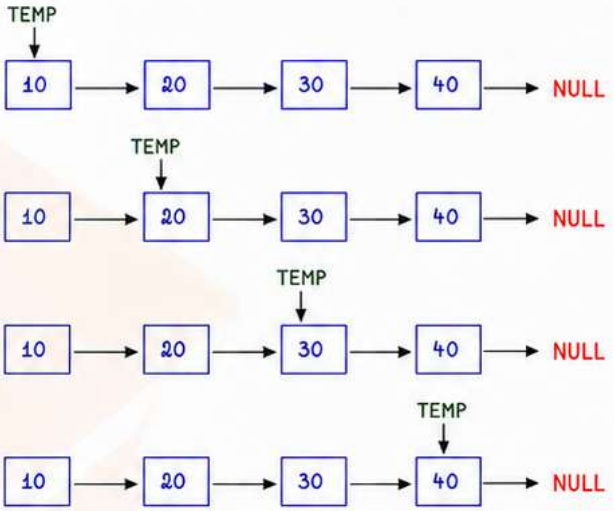
We start from HEAD and move from node to node by following LINK until we reach NULL. It is called Traversal.

Example :

Linked List :



Traversal Process :



After visiting last node , TEMP = NULL. Traversal stops.

Key Points about Traversal :

- ★ Every node is visited exactly once.
- ★ Start from HEAD and move using LINK.
- ★ Last node is identified when LINK = NULL.
- ★ Works in forward direction only.
- ★ Used for displaying or processing all elements.

Apna College Bihar

2. Searching :

Definition :

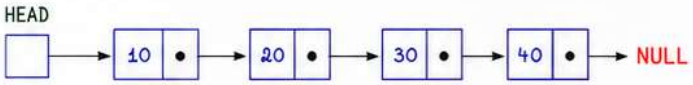
Searching means finding whether a required element (KEY) exists in the linked list or not.

Algorithm :

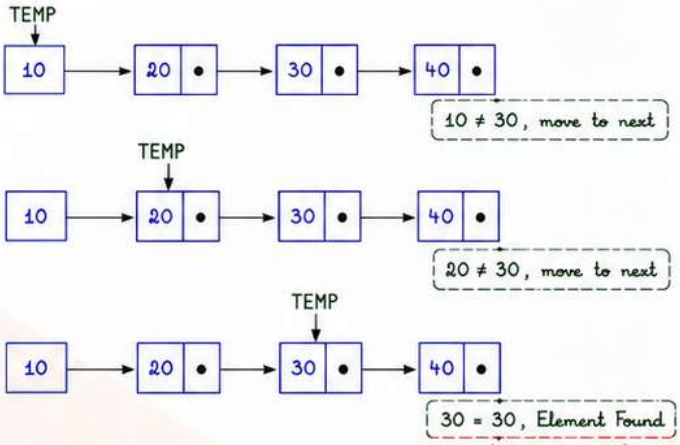
Step 1 : Set TEMP = HEAD.
Step 2 : If HEAD = NULL, display "List is Empty".
Step 3 : Compare TEMP → DATA with KEY.
Step 4 : If equal, display "Element Found" and stop.
Step 5 : Otherwise move TEMP = TEMP → LINK.
Step 6 : Repeat Steps 3 to 5 until TEMP = NULL.
Step 7 : If not found, display "Element Not Found".

Example 1 : Search KEY = 30

Linked List :

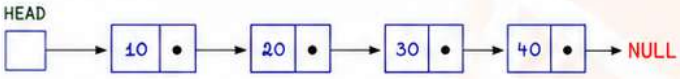


Searching Process :

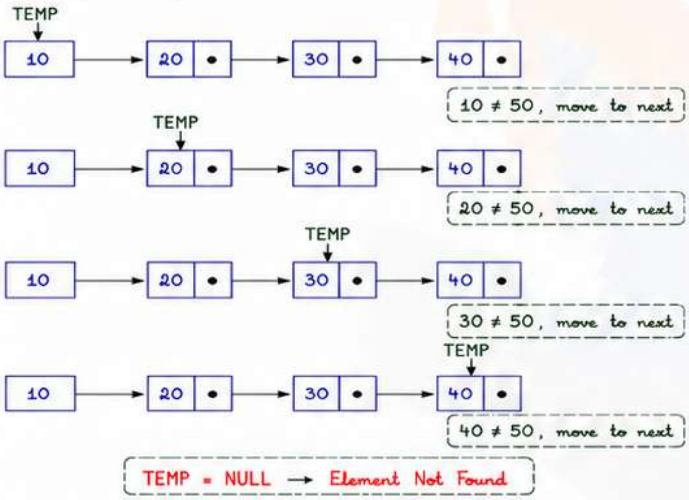


Example 2 : Search KEY = 50 (Not Present)

Linked List :



Searching Process :



Key Points about Searching :

- ★ Each node is checked one by one.
- ★ If data matches KEY, element is found.
- ★ If end of list (NULL) is reached and data is not found, element does not exist.
- ★ Works in forward direction only.
- ★ Suitable for unsorted linked list.

Time Complexity :

- ★ Best Case (Element at first node) = O(1)
- ★ Worst Case (Element at last node or not present) = O(n)
- ★ Average Case = O(n)

Important Note :

Searching in a sorted linked list is same as unsorted linked list because access is only sequential.

Summary :

- ★ Searching checks whether a given value exists.
- ★ Uses a temporary pointer to visit nodes one by one.
- ★ If value is matched, element is found.
- ★ If end of list (NULL) is reached without match, element not found.
- ★ Time complexity is O(n).

Quick View :



Apna College Bihar

3. INSERTION :

Insertion means adding a new node in the linked list.

Types : (1) Insertion at Beginning.
(2) Insertion at End
(3) Insertion at Any Position
(4) Insertion After a Given Node

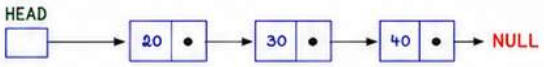
3.1. Insertion at Beginning :

Algorithim :

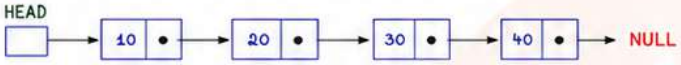
Step 1 : Create a new node (NEW).
Step 2 : Store data in NEW → DATA.
Step 3 : NEW → LINK = HEAD.
Step 4 : HEAD = NEW.
Step 5 : Stop.

Example : Insert 10 at beginning.

Before Insertion :



After Insertion :



Time Complexity : O(1)

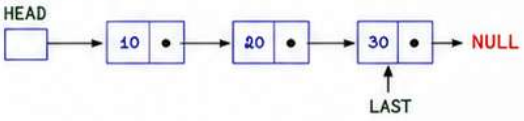
3.2. Insertion at End :

Algorithim :

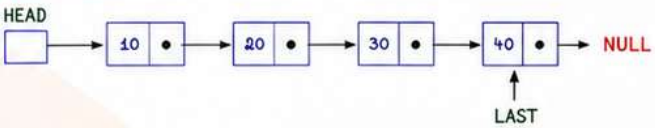
Step 1 : Create a new node (NEW).
Step 2 : Move TEMP to last node.
Step 3 : LAST → LINK = NEW.
Step 4 : NEW → LINK = NULL.
Step 5 : Stop.

Example : Insert 40 at end.

Before Insertion :



After Insertion :



Time Complexity : O(n)

3.3. Insertion at Any Position :

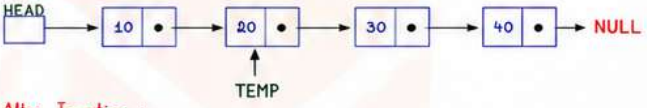
Algorithim :

Step 1 : Move TEMP to the node after which new node is to be inserted.
Step 2 : NEW → LINK = TEMP → LINK.
Step 3 : TEMP → LINK = NEW.
Step 4 : Stop.

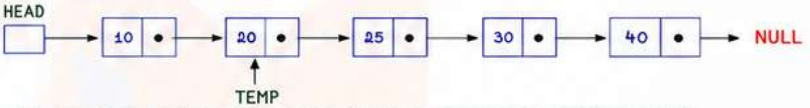
Time Complexity : O(n)

Example : Insert 25 after 20.

Before Insertion :



After Insertion :



Note : To insert at any position, first we reach to the required position.
★ LINK of new node points to next node.
★ Then previous node points to new node.

3.4. Insertion After a Given Node :

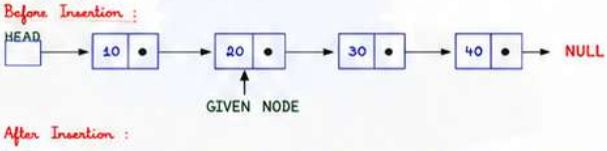
Algorithim :

Step 1 : Create a new node (NEW).
Step 2 : NEW → LINK = GIVEN → LINK.
Step 3 : GIVEN → LINK = NEW.
Step 4 : Stop.

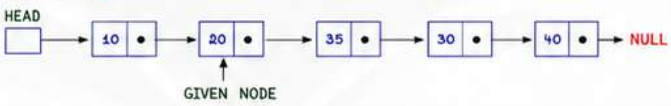
Time Complexity : O(1)

Example : Insert 25 after 20.

Before Insertion :



After Insertion :



Note :
★ GIVEN NODE is the node after which new node is to be inserted.
★ Useful when we already have reference of a node.

Summary :

★ Insertion adds a new node in the list.
★ At beginning → fastest (O(1)).
★ At end → need to reach last node (O(n)).
★ At any position → need to traverse (O(n)).

Quick View :

Operation	Time Complexity
Insert at Beginning	O(1)
Insert at End	O(n)
Insert at Position	O(n)
Insert After a Node	O(1)

Important Points :

★ Always update links in correct order.
★ If list is empty (HEAD = NULL), insertion at beginning and end are same.
★ Memory is allocated dynamically for new node.

Apna College Bihar

4. DELETION :

Deletion means removing a node from the linked list.

Types : (1) Deletion from Beginning
(2) Deletion from End
(3) Deletion of a Particular Node

4.1. Deletion from Beginning :

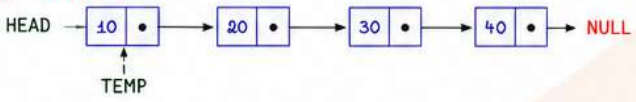
Algorithm :

Step 1 : TEMP = HEAD.
Step 2 : HEAD = HEAD → LINK.
Step 3 : Delete TEMP.
Step 4 : Stop.

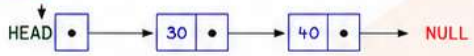
Time Complexity : O(1)

Example : Delete first node.

Before Deletion :



After Deletion :



4.2. Deletion from End :

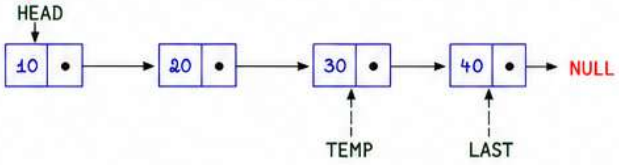
Algorithm :

Step 1 : Move TEMP to second last node.
Step 2 : Delete LAST node.
Step 3 : TEMP → LINK = NULL.
Step 4 : Stop.

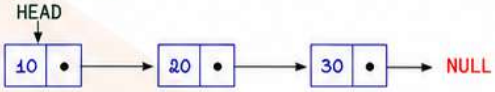
Time Complexity : O(n)

Example : Delete last node.

Before Deletion :



After Deletion :



4.3. Deletion of a Particular Node :

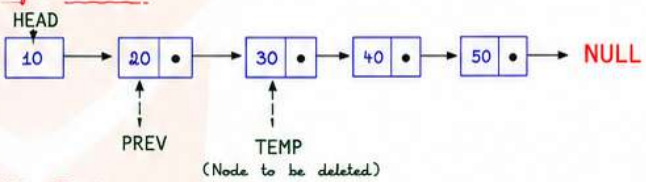
Algorithm :

Step 1 : Search the node to be deleted.
Step 2 : Let PREV be the previous node of the node to be deleted.
Step 3 : TEMP = PREV → LINK.
Step 4 : PREV → LINK = TEMP → LINK.
Step 5 : Delete TEMP.
Step 6 : Stop.

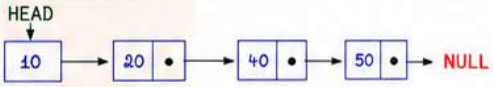
Time Complexity : O(n)

Example : Delete node with value 30.

Before Deletion :



After Deletion :



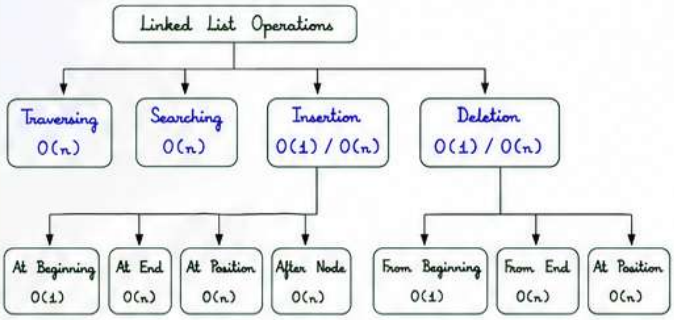
5. Complexity Analysis :

Operation	Time Complexity
Traversing	O(n)
Searching	O(n)
Insertion at Beginning	O(1)
Insertion at End	O(n)
Insertion at Position	O(n)
Insertion After a Node	O(n)
Deletion from Beginning	O(1)
Deletion from End	O(n)
Deletion of a Node	O(n)

6. Applications :

- ★ Dynamic memory allocation.
- ★ Polynomial manipulation.
- ★ Implementation of Stack and Queue.
- ★ Hash Table chaining.
- ★ Graph representation.
- ★ Music playlist.
- ★ Browser history.
- ★ Undo / Redo operations.

7. Linked List Operations Flowchart :



Summary :

- ★ Traversing visits all nodes in order.
- ★ Searching finds whether an element exists.
- ★ Insertion adds a new node at different positions.
- ★ Deletion removes a node from different positions.

Important Exam Points :

- ★ Insertion at beginning and deletion at beginning take O(1).
- ★ All other operations take O(n).
- ★ Linked list operations are done using pointers.
- ★ Linked lists are widely used in many applications.

Quick Revision :

Traverse → Search → Insert → Delete
Access Find Add Remove
Time : O(n) O(n) O(1) / O(n) O(1) / O(n)

Apna College Bihar

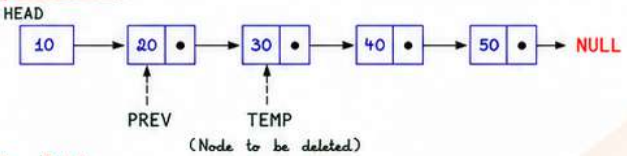
5. Deletion of a Particular Node :

Algorithm :

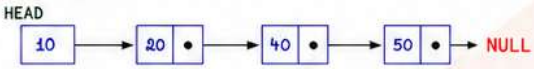
Step 1 : Search the node to be deleted.
Step 2 : Let PREV be the previous node of the node to be deleted.
Step 3 : TEMP = PREV → LINK.
Step 4 : PREV → LINK = TEMP → LINK.
Step 5 : Delete TEMP.
Step 6 : Stop.

Example : Delete node with value 30.

Before Deletion :

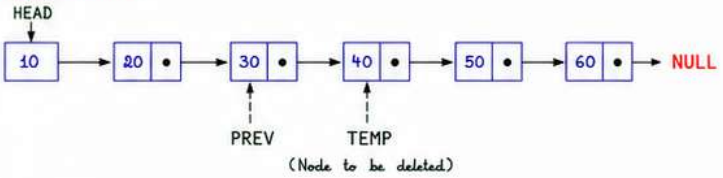


After Deletion :

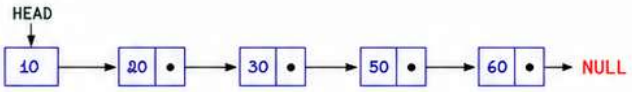


5.1. Another Example : Delete node with value 40.

Before Deletion :



After Deletion :



Key Points about Deletion :

★ We always keep track of PREV (previous node).
★ After deletion, we link PREV to TEMP → LINK.
★ Memory of deleted node is released.
★ If key is not found, list remains unchanged.

Time Complexity :

O(n)

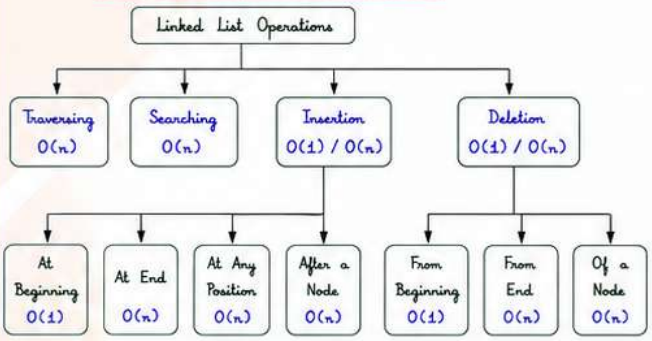
6. Complexity Analysis (Summary) :

Operation	Time Complexity
Traversing	O(n)
Searching	O(n)
Insertion at Beginning	O(1)
Insertion at End	O(n)
Insertion at Any Position	O(n)
Insertion After a Given Node	O(n)
Deletion from Beginning	O(1)
Deletion from End	O(n)
Deletion of a Particular Node	O(n)

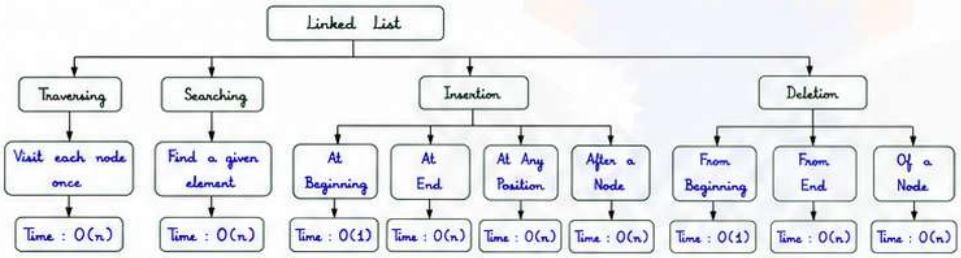
7. Applications of Linked List :

★ Dynamic memory allocation.
★ Polynomial representation.
★ Implementation of Stack.
★ Implementation of Queue.
★ Hash table chaining.
★ Graph representation.
★ Music playlist.
★ Browser history.
★ Undo / Redo operations.

8. Linked List Operations Flowchart :



9. Complete Linked List Operations Diagram :



10. Summary :

★ Linked list operations are performed using pointers.
★ Traversing and searching require O(n) time.
★ Insertion at beginning takes O(1) time.
★ All other insertions and deletions require O(n) time.
★ Linked lists are dynamic in size.
★ They are widely used in real life.

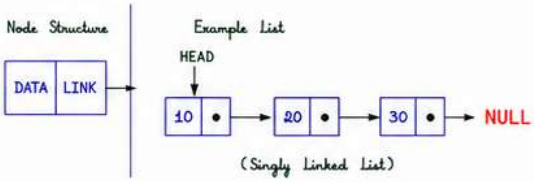
11. Important Exam Points :

★ Always update links in correct order.
★ At beginning → no need to traverse.
★ At end → need to reach last node.
★ At any position → need to reach that position.
★ Use temporary pointers (TEMP).
★ Check for empty list (HEAD = NULL).
★ Avoid memory leaks while deleting nodes.

12. Quick Revision Table :

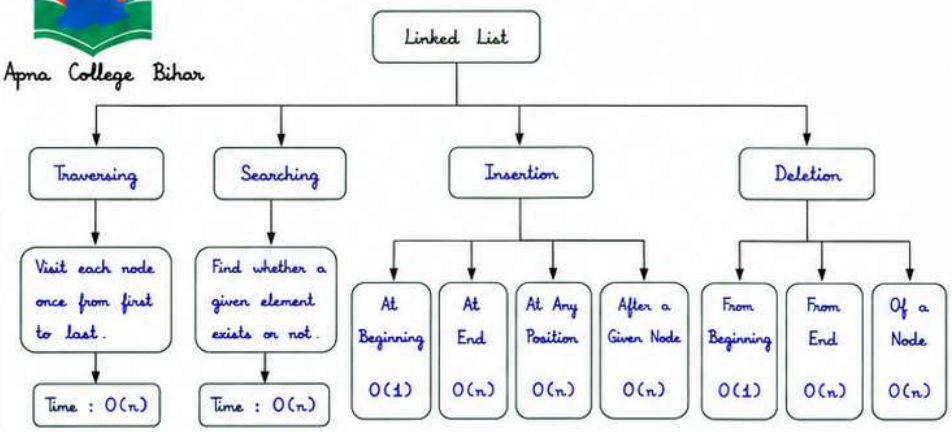
Operation	Best Case	Worst Case	Space Complexity
Traversing	O(1)	O(n)	O(1)
Searching	O(1)	O(n)	O(1)
Insert at Beginning	O(1)	O(1)	O(1)
Insert at End	O(n)	O(n)	O(1)
Insert at Position	O(n)	O(n)	O(1)
Delete at Beginning	O(1)	O(1)	O(1)
Delete at End	O(n)	O(n)	O(1)
Delete at Position	O(n)	O(n)	O(1)

13. Quick View (Memory Representation) :



Apna College Bihar

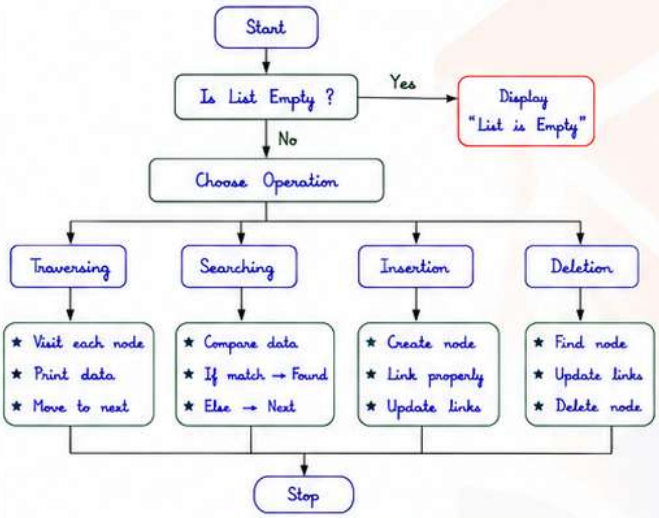
12. Complete Linked List Operations Diagram :



13. Summary :

- ★ Linked list is a linear data structure in which nodes are not stored in contiguous memory locations.
- ★ Operations on linked list are performed using pointers.
- ★ Traversing and searching take O(n) time.
- ★ Insertion at beginning and deletion at beginning take O(1) time.
- ★ Insertion at end, insertion at any position and all deletion operations take O(n) time.
- ★ Linked list is dynamic in size and memory is allocated when required.

14. Linked List Operations Flowchart :



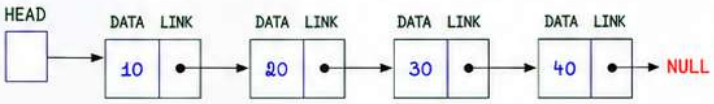
15. Important Exam Points :

1. Traversing means visiting each node from first to last.
2. Searching means finding whether a given element exists in the list.
3. Insertion can be done at beginning, end, at any position or after a given node.
4. Deletion can be done from beginning, end or of a particular node.
5. Insertion at beginning takes O(1) time.
6. Deletion at beginning takes O(1) time.
7. Traversing and searching require O(n) time.
8. Insertion at end and deletion at end take O(n) time.
9. Insertion at any position and deletion of a node take O(n) time.
10. Linked list is preferred when dynamic memory allocation is needed.
11. Pointers are used to access and update nodes.
12. Linked list is used in stacks, queues, graphs, polynomial representation, etc.

16. Quick Revision Table :

Operation	What It Does	Time Complexity	Remarks
Traversing	Visit all nodes	O(n)	From first to last
Searching	Find an element	O(n)	Linear search
Insertion at Beginning	Add node at start	O(1)	Fastest insertion
Insertion at End	Add node at last	O(n)	Need to reach last
Insertion at Any Position	Add node at position	O(n)	Need to reach position
Insertion After a Node	Add node after given	O(n)	Need to find node
Deletion from Beginning	Remove first node	O(1)	Fastest deletion
Deletion from End	Remove last node	O(n)	Need second last
Deletion of a Node	Remove particular node	O(n)	Need previous node

17. Memory Representation (Singly Linked List) :



Note :

- ★ Each node has two parts → DATA and LINK.
- ★ LINK contains address of next node.
- ★ Last node's LINK contains NULL.
- ★ HEAD stores address of first node.

Apna College Bihar

Linked Representation of Stack and Queue

Introduction :

A Stack and a Queue can be implemented using Linked List instead of array.
In linked representation, nodes are created dynamically in memory whenever required.
This removes the limitation of fixed size and avoids memory wastage.

★ Stack follows LIFO (Last In, First Out) principle.

★ Queue follows FIFO (First In, First Out) principle.

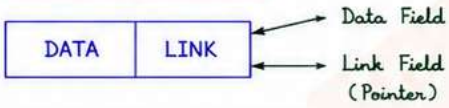
Both use linked list nodes consisting of Data and Link (Pointer) fields.

1. Linked Representation of Stack :

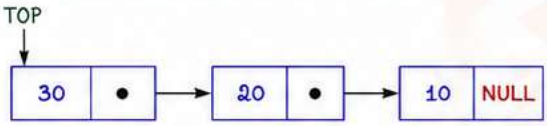
Definition :

A Linked Stack is a stack implemented using singly linked list. The TOP pointer always points to the first node of the linked list.

Node Structure (Stack)



Linked Stack Representation



Insertion (Push) and Deletion (Pop) are performed at the TOP.

2. Push Operation (Linked Stack) :

Definition :

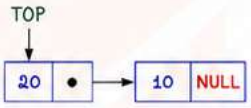
Push inserts a new node at the TOP of the stack.

Algorithm :

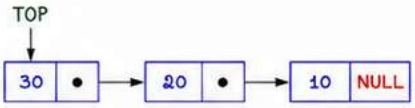
Step 1 : Create a new node.
Step 2 : Store data in the new node.
Step 3 : NEW → LINK = TOP.
Step 4 : TOP = NEW.
Step 5 : Stop.

Example : Push (30)

Before Push



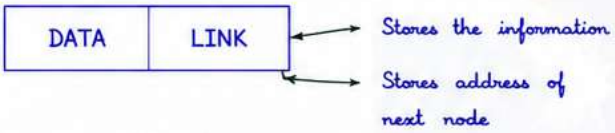
After Push (30)



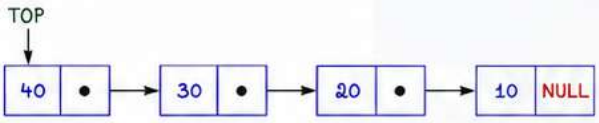
Time Complexity : O(1)

3. Structure of Linked Stack Node :

Each node contains :



Example : (Stack)



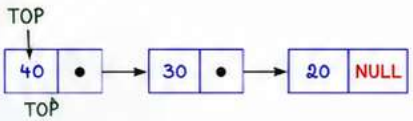
(Top Element) (Middle Elements) (Bottom Element)

The node at the TOP is the element that will be removed first.

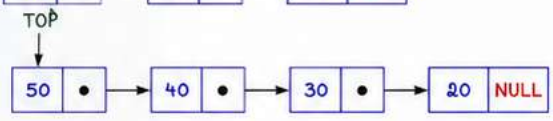
4. Push Operation Example and Algorithm :

(Example) Push (50)

Before Push :



After Push (50) :



Algorithm (Push) :

Step 1 : Create a new node (NEW).
Step 2 : Store data in NEW → DATA.
Step 3 : NEW → LINK = TOP.
Step 4 : TOP = NEW.
Step 5 : Stop.

Time Complexity : O(1)

Apna College Bihar

5. Pop Operation (Linked Stack) :

Definition :

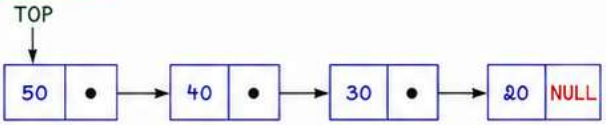
Pop removes the node from the TOP of the stack.

Algorithm :

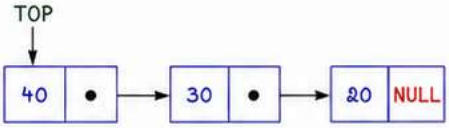
Step 1 : Check if TOP = NULL.
Step 2 : TEMP = TOP.
Step 3 : TOP = TOP → LINK.
Step 4 : Delete TEMP.
Step 5 : Stop.

Example : Pop ( )

Before Pop :



After Pop :



Time Complexity : O(1)

6. Advantages of Linked Stack :

- ★ Dynamic memory allocation.
- ★ No fixed size limitation.
- ★ No memory wastage.
- ★ Overflow occurs only when memory is exhausted.
- ★ Efficient Push and Pop operations.

7. Linked Stack Summary :

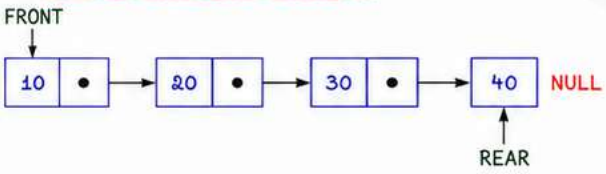
- ★ Implemented using singly linked list.
- ★ TOP pointer points to the first node.
- ★ Insertion (Push) and deletion (Pop) are done at TOP.
- ★ Both operations take O(1) time.
- ★ Follows LIFO (Last In, First Out) principle.

8. Linked Representation of Queue :

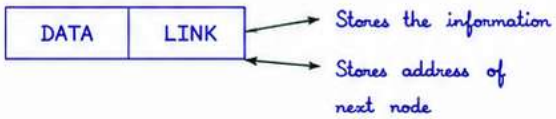
Definition :

A Linked Queue is a queue implemented using singly linked list. It uses two pointers FRONT and REAR.
Insertion is performed at REAR and deletion is performed at FRONT.

9. Structure of Linked Queue :



Each node contains :



10. Enqueue Operation (Linked Queue) :

Definition :

Enqueue inserts a new node at the REAR of the queue.

Algorithm :

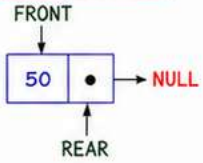
Step 1 : Create a new node.
Step 2 : Store data in the new node.
Step 3 : NEW → LINK = NULL.
Step 4 : If queue is empty
FRONT = REAR = NEW.
Step 5 : Otherwise
REAR → LINK = NEW.
Step 6 : REAR = NEW.
Step 7 : Stop.

Example : Enqueue (50)

Before Enqueue : (Empty Queue)

FRONT = NULL, REAR = NULL

After Enqueue (50) :



Time Complexity : O(1)

Apna College Bihar

11. Dequeue Operation (Linked Queue) :

Definition :

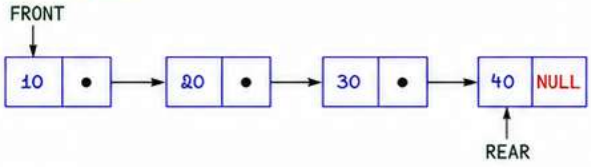
Dequeue removes the node from the FRONT of the queue.

Algorithm :

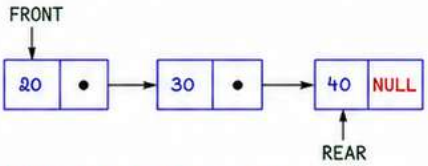
Step 1 : Check if FRONT = NULL.
Step 2 : TEMP = FRONT.
Step 3 : FRONT = FRONT → LINK.
Step 4 : Delete TEMP.
Step 5 : If FRONT = NULL,
REAR = NULL.
Step 6 : Stop.

Example : Dequeue ( )

Before Dequeue :



After Dequeue :



After Dequeue (when last element removed) :

FRONT = NULL, REAR = NULL

Time Complexity : O(1)

12. Array vs Linked Representation :

Feature	Array Implementation	Linked Implementation
Memory Allocation	Static	Dynamic
Size	Fixed	Dynamic
Memory Wastage	Possible	Very Less
Overflow	When size full	When memory exhausted
Insertion	Easy (but shifting may be required)	Easy
Deletion	Easy (but shifting may be required)	Easy
Implementation	Simple	Slightly Complex
Performance	Good	Better (Dynamic)

13. Linked Queue Summary :

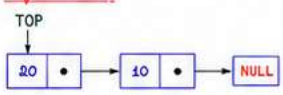
★ Queue uses two pointers FRONT and REAR.
★ Insertion (Enqueue) is done at REAR.
★ Deletion (Dequeue) is done at FRONT.
★ If queue is empty, both FRONT and REAR are NULL.
★ Enqueue and Dequeue operations take O(1) time.
★ Linked Queue is dynamic in size.
★ No memory wastage.
★ Overflow occurs only when memory is exhausted.

14. Before and After Diagrams Summary :

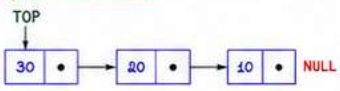
(A) Linked Stack

Push (before and after)

Before Push :

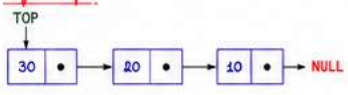


After Push (30) :

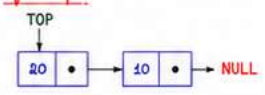


Pop (before and after)

Before Pop :



After Pop :



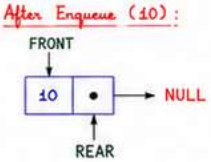
(B) Linked Queue

Enqueue (before and after)

Before Enqueue : (Empty Queue)

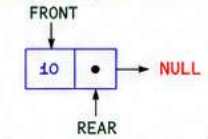
FRONT = NULL, REAR = NULL

After Enqueue (10) :

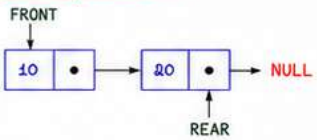


Enqueue (continued)

Before Enqueue (20) :

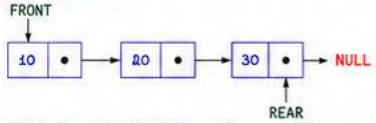


After Enqueue (20) :

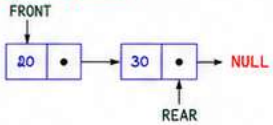


Dequeue (before and after)

Before Dequeue :



After Dequeue :



After Dequeue (last element removed) :

FRONT = NULL, REAR = NULL

Note :

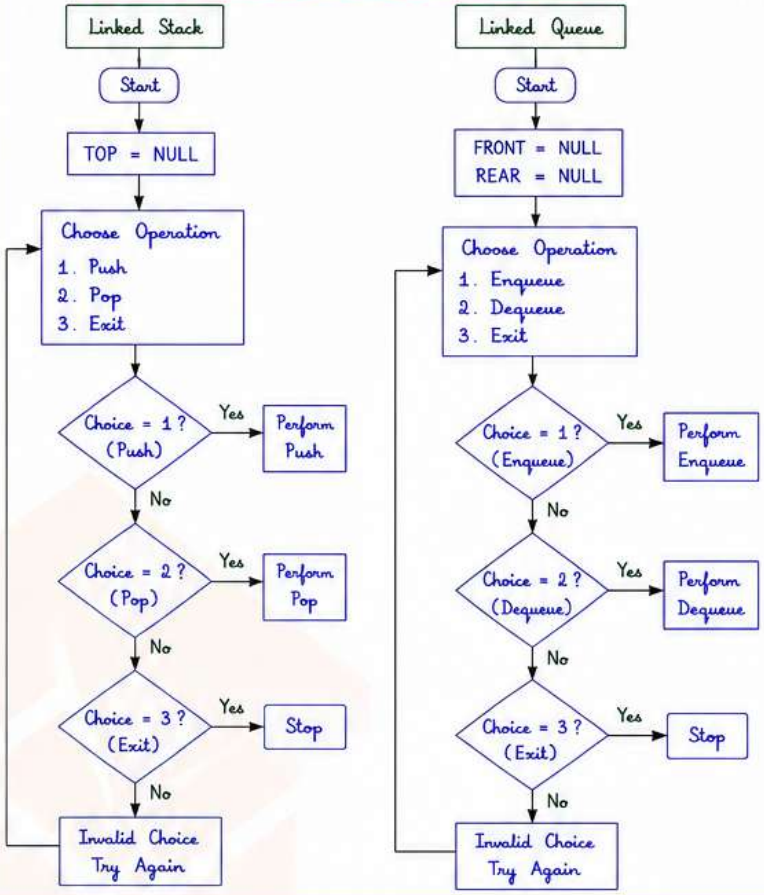
★ In Linked Stack, only TOP pointer is used.
★ In Linked Queue, both FRONT and REAR pointers are used.
★ All operations (Push, Pop, Enqueue, Dequeue) take O(1) time.
★ Linked structures are preferred when size is not known in advance.

Apna College Bihar

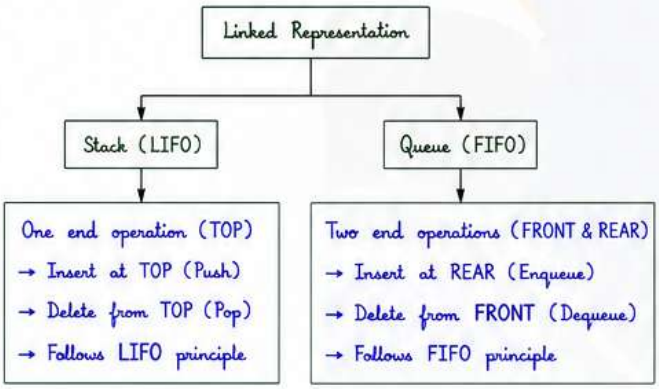
15. Applications :

- ★ Compiler design (expression evaluation, syntax parsing)
- ★ Function call stack (recursion handling)
- ★ CPU scheduling (ready queue, waiting queue)
- ★ Printer queue (print request management)
- ★ Breadth First Search (BFS) uses queue
- ★ Depth First Search (DFS) uses stack
- ★ Undo / Redo operations
- ★ Browser history (back / forward)
- ★ Expression conversion and evaluation
- ★ Memory management in operating systems

16. Complete Flowchart (Linked Stack & Queue) :



17. Comparison Diagram :



18. Quick Revision Table :

Operation	Stack (Linked)	Queue (Linked)
Data Structure	Singly Linked List	Singly Linked List
Pointers Used	TOP	FRONT, REAR
Insertion	At TOP (Push)	At REAR (Enqueue)
Deletion	From TOP (Pop)	From FRONT (Dequeue)
Empty Condition	TOP = NULL	FRONT = NULL
Time Complexity	O(1)	O(1)
Memory Allocation	Dynamic	Dynamic
Advantage	No fixed size Overflow only when memory full	No fixed size Overflow only when memory full

19. Important Exam Points :

- ★ Linked Stack uses only TOP pointer.
- ★ Linked Queue uses both FRONT and REAR pointers.
- ★ Push and Pop operations in Linked Stack take O(1) time.
- ★ Enqueue and Dequeue operations in Linked Queue take O(1) time.
- ★ Overflow in linked representation occurs only when memory is exhausted.
- ★ Linked representation is dynamic and memory efficient.
- ★ Linked Stack follows LIFO, Linked Queue follows FIFO.
- ★ In Linked Stack, insertion and deletion at same end (TOP).
- ★ In Linked Queue, insertion and deletion at different ends (REAR and FRONT).
- ★ Nodes are created dynamically as per requirement.

20. Summary :

Stack and Queue can be efficiently implemented using linked list. Linked Stack uses TOP pointer and performs Push and Pop at TOP, while Linked Queue uses FRONT and REAR pointers and performs Enqueue at REAR and Dequeue at FRONT. Both operations are dynamic and take constant time O(1). Linked representation removes the limitation of fixed size and provides better memory utilization.

Apna College Bihar

Header Nodes, Doubly Linked List: Operations and Algorithmic Analysis

Introduction :

A Header Linked List and a Doubly Linked List (DLL) are advanced forms of linked lists that improve data organization and traversal. A Header Linked List contains a special node called the Header Node, while a Doubly Linked List allows movement in both forward and backward directions using two pointers.

A Doubly Linked List stores three fields in every node : Previous Pointer (PREV), Data Field (DATA) and Next Pointer (NEXT).

These structures are widely used in operating systems, web browsers, music players, undo/redo systems, and memory management.

Note :

- ★ Header Node is not a data node.
- ★ It contains information about the list.
- ★ It helps in easy management of the list.

1. Header Node :

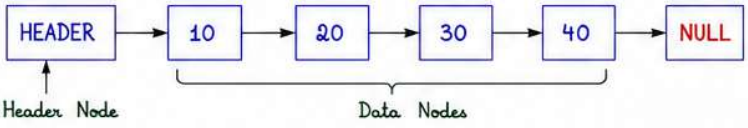
Definition :

A Header Node is a special node placed at the beginning of a linked list. It usually does not store user data. Instead, it stores information about the list, such as the number of nodes or the address of the first node.

Characteristics :

- ★ Present before the first data node.
- ★ Simplifies insertion and deletion.
- ★ Makes algorithms easier.
- ★ May store list information.
- ★ Improves linked list management.

Representation :



2. Advantages of Header Node :

- ★ Simplifies programming.
- ★ Reduces special cases.
- ★ Easier insertion and deletion.
- ★ Better memory management.
- ★ Improves readability of algorithms.

Information that can be stored in Header Node :

①	Count	Total number of nodes in the list.
②	Pointer	Address of the first node.
③	Status	Empty / Non-empty status.
④	Others	Other useful information.

Exam Point :

Header Node is an optional node but very useful in complex operations and large programs.

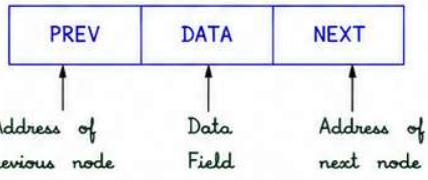
Apna College Bihar

3. Doubly Linked List (DLL) :

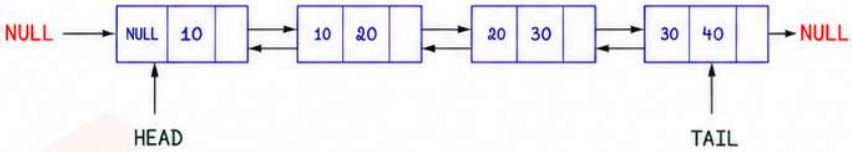
Definition :

A Doubly Linked List is a linked list in which each node contains two pointers : one pointer to the previous node (PREV) and another pointer to the next node (NEXT). It allows movement in both forward and backward directions.

4. Structure of a Doubly Linked List Node :



5. Representation of Doubly Linked List :



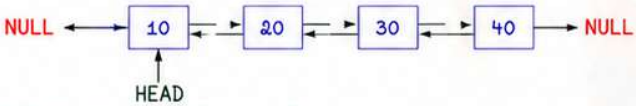
6. Characteristics of Doubly Linked List :

- ★ It is a linear data structure.
- ★ Each node contains two pointers (PREV and NEXT).
- ★ Traversal is possible in both forward and backward directions.
- ★ Dynamic memory allocation is used.
- ★ Insertion and deletion are easier.
- ★ It requires more memory than singly linked list.

7. Traversing a Doubly Linked List :

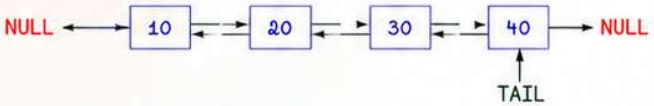
(A) Forward Traversal :

Start from the first node and follow the NEXT pointer until NULL is reached.



(B) Backward Traversal :

Start from the last node and follow the PREV pointer until NULL is reached.



(C) Algorithm for Forward Traversal :

Step 1 : TEMP = HEAD.
Step 2 : While (TEMP ≠ NULL)
Step 3 : Print TEMP → DATA.
Step 4 : TEMP = TEMP → NEXT.
Step 5 : Stop.

Time Complexity : O(n)

(D) Algorithm for Backward Traversal :

Step 1 : TEMP = TAIL.
Step 2 : While (TEMP ≠ NULL)
Step 3 : Print TEMP → DATA.
Step 4 : TEMP = TEMP → PREV.
Step 5 : Stop.

Important Note :

- ★ Forward traversal uses NEXT pointer.
- ★ Backward traversal uses PREV pointer.

Exam Point :

DLL allows movement in both directions, which is not possible in singly linked list.

Applications (DLL) :

- ★ Browser back and forward navigation.
- ★ Undo / Redo operations.
- ★ Music playlist (next / previous).
- ★ Operating systems.
- ★ Image viewer.
- ★ Memory management.
- ★ LRU Cache.
- ★ Text editors.

Apna College Bihar

8. Insertion in Doubly Linked List :

Insertion can be performed in DLL at :

- ★ At the beginning
- ★ At the end
- ★ Before a given node
- ★ After a given node.
- ★ At any position

General Steps (Idea) :

1. Create a new node.
2. Adjust the pointers.
3. Connect the node in the list.
4. Update HEAD / TAIL if required.
5. Stop.

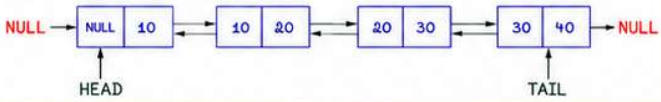
9. Insertion at Beginning :

Algorithim :

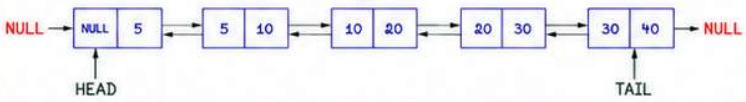
```
Step 1 : Create a new node NEW.
Step 2 : NEW → DATA = ITEM.
Step 3 : NEW → PREV = NULL.
Step 4 : NEW → NEXT = HEAD.
Step 5 : If (HEAD ≠ NULL)
           HEAD → PREV = NEW.
Step 6 : HEAD = NEW.
Step 7 : If (TAIL = NULL)
           TAIL = NEW.
Step 8 : Stop.
```

Time Complexity : O(1)

Before Insertion (DLL) :



After Insertion of 5 at Beginning :

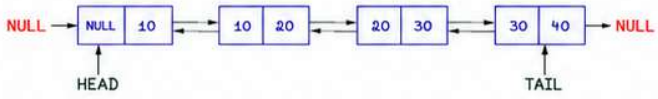


10. Insertion at End :

Algorithim :

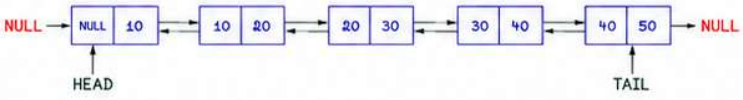
```
Step 1 : Create a new node NEW.
Step 2 : NEW → DATA = ITEM.
Step 3 : NEW → NEXT = NULL.
Step 4 : NEW → PREV = TAIL.
Step 5 : If (TAIL ≠ NULL)
           TAIL → NEXT = NEW.
Step 6 : TAIL = NEW.
Step 7 : If (HEAD = NULL)
           HEAD = NEW.
Step 8 : Stop.
```

Before Insertion (DLL) :



Time Complexity : O(1)

After Insertion of 50 at End :



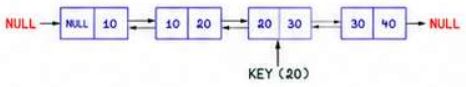
11. Insertion Before a Given Node (Key) :

Algorithim :

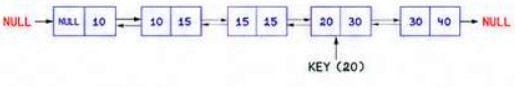
```
Step 1 : Locate the node (KEY).
Step 2 : Create a new node NEW.
Step 3 : NEW → DATA = ITEM.
Step 4 : NEW → NEXT = KEY.
Step 5 : NEW → PREV = KEY → PREV.
Step 6 : If (KEY → PREV ≠ NULL)
           (KEY → PREV) → NEXT = NEW.
         Else HEAD = NEW.
Step 7 : KEY → PREV = NEW.
Step 8 : Stop.
```

Example : Insert 15 before node 20

Before :



After :



Important Note :

- ★ Always update PREV and NEXT carefully.
- ★ If list is empty, update both HEAD and TAIL.
- ★ DLL gives faster insertion and deletion at both ends.

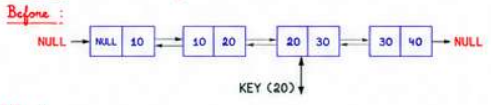
12. Insertion After a Given Node (Key) :

Algorithim :

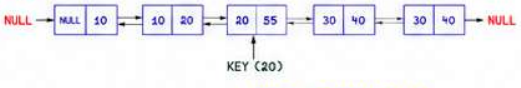
```
Step 1 : Locate the node (KEY).
Step 2 : Create a new node NEW.
Step 3 : NEW → DATA = ITEM.
Step 4 : NEW → PREV = KEY.
Step 5 : NEW → NEXT = KEY → NEXT.
Step 6 : If (KEY → NEXT ≠ NULL)
           (KEY → NEXT) → PREV = NEW.
         Else TAIL = NEW.
Step 7 : KEY → NEXT = NEW.
Step 8 : Stop.
```

Example : Insert 25 after node 20

Before :



After :





13. Deletion in Doubly Linked List :

Deletion can be performed :

- ★ From beginning
- ★ From end
- ★ From a given node (key)
- ★ From any position.

General Steps (Idea) :

1. Locate the node to be deleted.
2. Adjust PREV and NEXT pointers.
3. If first node deleted, update HEAD.
4. If last node deleted, update TAIL.
5. Free the memory.

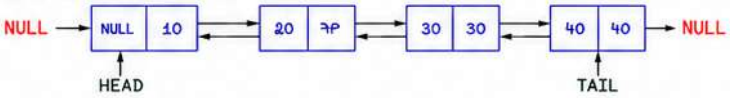
14. Deletion from Beginning :

Algorithim :

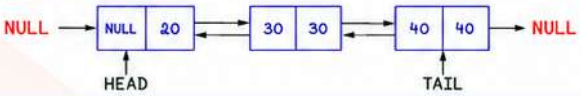
```
Step 1 : If HEAD = NULL, Underflow.
Step 2 : TEMP = HEAD.
Step 3 : HEAD = HEAD → NEXT.
Step 4 : If (HEAD ≠ NULL)
           HEAD → PREV = NULL.
Step 5 : Delete TEMP.
Step 6 : Stop.
```

Time Complexity : O(1)

Before Deletion (DLL) :



After Deletion (DLL) :

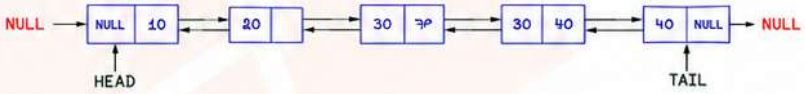


15. Deletion from End :

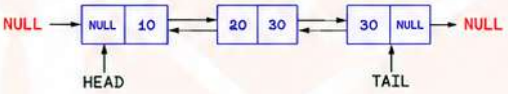
Algorithim :

```
Step 1 : If HEAD = NULL, Underflow.
Step 2 : TEMP = TAIL.
Step 3 : TAIL = TAIL → PREV.
Step 4 : If (TAIL ≠ NULL)
           TAIL → NEXT = NULL.
Step 5 : Delete TEMP.
Step 6 : Stop.
```

Before Deletion (DLL) :



After Deletion (DLL) :



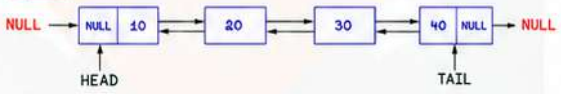
16. Deletion of a Given Node (Key) :

Algorithim :

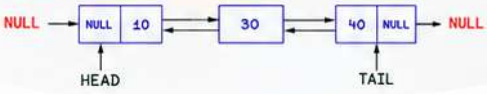
```
Step 1 : TEMP = HEAD.
Step 2 : Search the node with KEY.
Step 3 : If KEY not found → Underflow.
Step 4 : If TEMP = HEAD, delete from beginning.
Step 5 : If TEMP = TAIL, delete from end.
Step 6 : Else
           TEMP → PREV → NEXT = TEMP → NEXT.
           TEMP → NEXT → PREV = TEMP → PREV.
           Delete TEMP.
Step 7 : Stop.
```

Example : Delete node with key = 20

Before :



After :

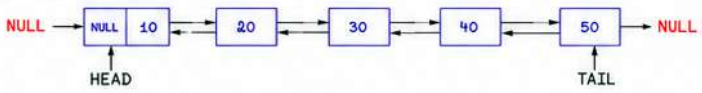


Important Notes :

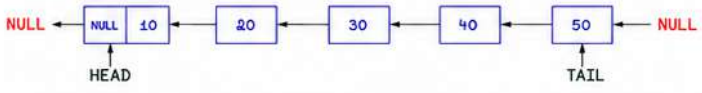
- ★ Always adjust PREV and NEXT pointers carefully.
- ★ Don't forget to update HEAD and TAIL if required.
- ★ In DLL, every node except first has PREV and every node except last has NEXT.
- ★ Deletion and insertion take O(1) time at both ends.

17. Complete Doubly Linked List (Summary Diagram) :

Forward Traversal →



Backward Traversal ←

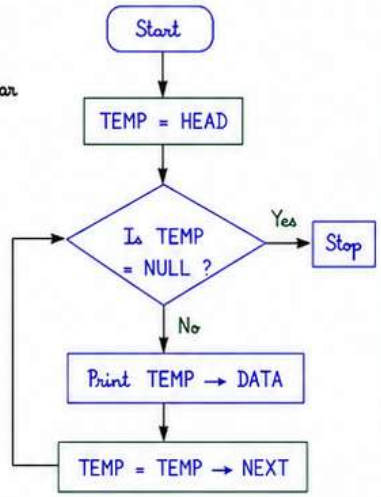


18. Singly vs Doubly Linked List :

Feature	Singly Linked List	Doubly Linked List
Pointers per node	One (NEXT)	Two (PREV, NEXT)
Traversal	Forward only	Forward & Backward
Memory required	Less	More
Insertion & Deletion	Moderate	Easier at both ends
Implementation	Simple	Complex
Applications	Simple lists	Browser, OS, playlists, Undo/Redo, etc.

Apna College Bihar

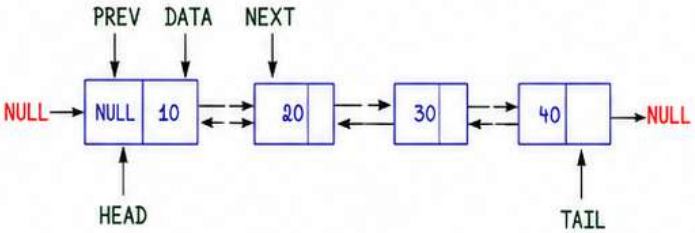
19. Flowchart : Traversal of Doubly Linked List :



Note :

- ★ For forward traversal use NEXT pointer.
- ★ For backward traversal use PREV pointer.

20. Complete Doubly Linked List Diagram :



Key Points :

- ★ HEAD points to the first node.
- ★ TAIL points to the last node.
- ★ NULL is present at both ends.
- ★ We can move in both forward and backward directions.

21. Applications of Doubly Linked List :

- ★ Browser back and forward navigation.
- ★ Undo and Redo operations.
- ★ Music playlist (next / previous).
- ★ Image viewer (previous / next image).
- ★ Text editor (undo / redo).
- ★ Operating systems (process scheduling).
- ★ Memory management.
- ★ LRU Cache (Least Recently Used).
- ★ File systems and directories.
- ★ Implementing Deque (Double Ended Queue).

22. Summary :

Header Node simplifies linked list operations by storing useful information and reducing special cases. A Doubly Linked List contains two pointers, PREV and NEXT, which allows traversal in both directions. Insertion and deletion operations are easier compared to singly linked list but require more memory.

Quick Revision Table :

Operation	At Beginning	At End	At Any Position
Insertion	O(1)	O(1)	O(n)
Deletion	O(1)	O(1)	O(n)
Traversal	O(n)	O(n)	O(n)

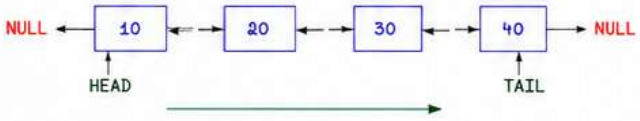
23. Important Exam Points :

- ★ Header Node is an optional node placed before the first data node.
- ★ It stores information about the list (count, address, status, etc.).
- ★ Doubly Linked List node contains three fields : PREV, DATA, NEXT.
- ★ Traversal is possible in both forward and backward directions.
- ★ Insertion and deletion at beginning or end take O(1) time.
- ★ Insertion and deletion at any position take O(n) time.
- ★ DLL requires more memory because each node has two pointers.
- ★ Every node (except first) has PREV and every node (except last) has NEXT.
- ★ In DLL, NULL is present at both the ends.
- ★ DLL is used in many real life applications like browser navigation, undo / redo, music playlist, OS, etc.

24. Final Revision Diagram (Both Traversal) :

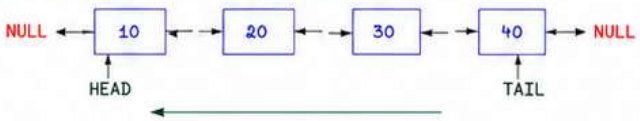
(A) Forward Traversal (Using NEXT) :

Start from HEAD and move towards TAIL.



(B) Backward Traversal (Using PREV) :

Start from TAIL and move towards HEAD.



Remember :

1. Use NEXT for forward.
2. Use PREV for backward.
3. Update both PREV and NEXT during insertion / deletion.
4. Always handle NULL conditions carefully.
5. Practice diagrams and algorithms for better understanding.

Apna College Bihar

Circular Linked Lists : All Operations and Complexity Analysis

1. Introduction :

A Circular Linked List (CLL) is a special type of linked list in which the last node points back to the first node instead of pointing to NULL. This forms a circular structure, allowing continuous traversal of the list.

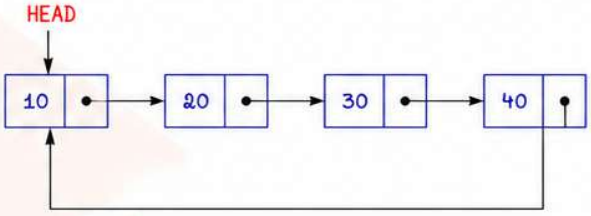
Circular linked lists are widely used in CPU scheduling, round-robin algorithms, multiplayer games, circular queues, and operating systems.

2. Circular Linked List :

Definition :

A Circular Linked List is a linked list in which the last node is connected to the first node, forming a circle.

Representation :



3. Characteristics of Circular Linked List :

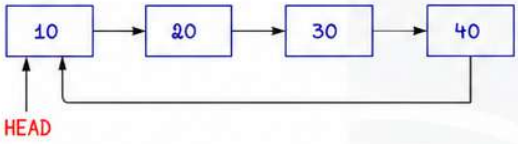
- ★ Last node points to the first node.
- ★ No NULL pointer at the end.
- ★ Traversal can start from any node.
- ★ Dynamic memory allocation.
- ★ Efficient implementation of circular queues.
- ★ Supports continuous traversal.

4. Types of Circular Linked List :

(A) Singly Circular Linked List :

- ★ Each node has one pointer (LINK).
- ★ Last node links to the first node.

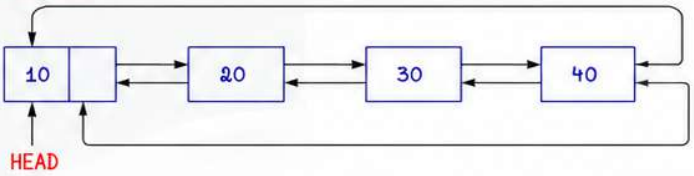
Example :



(B) Doubly Circular Linked List :

- ★ Each node has two pointers (PREV and NEXT).
- ★ First and last nodes are connected in both directions.

Example :



Note :

- ★ In Singly CLL, we keep a pointer LAST which always points to the last node.
- ★ In Doubly CLL, both first and last nodes are connected in forward and backward direction.
- ★ Traversal stops when we reach the starting node again.

Applications (Overview) :

- ★ Round Robin CPU Scheduling
- ★ Circular Queue
- ★ Multiplayer Games
- ★ Music Playlist
- ★ Traffic Signal Control
- ★ Operating Systems
- ★ Memory Management
- ★ Resource Sharing



Apna College Bihar

5. Traversing a Circular Linked List :

Definition :

Traversal means visiting every node of the list exactly once.

Algorithim :

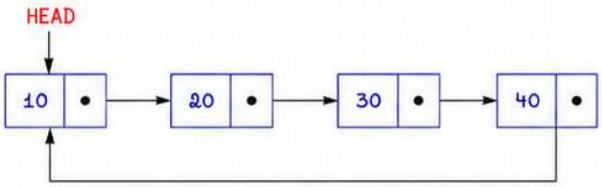
Step 1 : TEMP = HEAD.
Step 2 : If HEAD = NULL, stop.
Step 3 : Print TEMP → DATA.
Step 4 : TEMP = TEMP → LINK.
Step 5 : Repeat until TEMP = HEAD.
Step 6 : Stop.

Note :

★ Traversal stops when TEMP again reaches HEAD.

6. Traversal Example :

Example List :



Traversal Process :

10 → 20 → 30 → 40 → (back to 10) Stop.

Output :

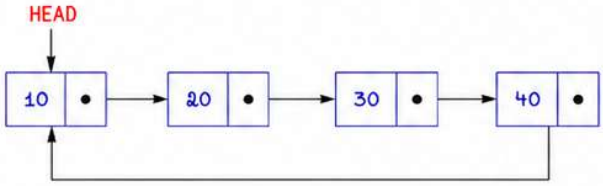
10 20 30 40

Time Complexity :

O(n)
(Linear Time)

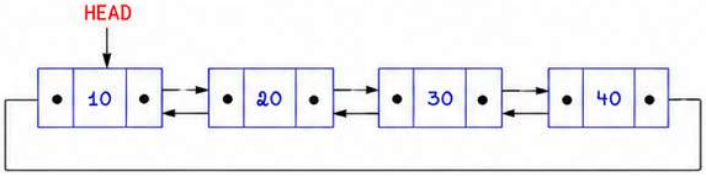
7. Types of Circular Linked List (Diagrams) :

(A) Singly Circular Linked List :



★ Each node contains one pointer (LINK).
★ Last node links back to the first node.

(B) Doubly Circular Linked List :



★ Each node contains two pointers (PREV and NEXT).
★ First and last nodes are connected in both directions.

8. Comparison : (Singly vs Doubly Circular Linked List)

Feature	Singly Circular Linked List	Doubly Circular Linked List
Pointers per node	One (LINK)	Two (PREV, NEXT)
Connection	Last → First	Both First ↔ Last
Traversal	Forward only	Forward & Backward
Memory Required	Less	More
Implementation	Simple	Complex
Applications	Circular Queue	Advanced Applications

9. Important Points :

★ In Circular Linked List, last node points to first node.
★ There is no NULL pointer at the end.
★ Traversal can start from any node.
★ Singly CLL is easier to implement.
★ Doubly CLL allows traversal in both directions.
★ Circular Linked List is widely used in real life.

Apna College Bihar

10. Insertion Operations in Circular Linked List :

Insertion can be performed at :

- ★ At Beginning
- ★ At End
- ★ After a Given Node
- ★ At Any Position

(A) Insertion at Beginning :

Algorithim :

Step 1 : Create a new node NEW.

Step 2 : NEW → LINK = HEAD.

Step 3 : Move LAST to the last node.

Step 4 : LAST → LINK = NEW.

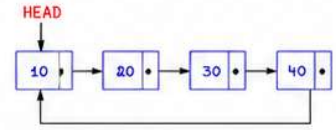
Step 5 : HEAD = NEW.

Step 6 : Stop.

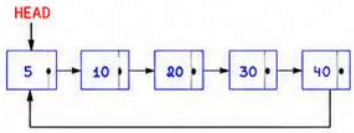
Time Complexity : O(n)

Example :

Before Insertion :



After Insertion (Insert 5) :



(B) Insertion at End :

Algorithim :

Step 1 : Create a new node NEW.

Step 2 : Move LAST to the last node.

Step 3 : LAST → LINK = HEAD.

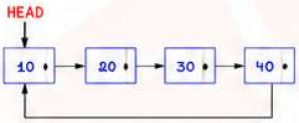
Step 4 : NEW → LINK = HEAD.

Step 5 : Stop.

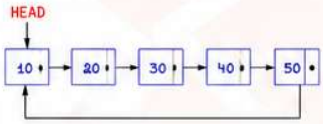
Time Complexity : O(n)

Example :

Before Insertion :



After Insertion (Insert 50) :



(C) Insertion After a Given Node :

Algorithim :

Step 1 : Search the required node.

Step 2 : Create a new node NEW.

Step 3 : NEW → LINK = TEMP → LINK.

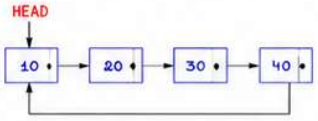
Step 4 : TEMP → LINK = NEW.

Step 5 : Stop.

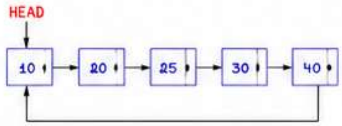
Time Complexity : O(n)

Example : Insert 25 after 20

Before Insertion :

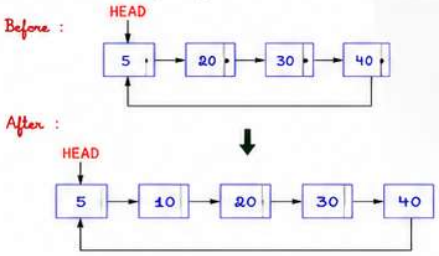


After Insertion :

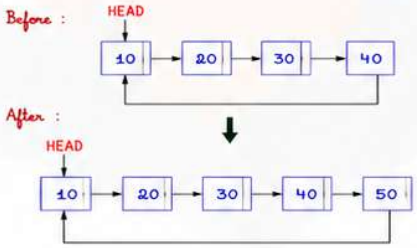


11. Important Diagrams (Insertions) :

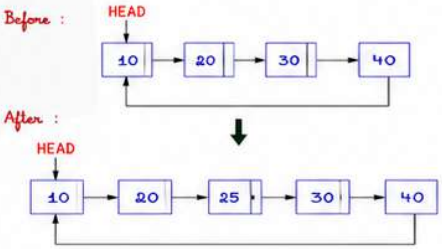
(A) Insert at Beginning (Insert 5)



(B) Insert at End (Insert 50)



(C) Insert After 20 (Insert 25)



Note :

- ★ In insertion at beginning, we must update LAST → LINK.
- ★ In insertion at end, NEW → LINK always points to HEAD.
- ★ In insertion after a node, only two links are changed.
- ★ All insertion operations in simple CLL take O(n) time.

Key Points :

- ★ Circular Linked List has no NULL at the end.
- ★ Last node always points to the first (HEAD).
- ★ We can start traversal from any node.
- ★ Insertion operations require proper pointer handling.

Apna College Bihar

12. Deletion Operations in Circular Linked List :

Deletion can be performed at :

★ From Beginning ★ From End

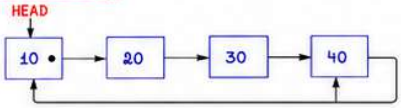
★ Deletion of a Given Node

(A) Deletion from Beginning :

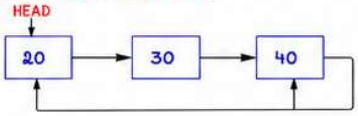
Algorithim :

Step 1 : If HEAD = NULL , stop.
Step 2 : Move LAST to last node.
Step 3 : TEMP = HEAD.
Step 4 : HEAD = HEAD → LINK.
Step 5 : LAST → LINK = HEAD.
Step 6 : Delete TEMP.
Step 7 : Stop.

Before Deletion :



After Deletion (delete 10) :



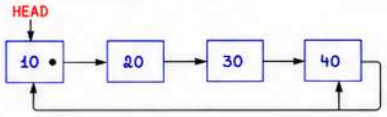
Time Complexity : O(n)

(B) Deletion from End :

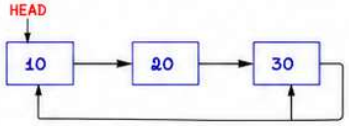
Algorithim :

Step 1 : If HEAD = NULL , stop.
Step 2 : Move to second last node.
Step 3 : SECOND_LAST → LINK = HEAD.
Step 4 : Delete LAST node.
Step 5 : Stop.

Before Deletion :



After Deletion (delete 40) :



Time Complexity : O(n)

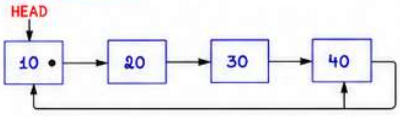
(C) Deletion of a Given Node :

Algorithim :

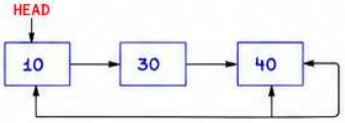
Step 1 : Search the required node.
Step 2 : Update the links.
Step 3 : Delete the node.
Step 4 : Stop.

Example : Delete 20

Before Deletion :



After Deletion (delete 20) :



Time Complexity : O(n)

13. Complexity Analysis :

Operation	Time Complexity
Traversing	O(n)
Searching	O(n)
Insertion at Beginning	O(n)
Insertion at End	O(n)
Insertion After a Node	O(n)
Deletion from Beginning	O(n)
Deletion from End	O(n)
Deletion of a Node	O(n)

14. Advantages :

★ Continuous traversal is possible.
★ Efficient implementation of circular queues.
★ Dynamic memory allocation.
★ Useful in round Robin CPU scheduling.
★ No NULL pointer at the end.
★ Suitable for cyclic processes.

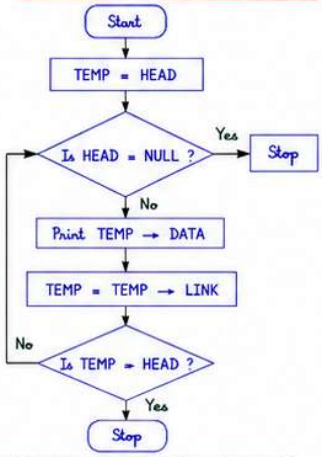
15. Disadvantages :

★ More complex implementation.
★ Infinite loop possible if condition is incorrect.
★ Searching takes O(n) time.
★ Pointer handling is difficult.

16. Applications :

★ Round Robin CPU Scheduling
★ Circular Queue
★ Multiplayer Games
★ Music Playlist
★ Traffic Signal Control
★ Operating Systems
★ Memory Management
★ Resource Sharing
★ Token Passing Network.

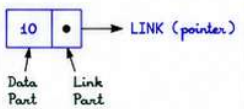
17. Circular Linked List Flowchart :



Important Notes :

★ In a Circular Linked List, last node points to the first node.
★ There is no NULL pointer at the end of the list.
★ Traversal starts from any node and stops when we reach HEAD again.
★ Proper pointer handling is required to avoid infinite loops.
★ All basic operations in simple CLL take O(n) time.

Legend :



Summary :

Circular Linked List is a linked list in which the last node connects to the first node, forming a circle. It is used in many real life applications like CPU scheduling, circular queues, games etc.

Apna College Bihar

Linear Search and Binary Search Techniques and Complexity Analysis

1. Introduction :

Searching is the process of finding a specific element (called the key) from a collection of data. It is one of the most important operations in computer science.

The two most common searching techniques are :

- ★ Linear Search
- ★ Binary Search

2. Searching :

Definition :

Searching is the process of locating a required element in a data structure and determining whether it exists.

Applications :

- ★ Database searching
- ★ Contact list searching
- ★ Student record management
- ★ Library management system
- ★ Search engines
- ★ File management systems
- ★ Information retrieval
- ★ Many real life applications

3. Linear Search :

Definition :

Linear Search is the simplest searching technique in which each element is checked one by one until the required element is found or the list ends.

Algorithm :

Step 1 : Start from the first element.
Step 2 : Compare the current element with the key.
Step 3 : If both are equal, display "Element Found".
Step 4 : Otherwise move to the next element.
Step 5 : Repeat until the last element.
Step 6 : If the key is not found, display "Element Not Found".
Step 7 : Stop.

Example :

Array :

10	25	.	40	50	60
0	1	2	3	4	5

Search Key = 40

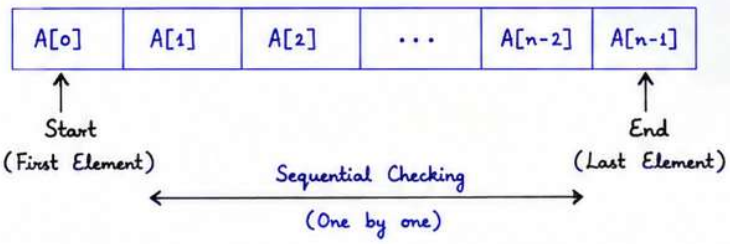
Comparisons :

10 ✗→ 25 ✗→ 35 ✗→ 40 ✓

Result :

Element Found at Position 4

4. Basic Searching Illustration :



5. Key Points about Linear Search :

- ★ Works on both sorted and unsorted data.
- ★ Checks each element sequentially.
- ★ Simple but slow for large datasets.
- ★ No extra memory required.
- ★ Suitable for small datasets.

Apna College Bihar

6. Time Complexity of Linear Search :

- ★ Best Case : O(1) (Key is at first position)
- ★ Average Case : O(n/2) ≈ O(n)
- ★ Worst Case : O(n) (Key is at last position or not present)
- ★ Space Complexity : O(1)

> **Note** : In Linear Search, we may have to check all n elements in worst case.

7. Advantages of Linear Search :

- ★ Very simple and easy to implement.
- ★ Works on both sorted and unsorted data.
- ★ No extra memory required.
- ★ Suitable for small datasets.

8. Disadvantages of Linear Search :

- ★ Slow for large datasets.
- ★ Requires more comparisons.
- ★ Not efficient for repeated searching.

9. Binary Search :

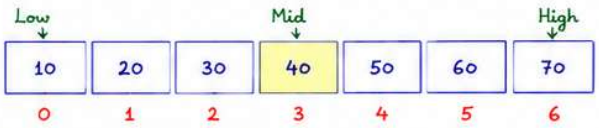
Definition :

Binary Search is a searching technique that works on sorted data. It repeatedly divides the search space into two halves until the required element is found.

Condition :

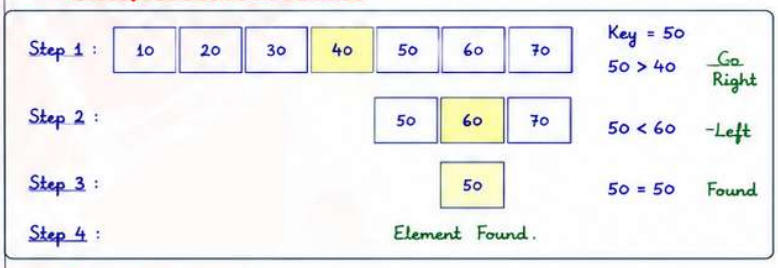
> Data must be sorted in ascending or descending order.

10. Basic Idea of Binary Search :



- ★ Compare KEY with middle element A[MID].
- ★ If KEY = A[MID] → Found.
- ★ If KEY < A[MID] → Search in left half.
- ★ If KEY > A[MID] → Search in right half.

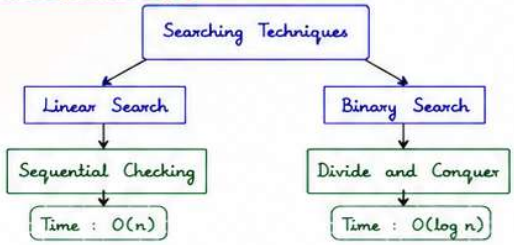
11. Binary Search Illustration :



12. Types of Data Suitable for Searching :

Linear Search	Binary Search
• Sorted Data ✓	• Sorted Data ✓
• Unsorted Data ✓	• Unsorted Data ✗
• Small Data ✓	• Small Data ✓
• Large Data ✗ (Slow)	• Large Data ✓ (Fast)

13. Searching Hierarchy :



14. Important Points :

- ★ Linear Search is simple but slow.
- ★ Binary Search is fast but works only on sorted data.
- ★ Binary Search follows Divide and Conquer approach.
- ★ Binary Search reduces time complexity from O(n) to O(log n).
- ★ Choice of algorithm depends on data size and sorting.

Quick Summary :

Linear Search	Binary Search
• Simple	• Requires sorted data
• Works on all data	• Fast Search
• Time Complexity : O(n)	• Time Complexity : O(log n)

Apna College Bihar

15. Binary Search Algorithm :

Step 1 : Set LOW = 0 and HIGH = n - 1.
Step 2 : Find MID = (LOW + HIGH) / 2.
Step 3 : If KEY = A[MID], display "Found" and Stop.
Step 4 : If KEY < A[MID], set HIGH = MID - 1.
Step 5 : If KEY > A[MID], set LOW = MID + 1.
Step 6 : Repeat Steps 2 to 5 until LOW > HIGH.
Step 7 : If key is not found, display "Element Not Found".
Step 8 : Stop.

16. Time Complexity of Binary Search :

- ★ Best Case : O(1) (Key is middle element)
- ★ Average Case : O(log n)
- ★ Worst Case : O(log n)
- ★ Space Complexity : O(1)

17. Advantages of Binary Search :

- ★ Much faster than Linear Search.
- ★ Requires only O(log n) time.
- ★ Efficient for large datasets.
- ★ Fewer comparisons.

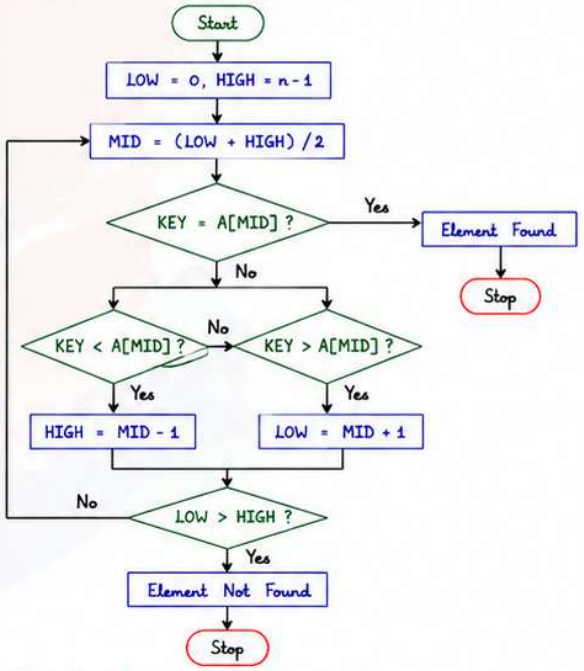
18. Limitations / Disadvantages :

- ★ Works only on sorted data.
- ★ Requires sorting before searching.
- ★ Insertion and deletion in sorted list can be costly.
- ★ Not suitable for linked list (requires random access).

19. Linear Search vs Binary Search (Comparison Table) :

Feature	Linear Search	Binary Search
Data Requirement	Sorted / Unsorted	Sorted Only
Approach	Sequential	Divide and Conquer
Best Case Time	O(1)	O(1)
Average Case Time	O(n)	O(log n)
Worst Case Time	O(n)	O(log n)
Space Complexity	O(1)	O(1)
Speed	Slow	Fast
Implementation	Very Easy	Moderate
Suitable For	Small or Unsorted Data	Large Sorted Data

20. Flowchart for Binary Search :



21. Applications of Searching :

- ★ Database Management Systems
- ★ Student Record Searching
- ★ Library Management Systems
- ★ File Searching in OS
- ★ Search Engines (Google, Bing)
- ★ Contact List Searching
- ★ Product Search in E-commerce
- ★ Inventory Management
- ★ Banking Applications
- ★ Dictionary / Spell Check

22. Complexity Summary :

Algorithm	Best Case	Average Case	Worst Case	Space Complexity
Linear Search	O(1)	O(n)	O(n)	O(1)
Binary Search	O(1)	O(log n)	O(log n)	O(1)

23. Key Takeaways (Summary) :

- ★ Linear Search checks elements one by one.
- ★ Binary Search divides the search space into halves.
- ★ Binary Search is much more efficient for large, sorted data.
- ★ Choice of algorithm depends on data size and sorting.
- ★ Binary Search follows Divide and Conquer approach.

Apna College Bihar

Objective and Properties of Different Sorting Algorithms

1. Introduction :

Sorting is the process of arranging the data (elements) in a particular order, either in ascending order or descending order. Sorting is one of the fundamental operations in computer science and it is widely used in many applications.

2. Definition of Sorting :

Sorting is the process of arranging a list of elements in a specific order (ascending or descending) based on some condition.

3. Example of Sorting :

Unsorted Array : 45 | 12 | 67 | 23 | 89 | 10

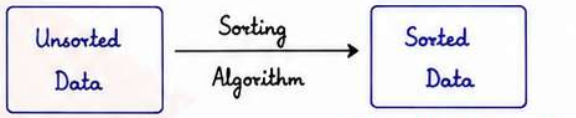
Ascending Order : 10 | 12 | 23 | 45 | 67 | 89

Small → Big

Descending Order : 89 | 67 | 45 | 23 | 12 | 10

Big → Small

4. Basic Idea of Sorting :



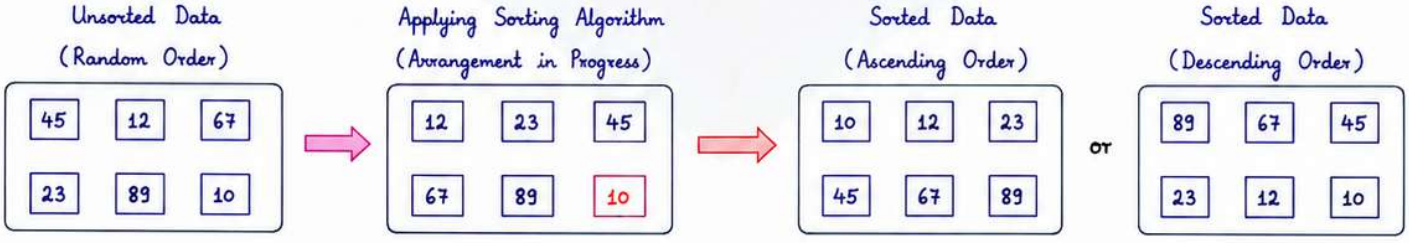
(Random Order)

(Ascending / Descending Order)

5. Objectives of Sorting :

1. Arrange data in a logical order.
2. Make searching operations faster and efficient.
3. Improve data organization and readability.
4. Simplify data analysis and processing.
5. Increase efficiency of other operations.
6. Reduce time required for data retrieval.
7. Improve performance of computer systems.
8. Support efficient database operations.

6. Sorting Process Illustration :



Note :

Sorting improves the efficiency of searching algorithms and helps in managing large amount of data efficiently.

Important :

The choice of sorting algorithm depends on the size of data, order of data, memory and time requirements.

Apna College Bihar

7. Stability in Sorting :

A sorting algorithm is said to be stable if two equal elements appear in the same relative order in the sorted output.

Example :

Before Sorting :

(25,A)	(18,B)	(25,C)	(12,D)	(18,E)

After Stable Sorting (Ascending) :

(12,D)	(18,B)	(18,E)	(25,A)	(25,C)

Here, A comes before C (same for B before E).
So, the algorithm is Stable.

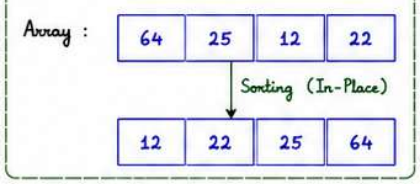
8. In-Place Sorting :

An in-place sorting algorithm uses a constant amount of extra memory space (O(1)).

Examples :

- ★ Selection Sort
- ★ Bubble Sort
- ★ Insertion Sort
- ★ Heap Sort

In-Place Illustration :



Only a small number of variables are used, no extra array is required.

9. Adaptive Sorting :

Adaptive sorting algorithms perform better when the input data in already partially sorted.

Examples :

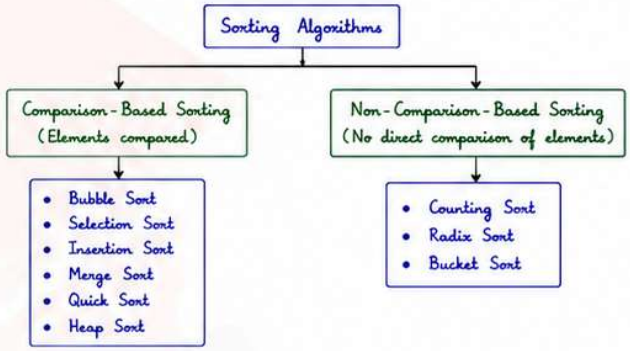
- ★ Insertion Sort
- ★ Bubble Sort (Optimized).

Partially Sorted Data :

10	26	30	25	40	50

Adaptive algorithms take less time for such data.

10. Comparison-Based vs Non-Comparison-Based Sorting :



11. Time Complexity :

Time complexity is used to measure the efficiency of an algorithm.

Case	Meaning
Best Case	Minimum time taken by the algorithm.
Average Case	Average time taken for all possible inputs.
Worst Case	Maximum time taken by the algorithm.

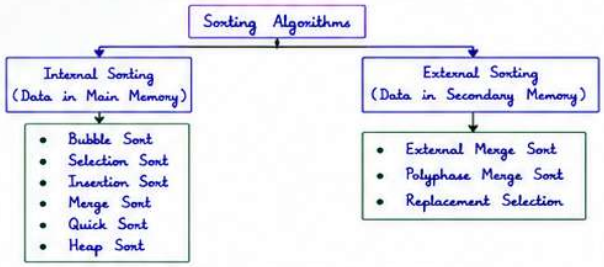
Example (Bubble Sort) :

Best Case : O(n) (Already sorted)
Average Case : $O(n^2)$
Worst Case : $O(n^2)$ (Reverse sorted)

12. Summary of Properties :

Property	Description	Example Algorithms
Stability	Maintains relative order of equal elements.	Merge Sort, Insertion Sort.
In-Place	Uses constant extra memory.	Selection Sort, Heap Sort
Adaptive	Works faster on partially sorted data.	Insertion Sort, Bubble Sort
Comparison-Based	Sorting done by comparing elements.	Quick Sort, Merge Sort
Non-Comparison-Based	Does not compare elements directly.	Counting Sort, Radix Sort

13. Sorting Algorithm Classification (Summary) :



Important Note :
Choice of sorting algorithm depends on data size, memory, stability requirement and time efficiency.

Apna College Bihar

14. Best, Average and Worst Case :

The running time of a sorting algorithm depends on the arrangement of input data.

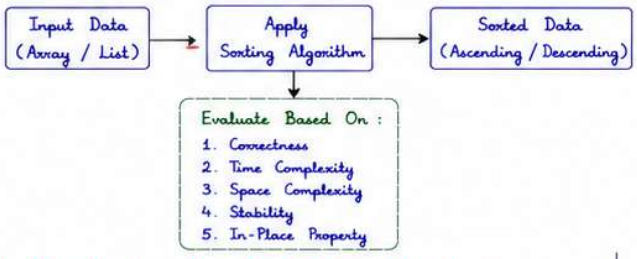
Case	Meaning
Best Case	Minimum number of comparisons / operations.
Average Case	Average number of comparisons / operations for all possible inputs.
Worst Case	Maximum number of comparisons / operations.

15. Comparison Table (Time Complexity Summary) :

Algorithm	Best Case	Average Case	Worst Case
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$
Radix Sort	$O(nk)$	$O(nk)$	$O(nk)$
Bucket Sort	$O(n + k)$	$O(n + k)$	$O(n^2)$

Where, n = number of elements, k = range / number of buckets / digits.

16. Flow of Sorting Evaluation :



17. Space Complexity (Extra Memory Used) :

Algorithm	Extra Space
Bubble Sort	$O(1)$
Selection Sort	$O(1)$
Insertion Sort	$O(1)$
Merge Sort	$O(n)$
Quick Sort	$O(\log n)$ [Average] $O(n)$ [Worst]
Heap Sort	$O(1)$
Counting Sort	$O(n + k)$
Radix Sort	$O(n + k)$
Bucket Sort	$O(n + k)$

Where,

n = number of elements

k = range / buckets / digits

18. Internal Sorting vs External Sorting :

Feature	Internal Sorting	External Sorting
Definition	Sorting done in main memory (RAM).	Sorting done when data does not fit in RAM.
Data Size	Small to moderate.	Very large.
Speed	Fast.	Slow.
Memory	Uses main memory.	Uses secondary memory.
Example Algorithms	Bubble Sort, Quick Sort, Merge Sort, Heap Sort.	External Merge Sort, Polyphase Merge Sort.
Example Use	Student records, small databases, arrays.	Big databases, large files, data warehouses.

19. Applications of Sorting Algorithms :

- ★ Database management systems
- ★ Banking and financial systems
- ★ Inventory and stock management
- ★ Student record management
- ★ File management systems
- ★ Search engines
- ★ Data analysis and reporting
- ★ E-commerce websites
- ★ Operating systems
- ★ Artificial Intelligence and ML.

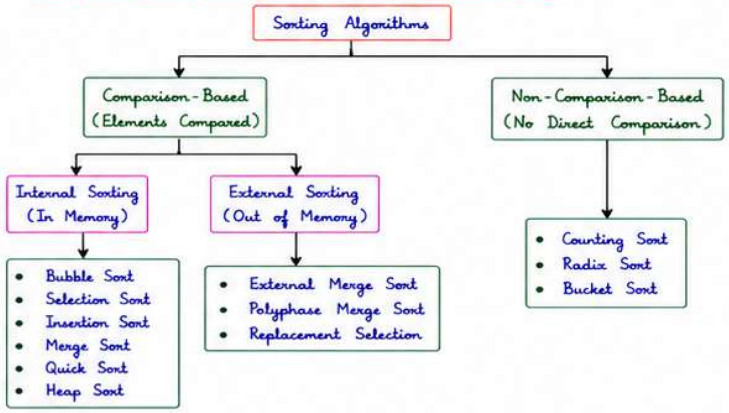
20. Characteristics of a Good Sorting Algorithm :

1. Correctness (Always produces correct output).
2. Efficiency (Minimum time and space complexity).
3. Stability (Maintains order of equal elements).
4. In-Place property (Uses minimal extra memory).
5. Adaptability (Works better on partially sorted data).
6. Scalability (Works well for large datasets).
7. Simplicity (Easy to understand and implement).
8. Reliability (Consistent performance for all inputs).

Summary :

Sorting arranges data in a particular order. Different sorting algorithms have different properties, time complexity and applications.

21. Sorting Algorithms Classification (Hierarchy) :





Apna College Bihar

Selection Sort , Bubble Sort , Insertion Sort , Quick Sort , Merge Sort , Heap Sort

Introduction :

Sorting algorithms are used to arrange the elements of an array or list in ascending or descending order. Different sorting algorithms follow different strategies and have different efficiencies.

The six most commonly used sorting algorithms are shown above.

1. Selection Sort :

→ **Definition :** Selection Sort repeatedly finds the smallest element from the unsorted part of the array and places it at the correct position (at the beginning of the unsorted part).

→ **Algorithm :**

① Find the smallest element in the unsorted part of the array.
② Swap it with the first unsorted element.
③ Move the boundary one position forward.
④ Repeat the above steps until the array is sorted.
⑤ Stop.

Time Complexity :

★ Best Case = $O(n^2)$
★ Average Case = $O(n^2)$
★ Worst Case = $O(n^2)$

→ **Example :**

Before Sorting :

64	25	12	22	11

⇨

After Sorting (Ascending) :

11	12	22	25	64

Advantages :

★ Simple to implement.
★ Performs fewer swaps.

Disadvantages :

★ Slow for large datasets.
★ Not a stable sorting algorithm.

Selection Sort Process (Ascending Order) :

Initial Array :	64	25	12	22	11	Smallest = 11 (Swap 64 and 11)
Pass 1 :	11	25	12	22	64	Smallest = 12 (Swap 25 and 12)
Pass 2 :	11	12	25	22	64	Smallest = 22 (Swap 25 and 22)
Pass 3 :	11	12	22	25	64	Smallest = 25 (Swap 25 and 25)
Pass 4 :	11	12	22	25	64	Smallest = 64 (Swap 64 and 64)

Array is Sorted

Apna College Bihar

2. Bubble Sort :

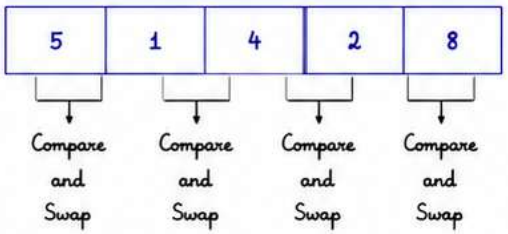
Definition : Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order.

Algorithm :

1. Compare adjacent elements.
2. If they are in wrong order, swap them.
3. Move to the next pair.
4. After first pass, the largest element moves to the last position.
5. Repeat the above steps for remaining elements.
6. Stop when no swaps are required in a complete pass.

Working Concept (One Pass)

Compare adjacent elements



★ In each pass, the largest element bubbles up to its correct position.

Time Complexity :

- ★ Best Case = O(n) (Already Sorted)
- ★ Average Case = $O(n^2)$
- ★ Worst Case = $O(n^2)$
- ★ Space Complexity = O(1) (In-Place)

Example (Pass-wise) :

Initial Array :	5	1	4	2	
Pass 1 :	1	4	2	5	8 (8 is in correct position)
Pass 2 :	1	2	4	5	8 (5 is in correct position)
Pass 3 :	1	2	4	5	8 (4 is in correct position)
Pass 4 :	1	2	4	5	8 (No swaps, array is sorted)
Sorted Array :	1	2	4	5	8

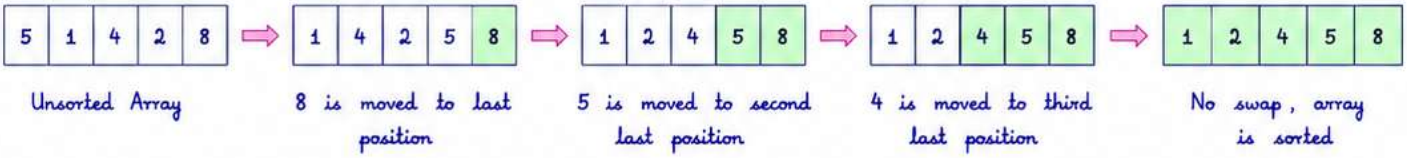
Advantages :

- ★ Very easy to understand.
- ★ Easy to implement.
- ★ Stable sorting algorithm.
- ★ Requires only one extra variable (Swapping).

Disadvantages :

- ★ Slow for large datasets.
- ★ Not efficient (Time complexity $O(n^2)$).

How Bubble Sort Works (Ascending Order) :



Note :
Bubble Sort works by repeatedly "bubbling up" the largest element to the end of the array in each pass.

Apna College Bihar

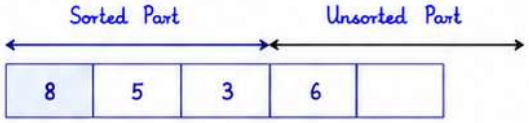
3. Insertion Sort :

Definition : Insertion Sort builds the sorted array one element at a time by inserting each element into its correct position in the sorted part.

Algorithm :

1. Assume the first element is sorted.
2. Pick the next element.
3. Compare it with the elements in the sorted part.
4. Shift all greater elements one position ahead.
5. Insert the element at its correct position.
6. Repeat until all elements are sorted.

Working Concept :



Initially, first element (8) is considered sorted. We take next element (5) and insert it into its correct position in the sorted part.

Time Complexity :

- ★ Best Case = O(n) (Already Sorted)
- ★ Average Case = $O(n^2)$
- ★ Worst Case = $O(n^2)$
- ★ Space Complexity = O(1) (In-Place)

Example :

Initial Array : 8 | 5 | 4 | 6

Pass 1 : 8 | 5 | 3 | 4 | 6 ⇒ 5 < 8 → Shift 8 to right, Insert 5 at position 1
⇒ 5 | 8 | 3 | 4 | 6

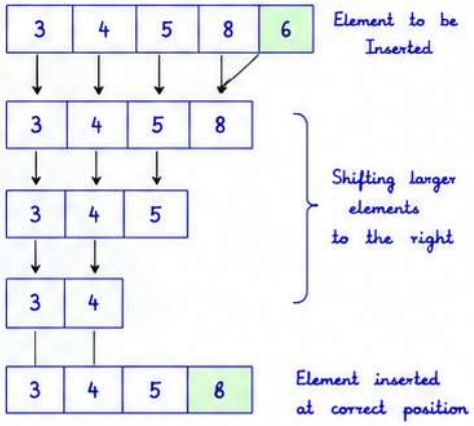
Pass 2 : 5 | 8 | 3 | 4 | 6
3 < 8 → Shift 8
3 < 5 → Shift 5
⇒ Insert 3 at position 1

Pass 3 : 3 | 5 | 8 | 4 | 6
4 < 8 → Shift 8
4 < 5 → Shift 5
4 > 3 → Insert 4 here

Pass 4 : 3 | 4 | 5 | 8 | 6
6 < 8 → Shift 8
6 > 5 → Insert 6 here

Sorted Array : 3 | 4 | 5 | 6 | 8 — Array is Sorted

Illustration (Insertion Process) :



Advantages :

- ★ Simple and easy to implement.
- ★ Stable sorting algorithm.
- ★ Adaptive (Works well for nearly sorted arrays).
- ★ Efficient for small datasets.

Disadvantages :

- ★ Not efficient for large datasets.
- ★ Time complexity is $O(n^2)$ in average and worst cases.
- ★ More number of shifts are required.

Applications :

- ★ Used in small datasets.
- ★ Used in online (real-time) sorting.
- ★ Used in adaptive sorting.
- ★ Used in embedded systems.

Note :

Insertion Sort is efficient for small or nearly sorted data, but it becomes slow as the size of the data increases.

Apna College Bihar

4. Quick Sort :

Definition : Quick Sort is a Divide and Conquer sorting algorithm. It selects a pivot element and partitions the array into two parts such that elements smaller than pivot are on left and greater elements are on right.

Time Complexity :

- ★ Best Case = $O(n \log n)$ (Balanced Partition)
- ★ Average Case = $O(n \log n)$
- ★ Worst Case = $O(n^2)$ (Unbalanced Partition)
- ★ Space Complexity = $O(\log n)$ (Due to Recursion Stack)

Algorithm :

1. Choose a pivot element from the array.
2. Partition the array into two parts :
   - Left part contains elements smaller than pivot.
   - Right part contains elements greater than pivot.
3. Recursively apply Quick Sort on the left part.
4. Recursively apply Quick Sort on the right part.
5. Repeat until arrays are of size 0 or 1.

Pivot Concept and Partitioning :

Let the Pivot be the last element of the array.

Example : Array =

10	80	30	90	40	50	70

Pivot

Partitioning Steps :

- Compare each element with pivot.
- Place smaller elements on left side.
- Place greater elements on right side.

After Partition :

10	30	40	50	70	80	90

Smaller Elements | Pivot | Greater Elements

Example of Quick Sort :

Initial Array :

10	80	30	90	40	50	70

Step 1 : Pivot = 70 (last element)

Partition →

10	30	40	50	70	80	90

Left Sub-array [10, 30, 40, 50] | Right Sub-array [80, 90]

Step 2 : Sort Left Sub-array [10, 30, 40, 50]

Pivot = 50 →

10	30	40	50

Step 3 : Sort [10, 30, 40]

Pivot = 40 →

10	30	40

Step 4 : Sort [10, 30]

Pivot = 30 →

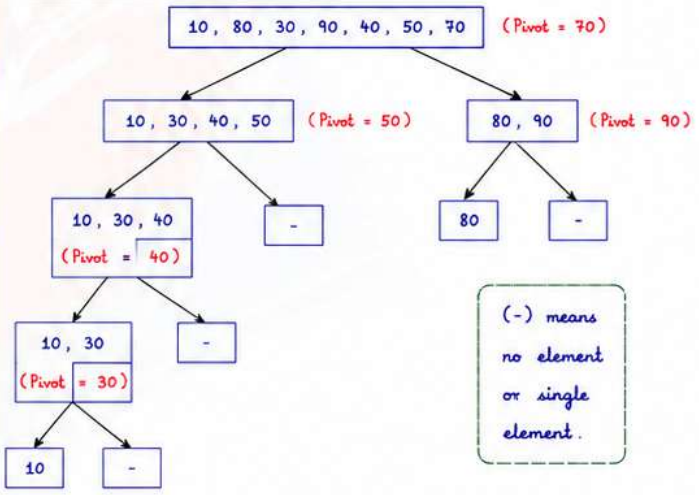
10	30

Single element arrays are already sorted.

Final Sorted Array :

10	30	40	50	70	80	90

Recursion Tree of Quick Sort :



(-) means no element or single element.

Advantages :

- ★ Very fast in practice.
- ★ Efficient for large datasets.
- ★ In-place sorting (requires little extra memory).
- ★ Good cache performance.

Disadvantages :

- ★ Worst case time complexity is $O(n^2)$.
- ★ Not stable sorting algorithm.
- ★ Performance depends on pivot selection.
- ★ Recursion causes stack overhead.

Applications :

- ★ Used in operating systems (process scheduling).
- ★ Used in database systems.
- ★ Used in searching and data organization.
- ★ Used in various programming libraries.

Apna College Bihar

5. Merge Sort :

Definition : Merge Sort is a Divide and Conquer sorting algorithm. It divides the array into two halves, recursively sorts both halves and then merges them to produce a single sorted array.

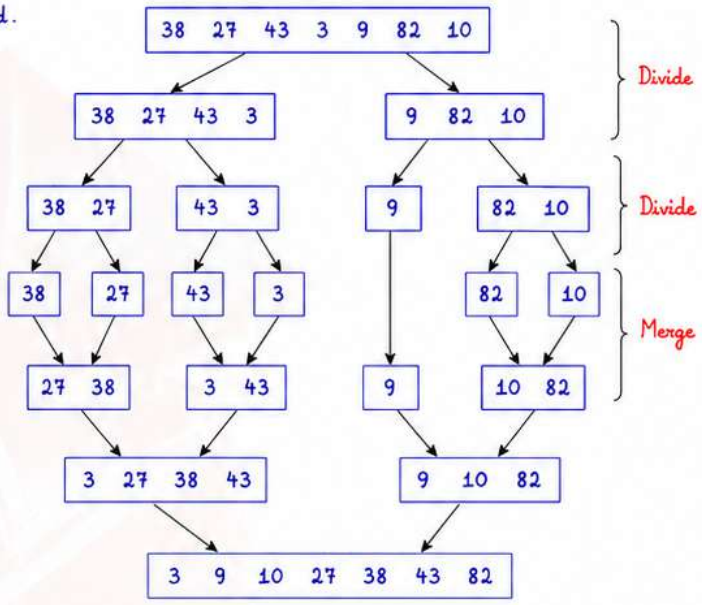
Time Complexity :

- ★ Best Case = O(n log n)
- ★ Average Case = O(n log n)
- ★ Worst Case = O(n log n)
- ★ Space Complexity = O(n) (Extra Space)

Algorithm :

1. If the array has 0 or 1 element, it is already sorted.
2. Divide the array into two halves.
3. Recursively sort the left half.
4. Recursively sort the right half.
5. Merge the two sorted halves.
6. Repeat until the whole array is sorted.

Divide and Merge Process :



Example :

Initial Array : 38 | 27 | 43 | 3 | 9 | 82 | 10

Step 1 : Divide — 38 27 43 3 | 9 82 10

Step 2 : Divide — 38 27 | 43 3 | 9 | 82 10

Step 3 : Divide — 38 | 27 | 43 | 3 | 9 | 82 | 10

Step 4 : Merge — 27 38 | 3 43 | 9 | 10 82

Step 5 : Merge — 3 27 38 43 | 9 10 82

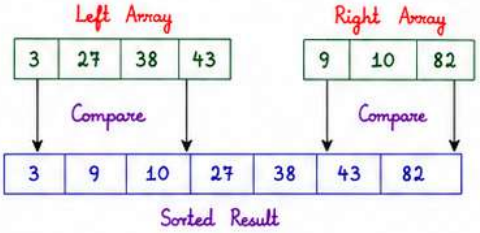
Step 6 : Merge — 3 9 10 27 38 43 82

Properties :

- ★ Stable sorting algorithm.
- ★ Works on Divide and Conquer approach.
- ★ Requires extra space for merging.
- ★ Guaranteed O(n log n) in all cases.

How Merge Operation Works.

Example : Merge [3 27 38 43] and [9 10 82]



Advantages :

- ★ Consistent O(n log n) performance.
- ★ Stable sorting algorithm.
- ★ Suitable for large data sets.
- ★ Works well with linked lists.

Disadvantages :

- ★ Requires extra memory O(n).
- ★ More complex than simple sorts.
- ★ Not an in-place sorting algorithm.

Note :

Merge Sort is very efficient for large data sets as its time complexity is O(n log n) in best, average and worst cases. It uses extra space for merging the sorted sub-arrays.

Apna College Bihar

6. Heap Sort :

Definition : Heap Sort is a comparison based sorting algorithm that uses Binary Heap data structure. It first builds a Max Heap from the array and then repeatedly extracts the maximum element and places it at the end of the array.

Step 1 : Build Max Heap (Bottom-Up Heapify)

Initial Array → After Heapify (i=2) → After Heapify (i=1) → Max Heap (Completed)

12	11	13	5	6	7

→

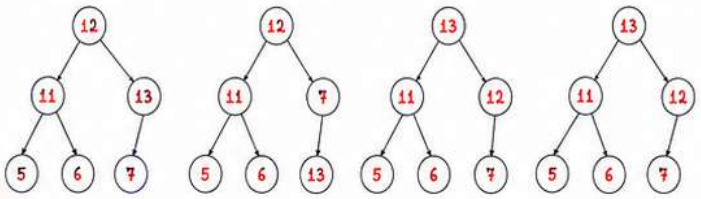
12	11	7	5	6	13

→

13	11	7	5	6	12

→

13	11	12	5	6	7



Algorithm :

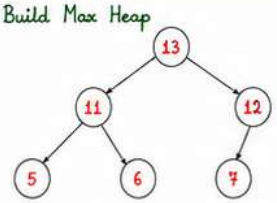
1. Build a Max Heap from the array.
2. Swap the root (maximum element) with the last element of heap.
3. Reduce the size of heap by one.
4. Heapify the root to restore Max Heap property.
5. Repeat steps 2 to 4 until the heap size becomes 1.

Example :

Initial Array :

12	11	13	5	6	7

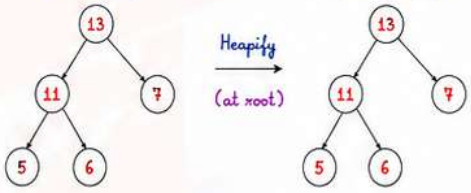
Step 1 : Build Max Heap



Max Heap Constructed

Heapify (Maintain Max Heap Property) :

Given Heap → Heapify (at root) → After Heapify



Root is greater than its children so heap property is satisfied.

Heap Sort Process :

Step	Action	Array	Heap Size
1	Swap root with last element	7 11 12 5 6 13	5
2	Heapify the root	12 11 7 5 6 13	5
3	Swap root with last element	6 11 7 5 12 13	4
4	Heapify the root	11 6 7 5 12 13	4
5	Swap root with last element	5 6 7 11 12 13	3
6	Heapify the root	7 6 5 11 12 13	3
7	Swap root with last element	5 6 7 11 12 13	2
8	Heapify the root	6 5 7 11 12 13	2
9	Swap root with last element	5 6 7 11 12 13	1

Sorted Array :

5	6	7	11	12	13

Time Complexity :

- ★ Best Case = O(n log n)
- ★ Average Case = O(n log n)
- ★ Worst Case = O(n log n)
- ★ Space Complexity = O(1) (In-Place)

Properties :

- ★ In-place sorting algorithm.
- ★ Not a stable sorting algorithm.
- ★ Uses Binary Heap (Max Heap).
- ★ Good performance in all cases.

Advantages :

- ★ Guaranteed O(n log n) in all cases.
- ★ In-place sorting.
- ★ Efficient for large datasets.
- ★ Does not require extra memory.

Disadvantages :

- ★ Not stable.
- ★ More complex than simple sorts.
- ★ Poor cache performance compared to Quick and Merge Sort.

Applications :

- ★ Used in Operating Systems.
- ★ Used in Priority Queues.
- ★ Used in Scheduling algorithms.
- ★ Used in Graph algorithms like Dijkstra's algorithm.

Note : Heap Sort is preferred when we need O(n log n) time complexity with constant extra space. It is very useful in systems where memory is limited.



Apna College Bihar

7. Comparison of Sorting Algorithms

Algorithm	Best Case	Average Case	Worst Case	Space Complexity	Stable	In-place
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ (Recursion Stack)	No	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$ (Extra Array)	Yes	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes

Advantages :

- ★ Sorting algorithms are used to arrange data in ascending or descending order.
- ★ Important in many applications like database, operating system, searching, etc.
- ★ Efficient algorithms (Quick, Merge, Heap) are suitable for large datasets.
- ★ Simple algorithms (Selection, Bubble, Insertion) are easy to implement and understand.

Disadvantages :

- ★ Simple sorts are slow for large datasets.
- ★ Some algorithms require extra space (Merge Sort).
- ★ Quick Sort has worst case $O(n^2)$.
- ★ Heap Sort and Quick Sort are not stable.
- ★ Recursive algorithms use stack space.

Applications :

- ★ Database systems - Sorting of records.
- ★ Operating Systems - Process scheduling, memory management.
- ★ Search Engines - Indexing and sorting of data.
- ★ Banking Systems - Sorting of transactions.
- ★ Data Analytics - Sorting large datasets.
- ★ Artificial Intelligence - Preprocessing of data.
- ★ E-commerce - Product sorting by price, rating, etc.
- ★ Compiler Design - Symbol table sorting.

In-place vs Not In-place

In-place Sorting

Uses constant extra space $O(1)$.

Algorithms:

- ★ Selection Sort
- ★ Bubble Sort
- ★ Insertion Sort
- ★ Quick Sort
- ★ Heap Sort

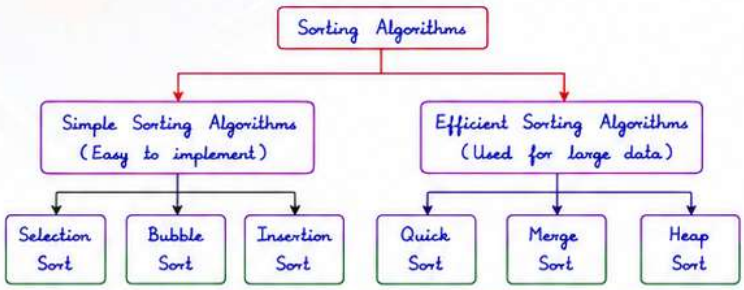
Not In-place Sorting

Requires extra space for sorting.

Algorithm:

- ★ Merge Sort

Sorting Algorithms Hierarchy



Summary Box

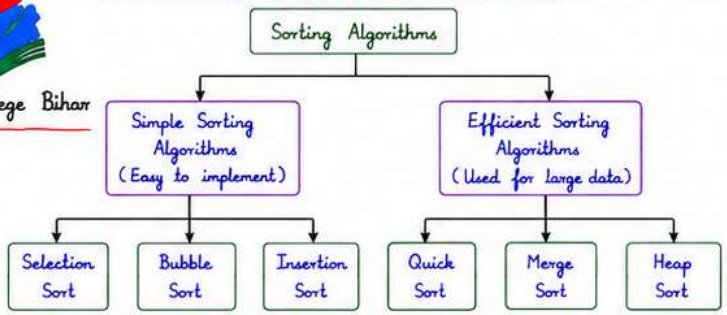
- ★ Bubble Sort and Insertion Sort are simple and stable.
- ★ Selection Sort is simple but not stable.
- ★ Quick Sort is fastest in practice but worst case $O(n^2)$.
- ★ Merge Sort is stable and has guaranteed $O(n \log n)$.
- ★ Heap Sort has good worst case and is in-place.
- ★ Choose algorithm based on size of data and requirement.

Important Exam Points

- ★ Remember Time Complexity of all algorithms.
- ★ Stable vs Not Stable.
- ★ In-place vs Not In-place.
- ★ Space Complexity.
- ★ Best, Average and Worst cases.
- ★ Suitable application of each algorithm.
- ★ Use diagrams for Quick, Merge and Heap Sort.
- ★ Comparison table is important.
- ★ Know advantages and disadvantages.

Apna College Bihar

8. Sorting Algorithms Hierarchy



Comparison Table of Sorting Algorithms

Algorithm	Best Case	Average Case	Worst Case	Space Complexity	Stable	In-place
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes

Note : Stable means relative order of equal elements is maintained.
In-place means no extra memory is used.

Advantages :

- ★ Helps in arranging data in a meaningful order.
- ★ Improves searching efficiency.
- ★ Used in many real-life applications.
- ★ Efficient algorithms handle large datasets.
- ★ Some algorithms are stable and in-place.

Disadvantages :

- ★ Simple algorithms are slow for large datasets.
- ★ Some algorithms require extra space.
- ★ Quick Sort has worst case $O(n^2)$.
- ★ Heap Sort and Quick Sort are not stable.
- ★ Complex algorithms are hard to implement.

Applications :

- ★ Database Management Systems.
- ★ Operating Systems (Process scheduling, memory mgmt.).
- ★ Searching Engines (Indexing and sorting).
- ★ Banking Systems (Transaction sorting).
- ★ Data Analytics (Large data sorting).
- ★ Artificial Intelligence (Preprocessing of data).
- ★ E-commerce (Product sorting by price, rating).
- ★ Compiler Design (Symbol table sorting).

Summary Box

- ★ Selection Sort - Repeatedly selects minimum and swaps.
- ★ Bubble Sort - Repeatedly swaps adjacent elements.
- ★ Insertion Sort - Inserts element in correct position.
- ★ Quick Sort - Divide and Conquer using Pivot.
- ★ Merge Sort - Divide, sort and merge.
- ★ Heap Sort - Uses Binary Heap (Max Heap).
- ★ Merge Sort is stable and guarantees $O(n \log n)$.
- ★ Quick Sort is fastest in practice.

Quick Revision Table

Algorithm	Key Idea
Selection Sort	Select minimum from unsorted part.
Bubble Sort	Swap adjacent elements.
Insertion Sort	Insert element in correct position.
Quick Sort	Partition around pivot and sort.
Merge Sort	Divide into halves and merge.
Heap Sort	Build Max Heap and remove max.

Sorting Algorithms Mind Map



Important Exam Points

- ★ Time complexity of all algorithms.
- ★ Stable vs Not Stable.
- ★ In-place vs Not In-place.
- ★ Best, Average and Worst cases.
- ★ Space complexity.
- ★ Key idea of each algorithm.
- ★ Applications of sorting algorithms.
- ★ Comparison table is very important.
- ★ How each algorithm works (steps).



Apna College Bihar

Performance and Comparison among All Sorting Methods

1. Introduction :

Different sorting algorithms have different performance depending on the size of the input, memory usage, stability, and time complexity. Choosing the appropriate sorting algorithm improves the efficiency of a program.

2. Performance of Sorting Algorithms :

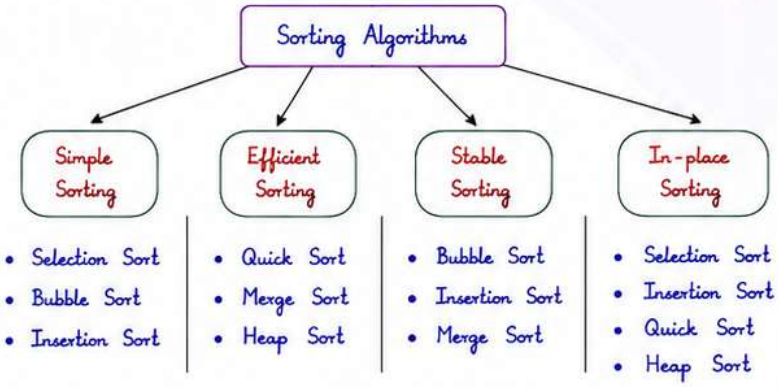
Definition : The performance of a sorting algorithm is measured by how efficiently it sorts data in terms of the following parameters.

- ★ Time Complexity
- ★ Space Complexity
- ★ Stability
- ★ In-place Property
- ★ Adaptiveness
- ★ Number of Comparisons
- ★ Number of Swaps

3. Time Complexity Comparison :

Algorithm	Best Case	Average Case	Worst Case
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

4. Basic Comparison Diagram :



5. Performance Parameters (Meaning) :

- ★ Time Complexity : Total time taken by algorithm to sort n elements.
- ★ Space Complexity : Extra memory used by the algorithm.
- ★ Stability : If equal elements maintain their relative order, algorithm is stable.
- ★ In-place Property : If sorting is done using constant extra space.
- ★ Adaptiveness : If algorithm works better for already sorted or nearly sorted data.
- ★ Comparisons : Number of times elements are compared during sorting.
- ★ Swaps : Number of times elements are swapped or moved.

Important Note :

Performance of an algorithm depends on data size, input order (sorted or unsorted), memory availability and application requirements.



Apna College Bihar

5. Space Complexity Comparison :

Algorithm	Space Complexity
Selection Sort	O(1)
Bubble Sort	O(1)
Insertion Sort	O(1)
Quick Sort	O(log n) (Average) O(n) (Worst)
Merge Sort	O(n)
Heap Sort	O(1)

6. Stability Comparison :

Algorithm	Stable ?
Selection Sort	No
Bubble Sort	Yes
Insertion Sort	Yes
Quick Sort	No
Merge Sort	Yes
Heap Sort	No

Note :
A sorting algorithm is Stable if equal elements maintain their original relative order after sorting.

7. In-place Comparison :

Algorithm	In-place ?
Selection Sort	Yes
Bubble Sort	Yes
Insertion Sort	Yes
Quick Sort	Yes
Merge Sort	No
Heap Sort	Yes

Note :
In-place algorithm uses very little extra memory for sorting.

8. Adaptive Comparison :

Algorithm	Adaptive ?
Selection Sort	No
Bubble Sort (Optimized)	Yes
Insertion Sort	Yes
Quick Sort	No
Merge Sort	No
Heap Sort	No

Note :
Adaptive algorithm performs better when input is already partially sorted.

9. Number of Swaps / Movements Comparison :

Algorithm	Comparisons	Swaps / Movements	Remarks
Selection Sort	High ($\sim n^2/2$)	Very Few ($n-1$)	Min. swaps but max. comparisons
Bubble Sort	High ($\sim n^2/2$)	Many	Large number of swaps
Insertion Sort	Medium ($\sim n^2/4$)	Few Shifts	Moves elements to correct position
Quick Sort	Low (Avg. $\sim n \log n$)	Moderate	Depends on pivot selection
Merge Sort	Low ($\sim n \log n$)	No Direct Swaps	Uses extra array for merging
Heap Sort	Low ($\sim n \log n$)	Moderate	Heapify operations cause swaps

Remember :
Swaps / Movements affect the speed of algorithm along with comparisons.

10. Comparison Based on Performance :

- ★ Selection Sort : Easy to implement. Performs minimum swaps. Slow for large datasets.
- ★ Bubble Sort : Very simple. Stable. Good only for small datasets.
- ★ Insertion Sort : Adaptive and stable. Efficient for nearly sorted data.
- ★ Quick Sort : Fastest in practice. Efficient for large datasets. Worst case is $O(n^2)$.
- ★ Merge Sort : Stable. Guaranteed $O(n \log n)$. Requires extra memory.
- ★ Heap Sort : In-place. Guaranteed $O(n \log n)$. Not stable.

11. Best Algorithm for Different Situations :

Situation	Recommended Algorithm
Small Dataset	Bubble Sort / Insertion Sort
Nearly Sorted Data	Insertion Sort
Large Dataset	Quick Sort
Guaranteed Performance	Merge Sort / Heap Sort
Low Memory Available	Heap Sort
Stable Sorting Required	Merge Sort / Insertion Sort

Apna College Bihar

12. Number of Swaps Comparison :

Algorithm	Swaps / Movements
Selection Sort	Very Few (n-1)
Bubble Sort	Many
Insertion Sort	Few Shifts
Quick Sort	Moderate
Merge Sort	No Direct Swaps (Uses Extra Array)
Heap Sort	Moderate

13. Performance of Each Sorting Algorithm :

★ Selection Sort : Easy to implement. Performs minimum swaps. Slow for large datasets.

★ Bubble Sort : Very simple. Stable. Good only for small datasets.

★ Insertion Sort : Adaptive. Stable. Efficient for nearly sorted data.

★ Quick Sort : Fastest in practice. Efficient for large datasets. Worst case is $O(n^2)$.

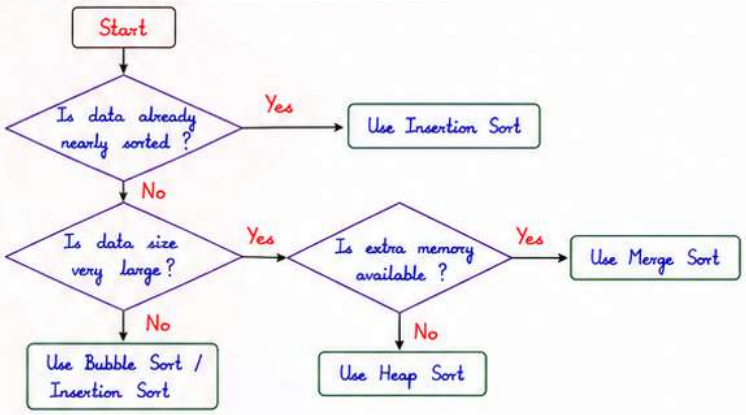
★ Merge Sort : Stable. Guaranteed $O(n \log n)$. Requires extra memory.

★ Heap Sort : In-place. Guaranteed $O(n \log n)$. Not stable.

14. Best Algorithm for Different Situations :

Situation	Recommended Algorithm
Small Dataset	Bubble Sort / Insertion Sort
Nearly Sorted Data	Insertion Sort
Large Dataset	Quick Sort
Guaranteed Performance	Merge Sort / Heap Sort
Low Memory Requirement	Heap Sort
Stable Sorting Required	Merge Sort / Insertion Sort

15. Best Algorithm Selection Flowchart :



16. Advantages of Comparison :

★ Helps choose the correct algorithm for a problem.

★ Improves execution speed of programs.

★ Optimizes memory usage.

★ Reduces processing time.

★ Enhances overall application performance.

17. Applications :

- Database Management Systems
- Banking Software
- Search Engines
- Data Analytics
- Artificial Intelligence
- Operating Systems
- E-commerce Websites
- Compiler Design

18. Overall Comparison Table :

Algorithm	Best Case	Average Case	Worst Case	Space Complexity	Stable	In-place	Adaptive	Swaps / Movements
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes	No	Very Few (n-1)
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes	Yes (Opt.)	Many
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes	Yes	Few Shifts
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ (Avg.)	No	Yes	No	Moderate
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No	No	No Direct Swaps
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes	No	Moderate

Summary Box :

★ Quick Sort is fastest on average for large datasets.

★ Merge Sort and Heap Sort guarantee $O(n \log n)$ in worst case.

★ Merge Sort is stable but needs extra memory.

★ Heap Sort is in-place but not stable.

★ Insertion Sort is best for small or nearly sorted data.

★ Bubble Sort is easy but inefficient for large datasets.

★ Selection Sort performs minimum number of swaps.

★ Choice depends on time, space, stability and data condition.

Apna College Bihar

Hashing

1. Introduction :

Hashing is a technique used to store and retrieve data quickly. It uses a Hash Function to convert a key into an index (address) of a table called a Hash Table. This allows searching, insertion and deletion operations to be performed very efficiently.

Applications : Databases, Compiler Design, Operating Systems, Dictionaries, Password Storage, Caching etc.

2. Hashing :

Definition :

Hashing is the process of converting a key into a fixed-size integer value (called Hash Value) using a Hash Function.

The generated hash value determines the location where the data will be stored.

3. Hash Table :

Definition : A Hash Table is a data structure that stores data using the index generated by a hash function.

Example :

Index	0	1	2	3	4	5	6
Data	-	12	45	23	-	67	89

Each element is stored at its calculated index.

4. Hash Function :

Definition : A Hash Function is a mathematical function that converts a key into an index of the hash table.

General Formula :

Hash(Key) = Key mod Table Size

Example : Table Size = 10

Keys = 25, 36, 42

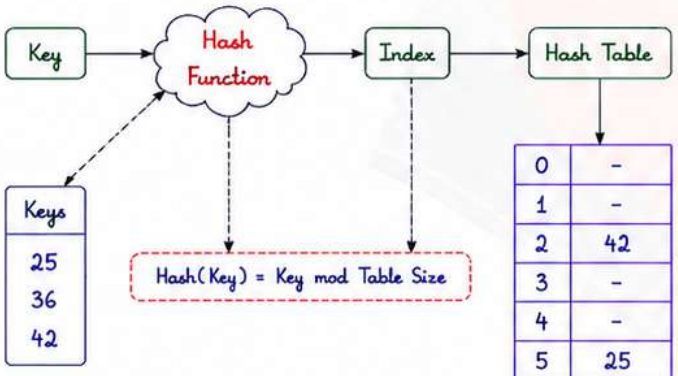
25 mod 10 = 5
36 mod 10 = 6
42 mod 10 = 2

Stored positions :
Index 2 → 42
Index 5 → 25
Index 6 → 36

Final Hash Table :

Index	0	1	2	3	4	5	6	7	8	9
Data	-	-	42	-	-	25	36	-	-	-

5. Simple Illustration of Hashing :



0	-
1	-
2	42
3	-
4	-
5	25
6	36
7	-
8	-
9	-

Important Note :

- ★ A good hash function distributes keys uniformly.
- ★ Collisions may occur even with good hash function.
- ★ Efficiency of hashing depends on hash function and table size.
- ★ Large table size reduces collisions.

Formula Box :

- ★ Hash(Key) = Key mod Table Size
- ★ Index = Hash(Key)
- ★ Load Factor (α) = $\frac{\text{Number of Elements}}{\text{Table Size}}$

Key Points :

- ✓ Hashing provides fast access to data.
- ✓ Hash Function converts key into index.
- ✓ Data is stored in Hash Table.
- ✓ Collisions must be handled using techniques.
- ✓ Average time complexity is O(1).

Apna College Bihar

5. Collision :

Definition : A Collision occurs when two or more keys are mapped to the same index in the hash table.

Example : Table Size = 10

25 mod 10 = 5

35 mod 10 = 5 → Both keys want to be stored at Index 5.

Collision Example :

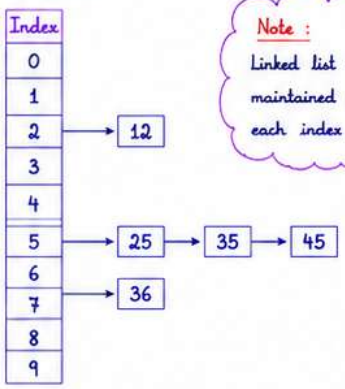
Index	0	1	2	3	4	5	6	7	8	9
Data	-	-	-	-	-	25,	35	-	-	-

6. Collision Resolution Techniques :

(A) Separate Chaining :

- Each table index stores a linked list of all elements that hash to the same index.
- Easy to implement.
- Handles many collisions.

Example :



Note : Linked list is maintained at each index.

(B) Linear Probing :

- If the calculated position is occupied, search the next available position sequentially.
- Simple and easy.

Example :

Table Size = 10

Insert : 25, 35, 45

25 mod 10 = 5 → Index 5 (Empty) → Place 25

35 mod 10 = 5 → Index 5 (Occupied)
→ Check 6 (Empty) → Place 35

45 mod 10 = 5 → Index 5,6 (Occupied)
→ Check 7 (Empty) → Place 45

Index	0	1	2	4	5	5	6	7	8	9
Data	-	-	-	-	-	25	35	45	-	-

(C) Quadratic Probing :

- Next position is found using square increments.
- Reduces primary clustering.

Formula :

$$\text{Index} = (\text{Hash} + i^2) \bmod \text{Table Size}$$

where $i = 0, 1, 2, 3, \ldots$

Example :

Table Size = 10

Insert : 25, 35, 45

Hash = key mod 10

All keys hash to 5.

Insert 25 → Index 5

Insert 35 → $(5 + 1^2) \bmod 10 = 6$

Insert 45 → $(5 + 2^2) \bmod 10 = 9$

Index	0	1	2	3	4	5	6	7	8	9
Data	-	-	-	-	-	25	35	-	-	45

(D) Double Hashing :

- A second hash function is used to calculate the next position.
- Reduces clustering and gives better distribution.

Formula :

$$\text{New Index} = (\text{Hash1} + i \times \text{Hash2}) \bmod \text{Table Size}$$

where $i = 0, 1, 2, 3, \ldots$

Example :

Table Size = 10

Let, Hash1 = Key mod 10

Hash2 = 7 - (Key mod 7)

Insert : 25, 35, 45

Ex 25 → Hash1 = 5 , Hash2 = 7 - (25 mod 7) = 3

$i = 0$ → $(5 + 0 \times 3) \bmod 10 = 5$ → Place 25

Ex 35 → Hash1 = 5 , Hash2 = 7 - (35 mod 7) = 2

$i = 0$ → $(5 + 0 \times 2) \bmod 10 = 5$ (Occupied)

$i = 1$ → $(5 + 1 \times 2) \bmod 10 = 7$ → Place 35

Ex 45 → Hash1 = 5 , Hash2 = 7 - (45 mod 7) = 4

$i = 0$ → $(5 + 0 \times 4) \bmod 10 = 5$ (Occupied)

$i = 1$ → $(5 + 1 \times 4) \bmod 10 = 9$ → Place 45

Index	0	1	2	3	4	5	6	7	8	9
Data	-	-	-	-	-	25	-	35	-	45

7. Comparison of Collision Resolution Techniques :

Technique	Idea / Method	Advantages	Disadvantages
Separate Chaining	Linked list at each index.	Easy to implement. Handles many collisions.	Requires extra memory.
Linear Probing	Next available position.	Simple, easy. Cache friendly.	Primary clustering may occur.
Quadratic Probing	Square increment search.	Reduces primary clustering.	Secondary clustering may occur.
Double Hashing	Second hash function.	Best distribution. Very less clustering.	More computation due to 2 hash functions.

Remember :

★ Good hash function + proper collision handling = Efficient Hashing.

★ Load factor should be kept low for better performance.

★ Hashing is widely used in real life applications.



Apna College Bihar

8. Quadratic Probing (Example in Detail) :

Formula : Index = (Hash + i^2) mod Table Size

where i = 0, 1, 2, 3, ...

Example : Table Size = 10

Insert : 25, 35, 45, 55

Hash = key mod 10

All keys hash to 5.

Steps :

Key	i	Calculation	Index	Status
25	0	$(5 + 0^2)$ mod 10 = 5	5	Place 25
	1	$(5 + 1^2)$ mod 10 = 6	6	-
35	0	$(5 + 0^2)$ mod 10 = 5	5	Occupied
	1	$(5 + 1^2)$ mod 10 = 6	6	Occupied
	2	$(5 + 2^2)$ mod 10 = 9	9	Place 35
45	0	$(5 + 0^2)$ mod 10 = 5	5	Occ.
	1	$(5 + 1^2)$ mod 10 = 6	6	Occ.
	2	$(5 + 2^2)$ mod 10 = 9	9	Occ.
	3	$(5 + 3^2)$ mod 10 = 4	4	Place 45
55	0	$(5 + 0^2)$ mod 10 = 5	5	Occ.
	1	$(5 + 1^2)$ mod 10 = 6	6	Occ.
	2	$(5 + 2^2)$ mod 10 = 9	9	Occ.
	3	$(5 + 3^2)$ mod 10 = 4	4	Occ.
	4	$(5 + 4^2)$ mod 10 = 1	1	Place 55

Final Table :

Index	0	1	2	3	4	5	6	7	8	9
Data	-	55	-	-	45	25	35	-	-	35

9. Double Hashing (Example in Detail) :

Formula : New Index = (Hash1 + i x Hash2) mod Table Size

where Hash1 = Key mod Table Size

Hash2 = R - (Key mod R) (R is a prime less than Table Size)

i = 0, 1, 2, 3, ...

Example : Table Size = 10, R = 7

Insert : 25, 35, 45, 55

Key	Hash1 (Key mod 10)	Hash2 (7 - key mod 7)	Probing (i)	New Index	Status
25	5	7 - (25 mod 7) = 7 - 4 = 3	i = 0	(5 + 0x3) mod 10 = 5	Place 25
35	5	7 - (35 mod 7) = 7 - 0 = 7	i = 0	5	Occ.
			i = 1	(5 + 1x7) mod 10 = 2	Place 35
45	5	7 - (45 mod 7) = 7 - 3 = 4	i = 0	5	Occ.
			i = 1	(5 + 1x4) mod 10 = 9	Place 45
55	5	7 - (55 mod 7) = 7 - 6 = 1	i = 0	5	Occ.
			i = 1	(5 + 1x1) mod 10 = 6	Place 55

Final Table :

Index	0	1	2	4	5	6	7	8	9
Data	-	-	35	-	25	55	-	-	45

Important Note :

- ★ Double hashing reduces primary and secondary clustering.
- ★ Choose Table Size as a prime number for better distribution.
- ★ R must be a prime number less than Table Size.

10. Load Factor :

Definition :

Load Factor (α) is the ratio of number of stored elements to the size of hash table.

Formula : $\alpha = \frac{\text{Number of Elements}}{\text{Table Size}}$

Example :

Number of Elements = 8

Table Size = 10

α = 8 / 10 = 0.8

High load factor increases collisions and decreases performance.

11. Advantages :

- ★ Very fast searching.
- ★ Fast insertion and deletion.
- ★ Average time complexity is O(1).
- ★ Efficient for large datasets.
- ★ Direct access to data.
- ★ Used in many real life applications.

12. Disadvantages :

- ★ Collisions may occur.
- ★ Requires a good hash function.
- ★ Performance decreases if load factor is high.
- ★ Extra memory required for collision handling.

13. Time Complexity :

Operation	Average Case	Worst Case
Search	O(1)	O(n)
Insert	O(1)	O(n)
Delete	O(1)	O(n)

Key Points :

- ✓ Hash table provides constant time operations.
- ✓ Worst case occurs when all keys collide.
- ✓ Good hash function keeps load factor low.

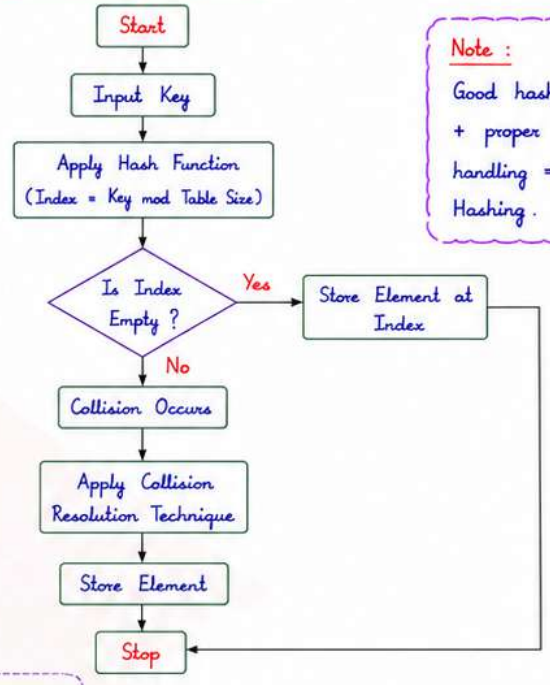
Ideal Load Factor (α) should be 0.5 to 0.7 for best performance.

Apna College Bihar

14. Applications of Hashing :

- ★ Database indexing and searching.
- ★ Password storage in systems.
- ★ Compiler design (symbol tables).
- ★ Dictionary and word look-up.
- ★ Cache memory management.
- ★ Search engines and web indexing.
- ★ Routing tables in networks.
- ★ File systems and disk management.
- ★ Blockchain technology.
- ★ Artificial Intelligence and Machine Learning.

15. Hashing Flowchart :



Note :
Good hash function + proper collision handling = Efficient Hashing.

16. Howh Table Works ?

1. Insert 2. Search 3. Delete

1. Insert

- Key is hashed using hash function.
- If index is empty, insert element.
- If not, apply collision resolution.

Example : Insert 25

25 mod 10 = 5

Index	0	1	2	3	4	5	6	7	8	9
Data	-	-	-	-	-	25	-	-	-	-

2. Search

- Key is hashed.
- Index is checked.
- If key found → Success
- Else → apply collision handling and search.

Example : Search 25

25 mod 10 = 5

Index	0	1	2	3	4	5	6	7	8	9
Data	-	-	-	-	-	25	-	-	-	-

Found at Index 5 ✓

3. Delete

- Key is hashed.
- Element is located.
- Element is removed or marked as deleted.

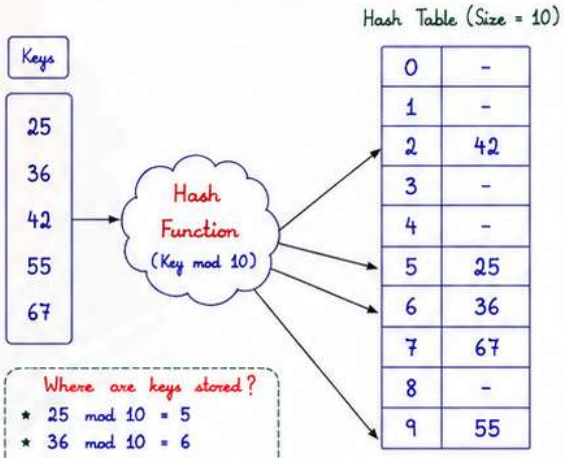
Example : Delete 25

25 mod 10 = 5

Index	0	1	2	3	4	5	6	7	8	9
Data	-	-	-	-	-	-	-	-	-	-

25 deleted from Index 5 ✓

17. Hash Table Working Diagram :



Where are keys stored ?

- ★ 25 mod 10 = 5
- ★ 36 mod 10 = 6
- ★ 42 mod 10 = 2
- ★ 55 mod 10 = 5 (Collision)
- ★ 67 mod 10 = 7

18. Summary Box :

- ✓ Hashing stores data using Hash Function.
- ✓ Hash Table stores data at calculated index.
- ✓ Collision occurs when keys map to same index.
- ✓ Collision can be resolved using Chaining / Probing techniques.
- ✓ Load Factor (α) = $\frac{\text{Number of Elements}}{\text{Table Size}}$
- ✓ Average time complexity of Search, Insert, Delete is O(1).
- ✓ Hashing is widely used in databases, compilers, dictionaries, caching, password storage and many more applications.

Ideal Load Factor (α)
0.5 to 0.7
(Best Performance)

19. Important Note :

- ★ Table Size should be prime number for better distribution.
- ★ Good hash function reduces collisions.
- ★ High load factor increases collisions.
- ★ Double hashing gives better distribution.
- ★ Hashing provides fastest average time operations.
- ★ Worst case occurs when all keys collide.
- ★ Use proper collision handling technique.

Apna College Bihar

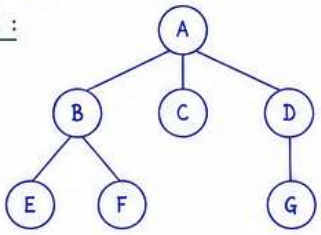
Basic Tree Terminologies

Introduction :
A Tree is a non-linear hierarchical data structure consisting of nodes connected by edges. It has no cycles and there is exactly one path between any two nodes.

1. Tree :

Definition : A Tree is a collection of nodes connected by edges in a hierarchical structure.

Example :



Root = A

Nodes = A, B, C, D, E, F, G

Edges = 6

2. Node :

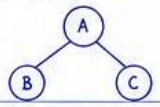
Definition : A Node is the basic element of a tree that stores data.

Example : (A) Here, 'A' is a node.

3. Root Node :

Definition : The topmost node of the tree is called Root Node. Every tree has only one root.

Example :

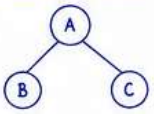


Root = A

4. Parent Node :

Definition : A node having one or more child nodes is called Parent Node.

Example :

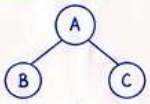


Parent of B and C = A

5. Child Node :

Definition : A node directly connected below a parent node is called Child Node.

Example :

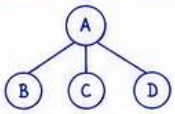


Children of A = B, C

6. Sibling Nodes :

Definition : Nodes having the same parent are called Sibling Nodes.

Example :

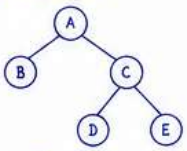


Sibling Nodes = B, C, D

7. Leaf Node :

Definition : A Leaf Node is a node that has no children.

Example :

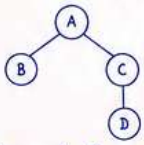


Leaf Nodes = B, D, E

8. Internal Node :

Definition : A node having at least one child is called Internal Node.

Example :

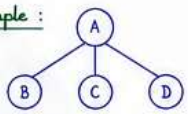


Internal Nodes = A, C

9. Degree of a Node :

Definition : The degree of a node is the number of children it has.

Example :

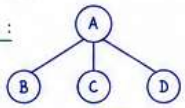


Degree(A) = 3
Degree(B) = 0
Degree(C) = 0
Degree(D) = 0

10. Degree of a Tree :

Definition : The degree of a tree is the maximum degree of any node in the tree.

Example :

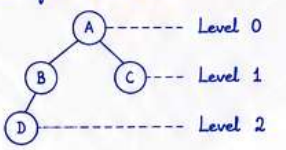


Degree of Tree = 3

11. Level of a Node :

Definition : The level of a node is its distance from the root.

Example :

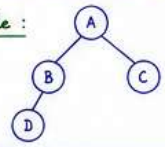


Level(A) = 0, Level(B) = 1, Level(D) = 2

12. Height of a Tree :

Definition : The height of a tree is the number of edges in the longest path from the root to a leaf.

Example :

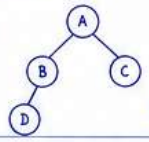


Height = 2

13. Depth of a Node :

Definition : The depth of a node is the maximum of edges from the root to that node.

Example :

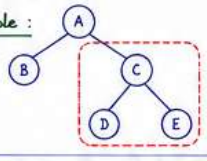


Depth(D) = 2

14. Subtree :

Definition : A Subtree is a tree formed by a node and all of its descendants.

Example :

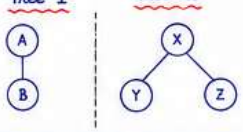


Subtree rooted at C contains nodes C, D, E.

15. Forest :

Definition : A forest is a collection of one or more disjoint trees.

Example :



Applications of Trees :

- File systems
- Database indexing
- Expression trees
- Decision trees
- XML / HTML documents
- Compiler design
- Artificial Intelligence
- Network routing

Important Points (Must Remember) :

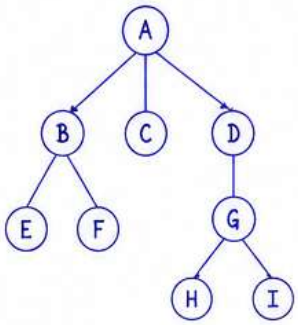
- Tree is a hierarchical non-linear data structure.
- Tree has one root and no cycles.
- Leaf Nodes have no children.
- Internal Nodes have one or more children.
- Degree of a node = Number of children.
- Degree of a tree = Maximum degree of any node.
- Height = Longest path from root to a leaf.
- Depth = Distance from root to a node.
- Forest = Collection of disjoint trees.

Apna College Bihar

16. Types of Nodes in a Tree :

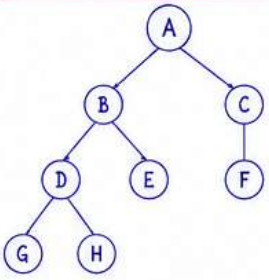
- ★ Root Node : Topmost node.
- ★ Internal Node : Node having at least one child.
- ★ Leaf Node : Node having no children.
- ★ Sibling Node : Nodes having same parent.
- ★ Ancestor : All nodes on the path from root to a given node.
- ★ Descendant : All nodes in the subtree of a given node.

17. Example of Various Terminologies :



- ★ Root : A
- ★ Parent of E,F : B
- ★ Parent of G : D
- ★ Children of A : B, C, D
- ★ Children of D : G
- ★ Sibling of E : F
- ★ Sibling of G : — (No sibling)
- ★ Leaf Nodes : E, F, C, H, I
- ★ Internal Nodes : A, B, D, G
- ★ Degree(A) = 3
- ★ Degree(B) = 2, Degree(D) = 1
- ★ Degree(G) = 2
- ★ Degree of Tree = 3
- ★ Level(H) = 2
- ★ Depth(I) = 3
- ★ Height of Tree = 3

18. Level and Depth (Example) :

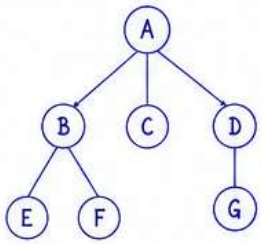


Node	Level	Depth
A	0	0
B	1	1
C	1	1
D	2	2
E	2	2
F	2	2
G	3	3
H	3	3

Note : Level of Root = 0
Depth of Root = 0

19. Degree of Nodes and Degree of Tree (Example) :

Example :

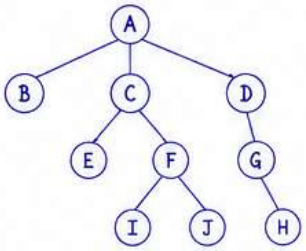


Degree of each node :

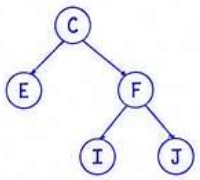
- Degree(A) = 3
- Degree(B) = 2
- Degree(C) = 0
- Degree(D) = 1
- Degree(E) = 0
- Degree(F) = 0
- Degree(G) = 0

Degree of Tree = 3
(Maximum degree of any node)

20. Subtree (Example) :



Subtree rooted at C contains nodes C, E, F, I, J.

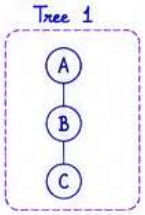


21. Forest (Example) :

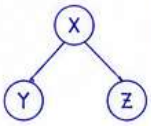
A Forest is a collection of one or more disjoint trees.

Example :

Tree 1



Tree 2



Here,
Tree 1 and Tree 2 are disjoint.
Forest = { Tree1, Tree2 }

22. Summary Table of Tree Terminologies :

Terminology	Meaning
Node	Basic unit which stores data.
Root Node	Topmost node of the tree.
Parent Node	Node having one or more children.
Child Node	Node directly connected to a parent node.
Sibling Nodes	Nodes having the same parent.
Leaf Node	Node having no children.
Internal Node	Node having at least one child.
Degree of Node	Number of children of that node.
Degree of Tree	Maximum degree of any node in the tree.
Level	Distance of node from root (Level of root = 0).
Depth	Number of edges from root to that node.
Height of Tree	Number of edges in the longest path from root to any leaf.
Subtree	Tree formed by a node and all its descendants.
Forest	Collection of one or more disjoint trees.

23. Important Points (Must Remember) :

- ★ Tree is a hierarchical non-linear data structure.
- ★ Tree has one root node and no cycles.
- ★ There is exactly one path between any two nodes.
- ★ Leaf nodes have no children.
- ★ Internal nodes have one or more children.
- ★ Degree of a node = Number of children.
- ★ Degree of a tree = Maximum degree of any node.
- ★ Level of root = 0.
- ★ Height of tree = Longest path from root to leaf.
- ★ Depth of a node = Distance from root to that node.
- ★ Subtree is a tree formed by a node and its descendants.
- ★ Forest is a collection of disjoint trees.

Applications of Trees :

- ★ File systems
- ★ Database indexing
- ★ Expression trees
- ★ Decision trees
- ★ XML / HTML documents
- ★ Compiler design
- ★ Artificial Intelligence
- ★ Network routing
- ★ Organization hierarchy

Apna College Bihar

Unit 5.0: Trees

Topic 2: Different Types of Trees - Binary Tree, Threaded Binary Tree, Binary Search Tree (BST), AVL Tree

Introduction :

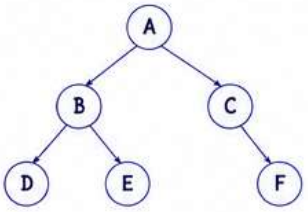
Trees are classified into different types based on their structure and properties. Different trees are used to solve different problems efficiently in computer science.

1. Binary Tree :

Definition :

A Binary Tree is a tree in which each node can have at most two children, called Left Child and Right Child.

Example :



Properties :

- ★ Maximum two children per node.
- ★ Left and right child are distinct.
- ★ Simple hierarchical structure.
- ★ Basis for many advanced tree structures.

Applications :

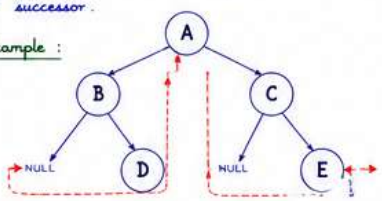
- ★ Expression Trees
- ★ Binary Heaps
- ★ Decision Trees
- ★ File Systems

2. Threaded Binary Tree :

Definition :

A Threaded Binary Tree is a binary tree in which NULL pointers are replaced with special links called Threads to point to the inorder predecessor or successor.

Example :



(Inorder sequence : B → D → A → C → E)

Advantages :

- ★ Faster inorder traversal.
- ★ No recursion required.
- ★ No stack needed for traversal.
- ★ Better utilization of NULL pointers.

Disadvantages :

- ★ More complex implementation.
- ★ Extra maintenance during insertion and deletion.

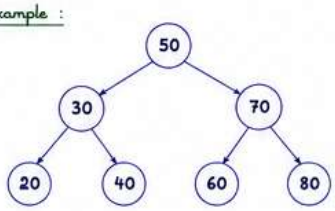
3. Binary Search Tree (BST) :

Definition :

A BST is a binary tree in which :

- All nodes in the left subtree are smaller than the root.
- All nodes in the right subtree are greater than the root.
- Both left and right subtrees are also BSTs.

Example :



Properties :

- ★ Left < Root < Right
- ★ Inorder traversal gives sorted order.
- ★ Efficient searching, insertion and deletion.

Time Complexity :

Operation	Average Case	Worst Case
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)

Applications :

- ★ Database indexing
- ★ Dictionary implementation
- ★ Symbol tables
- ★ Searching applications

4. AVL Tree :

Definition :

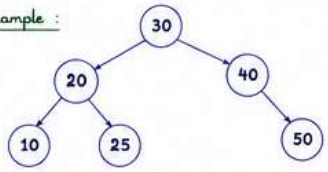
An AVL Tree is a self-balancing Binary Search Tree in which the height difference (Balance Factor) between the left and right subtrees of every node is at most 1.

Balance Factor :

Balance Factor = Height (Left Subtree) - Height (Right Subtree)

Allowed values : -1, 0, +1

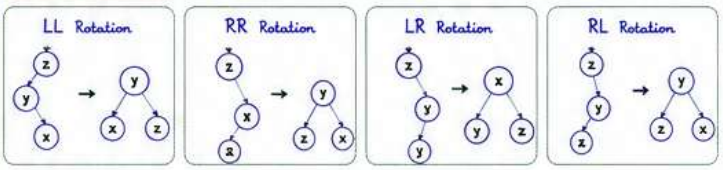
Example :



Properties :

- ★ Self-balancing.
- ★ Height remains approximately O(log n).
- ★ Faster searching than an unbalanced BST.
- ★ Rotations are used to maintain balance.

Types of Rotations :



Time Complexity :

Operation	Complexity
Search	O(log n)
Insert	O(log n)
Delete	O(log n)

Applications :

- ★ Database systems
- ★ Memory management
- ★ Operating systems
- ★ Search engines

5. Comparison of Different Trees :

Feature	Binary Tree	Threaded Binary Tree	BST	AVL Tree
Maximum Children	2	2	2	2
Ordered Data	No	No	Yes	Yes
Self-Balancing	No	No	No	Yes
Search Efficiency	Moderate	Moderate	Good	Excellent
Inorder Traversal	Normal	Faster	Sorted Output	Sorted Output
Insertion	Easy	Complex	Moderate	Moderate
Deletion	Easy	Complex	Moderate	Moderate
Space Utilization	Normal	Better	Normal	Normal

6. Advantages (Summary) :

Binary Tree :

- ★ Simple structure.
- ★ Easy implementation.

Threaded Binary Tree :

- ★ Faster traversal.
- ★ Better use of memory.

Binary Search Tree :

- ★ Efficient searching.
- ★ Sorted data storage.

AVL Tree :

- ★ Guaranteed O(log n) operations.
- ★ Balanced height.
- ★ High search efficiency.

7. Applications (General) :

- ★ Database indexing
- ★ Compiler design
- ★ Artificial Intelligence
- ★ Expression evaluation
- ★ Decision-making systems
- ★ File systems
- ★ Network routing
- ★ Memory management

Important Points (Very Important) :

- ★ A Binary Tree has at most two children per node.
- ★ A Threaded Binary Tree replaces NULL pointers with threads.
- ★ A BST follows the rule Left < Root < Right.
- ★ Inorder traversal of a BST gives elements in sorted order.
- ★ An AVL Tree is a self-balancing BST.
- ★ The Balance Factor of an AVL Tree must be -1, 0, or +1.
- ★ AVL Trees use LL, RR, LR, and RL rotations to maintain balance.
- ★ AVL Trees provide O(log n) time complexity for search, insertion, and deletion.

Apna College Bihar

Unit 5.0 : Trees

Topic 3 : Traversal of Trees - Inorder, Preorder, Postorder

Level Order Traversal

Introduction :

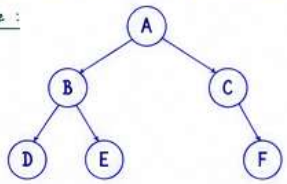
Traversal means visiting all the nodes of a tree in a systematic order. Different traversal techniques are used for different applications.

1. Inorder Traversal (Left - Root - Right)

Definition :

First visit the left subtree, then the root node and then the right subtree.

Example :



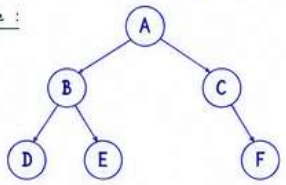
Inorder Sequence : D, B, E, A, C, F

2. Preorder Traversal (Root - Left - Right)

Definition :

First visit the root node, then the left subtree and then the right subtree.

Example :



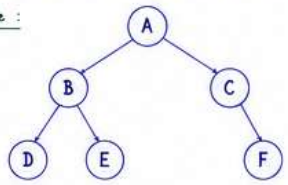
Preorder Sequence : A, B, D, E, C, F

3. Postorder Traversal (Left - Right - Root)

Definition :

First visit the left subtree, then the right subtree and then the root node.

Example :



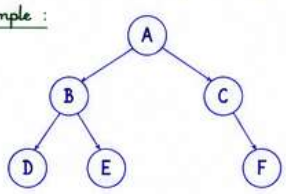
Postorder Sequence : D, E, B, F, C, A

4. Level Order Traversal (Breadth First Traversal)

Definition :

Visit the nodes level by level from left to right.

Example :



Step by Step Process :

Level	Nodes at that Level	Traversal Order
0	A	A
1	B , C	A, B, C
2	D , E , F	A, B, C, D, E, F

Level Order Sequence : A, B, C, D, E, F

Queue Based Approach :

1. Create an empty queue.
2. Enqueue the root node.
3. Dequeue a node, visit it.
4. Enqueue its left child (if exists).
5. Enqueue its right child (if exists).
6. Repeat steps 3 to 5 until queue becomes empty.

5. Comparison of Different Traversals

Traversal Type	Order	Process
Inorder	Left - Root - Right	Left → Root → Right
Preorder	Root - Left - Right	Root → Left → Right
Postorder	Left - Right - Root	Left → Right → Root
Level Order	Level by Level	Left to Right at each level

6. Applications of Traversals

★ **Inorder Traversal :**
Gives sorted order in BST.
Used in expression evaluation.

★ **Preorder Traversal :**
Used for copying a tree.
Used in prefix expression.

★ **Postorder Traversal :**
Used for deleting a tree.
Used in postfix expression.

★ **Level Order Traversal :**
Used in printing nodes level by level.
Used in breadth first search.

7. Recursive Algorithms

(a) Inorder Traversal

```
Inorder(node)
If node == NULL
    return
Inorder(node -> left)
Visit(node)
Inorder(node -> right)
```

(b) Preorder Traversal

```
Preorder(node)
If node == NULL
    return
Visit(node)
Preorder(node -> left)
Preorder(node -> right)
```

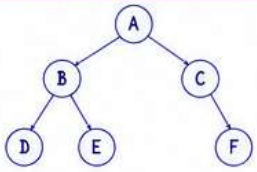
(c) Postorder Traversal

```
Postorder(node)
If node == NULL
    return
Postorder(node -> left)
Postorder(node -> right)
Visit(node)
```

(d) Level Order Traversal

```
LevelOrder(root)
Create empty queue Q
Enqueue(root)
While Q is not empty
    Dequeue(node)
    Visit(node)
    If node -> left  Enqueue(node -> left)
    If node -> right Enqueue(node -> right)
```

8. Example (All Traversals)



Traversal Sequences :

Inorder : D, B, E, A, C, F
Preorder : A, B, D, E, C, F
Postorder : D, E, B, F, C, A
Level Order : A, B, C, D, E, F

9. Important Points (Must Remember)

★ Inorder = Left, Root, Right
★ Preorder = Root, Left, Right
★ Postorder = Left, Right, Root
★ Level Order = Level by Level (Left to Right)
★ Inorder of BST gives sorted order.
★ Recursive solution is easy to implement.
★ Level order traversal uses Queue (BFS).

Quick Revision

✓ Inorder : Used in sorting, BST.
✓ Preorder : Used in copying, prefix.
✓ Postorder : Used in deleting, postfix.
✓ Level Order : Used in level wise printing, BFS.
✓ Use Recursion for Inorder, Preorder, Postorder.
✓ Use Queue for Level Order.
✓ Choose traversal based on the requirement.
✓ Understanding of traversal is essential for tree problems.

Traversal Order Summary

Traversal	Order	Abbreviation
Inorder	Left - Root - Right	L - R - R
Preorder	Root - Left - Right	R - L - R
Postorder	Left - Right - Root	L - R - R
Level Order	Level by Level	L → R (Each Level)

Apna College Bihar

Unit 5.0: Trees

Tree Operations and Algorithms with Complexity Analysis

Introduction :
In trees, various operations are performed to manage data efficiently. The main operations are Traversal, Search, Insertion, Deletion, Height Calculation, and other related algorithms.

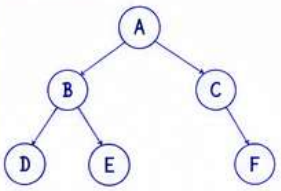
1. Tree Traversal

Traversal means visiting all the nodes of a tree exactly once in a systematic order.

Types of Traversal :

- ★ Depth First Traversal (DFT)
- ★ Breadth First Traversal (BFT)
- ★ Level Order Traversal

Example Tree :



Traversal Orders for the above Tree :

Traversal Type	Order
Inorder (L - R - R)	D, B, E, A, C, F
Preorder (R - L - R)	A, B, D, E, C, F
Postorder (L - R - R)	D, E, B, F, C, A
Level Order (BFS)	A, B, C, D, E, F

2. Depth First Traversal (DFT)

In DFT, we explore as far as possible along each branch before backtracking.

(a) Inorder Traversal (Left - Root - Right)

Algorithm (Recursive) :
1. If root is NULL, return.
2. Inorder (root → left)
3. Visit (root)
4. Inorder (root → right)

Time Complexity :
- ★ Best Case : O(n)
- ★ Average Case : O(n)
- ★ Worst Case : O(n)
- ★ Space (Recursion) : O(h)
  where h = Height of tree
  (h = log n for balanced tree, h = n for skewed tree)

(b) Preorder Traversal (Root - Left - Right)

Algorithm (Recursive) :
1. If root is NULL, return.
2. Visit (root)
3. Preorder (root → left)
4. Preorder (root → right)

Time Complexity :
- ★ Best Case : O(n)
- ★ Average Case : O(n)
- ★ Worst Case : O(n)
- ★ Space (Recursion) : O(h)

(c) Postorder Traversal (Left - Right - Root)

Algorithm (Recursive) :
1. If root is NULL, return.
2. Postorder (root → left)
3. Postorder (root → right)
4. Visit (root)

Time Complexity :
- ★ Best Case : O(n)
- ★ Average Case : O(n)
- ★ Worst Case : O(n)
- ★ Space (Recursion) : O(h)

3. Breadth First Traversal (BFT) / Level Order Traversal

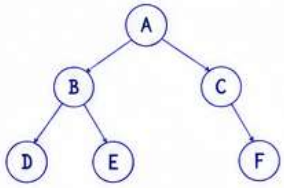
In BFT, we visit nodes level by level from left to right.

Algorithm (Using Queue) :
1. Create an empty queue Q.
2. Enqueue the root node.
3. While Q is not empty
   a) Dequeue a node, say temp.
   b) Visit (temp).
   c) Enqueue temp's left child (if exists).
   d) Enqueue temp's right child (if exists).

Time Complexity :
- ★ Best Case : O(n)
- ★ Average Case : O(n)
- ★ Worst Case : O(n)
- ★ Space (Queue) : O(w)

where w = Maximum width of the tree.

Example (Level Order Traversal) :



Step	Visited	Queue (Front → Rear)
1	A	B, C
2	B	C, D, E
3	C	D, E, F
4	D	E, F
5	E	F
6	F	Empty

Level Order Result : A, B, C, D, E, F

4. Searching in a Tree

Algorithm (DFS based Search) :
1. If root is NULL, return NOT FOUND.
2. If root → data == key, return FOUND.
3. Search key in left subtree.
4. If not found, search key in right subtree.

Time Complexity :
- ★ Best Case : O(1)
- ★ Average Case : O(n)
- ★ Worst Case : O(n)
- ★ Space : O(h)

5. Insertion in a Tree (Binary Tree)

Algorithm (Level Order Insertion) :
1. If tree is empty, create new node and make it root.
2. Else, create new node.
3. Traverse the tree in level order.
4. Insert the new node at the first position where a node has less than two children.

Time Complexity :
- ★ Best Case : O(1)
- ★ Average Case : O(n)
- ★ Worst Case : O(n)
- ★ Space : O(n)

6. Deletion in a Tree (Binary Tree)

Algorithm (Level Order Deletion) :
1. If tree is empty, return.
2. Traverse the tree and find the node to be deleted.
3. Traverse again to find the deepest rightmost node.
4. Replace the data of node to be deleted with deepest rightmost node's data.
5. Delete the deepest rightmost node.

Time Complexity :
- ★ Best Case : O(1)
- ★ Average Case : O(n)
- ★ Worst Case : O(n)
- ★ Space : O(h)

7. Height of a Tree

Algorithm (Recursive) :
1. If root is NULL, return -1.
2. lh = Height (root → left)
3. rh = Height (root → right)
4. Return (max(lh, rh) + 1)

Time Complexity :
- ★ Best Case : O(log n)
- ★ Average Case : O(n)
- ★ Worst Case : O(n)
- ★ Space : O(h)

8. Count of Nodes in a Tree

Algorithm (Recursive) :
1. If root is NULL, return 0.
2. leftCount = Count (root → left)
3. rightCount = Count (root → right)
4. Return (leftCount + rightCount + 1)

Time Complexity :
- ★ Best Case : O(1)
- ★ Average Case : O(n)
- ★ Worst Case : O(n)
- ★ Space : O(h)

Important Points (Must Remember)
- ★ Recursive solutions are simple and widely used.
- ★ Level Order Traversal uses Queue (BFS).
- ★ Height (h) = log n for balanced tree, h = n for skewed tree.
- ★ Insertion and Deletion in Binary Tree are done in Level Order.
- ★ Time complexity of most tree operations is O(n).
- ★ Space complexity depends on Height (h) or Width (w).

9. Summary Table of Operations

Operation	Technique	Time Complexity			Space
		Best	Average	Worst	
Inorder Traversal	Recursive	O(n)	O(n)	O(n)	O(h)
Preorder Traversal	Recursive	O(n)	O(n)	O(n)	O(h)
Postorder Traversal	Recursive	O(n)	O(n)	O(n)	O(h)
Level Order Traversal	Queue (BFT)	O(n)	O(n)	O(n)	O(w)
Search	DFS	O(1)	O(n)	O(n)	O(h)
Insertion (Binary Tree)	Level Order	O(1)	O(n)	O(n)	O(n)
Deletion (Binary Tree)	Level Order	O(1)	O(n)	O(n)	O(n)
Height of Tree	Recursive	O(log n)	O(n)	O(n)	O(h)
Count of Nodes	Recursive	O(1)	O(n)	O(n)	O(h)

10. Calculation of Number of Nodes in a Tree

Algorithm (Recursive) :

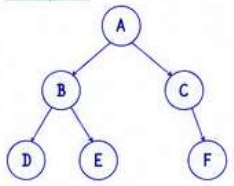
1. If root is NULL, return 0.
2. leftCount = Count(root → left)
3. rightCount = Count(root → right)
4. Return (leftCount + rightCount + 1)

Time Complexity :

- ★ Best Case : O(1)
- ★ Average Case : O(n)
- ★ Worst Case : O(n)
- ★ Space (Recursion) : O(h)

where h = Height of tree

Example :



Count(D) = 1
Count(E) = 1
Count(B) = 1 + 1 + 1 = 3
Count(F) = 1
Count(C) = 0 + 1 + 1 = 2
Count(A) = 3 + 2 + 1 = 6

Total Nodes = 6

11. Calculation of Sum of All Nodes in a Tree

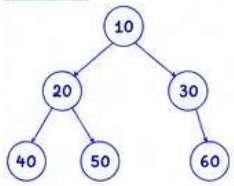
Algorithm (Recursive) :

1. If root is NULL, return 0.
2. leftSum = Sum(root → left)
3. rightSum = Sum(root → right)
4. Return (root → data + leftSum + rightSum)

Time Complexity :

- ★ Best Case : O(1)
- ★ Average Case : O(n)
- ★ Worst Case : O(n)
- ★ Space (Recursion) : O(h)

Example :



Sum(40) = 40
Sum(50) = 50
Sum(20) = 20 + 40 + 50 = 110
Sum(60) = 60
Sum(30) = 30 + 60 = 90
Sum(10) = 10 + 110 + 90 = 210

Total Sum = 210

12. Finding Height (or Depth) of a Tree

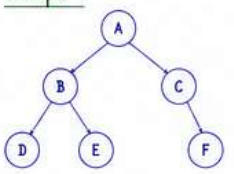
Algorithm (Recursive) :

1. If root is NULL, return -1.
2. leftHeight = Height(root → left)
3. rightHeight = Height(root → right)
4. Return (max(leftHeight, rightHeight) + 1)

Time Complexity :

- ★ Best Case : O(1)
- ★ Average Case : O(n)
- ★ Worst Case : O(n)
- ★ Space (Recursion) : O(h)

Example :



Height(D) = -1 + 1 = 0
Height(E) = -1 + 1 = 0
Height(B) = max(0, 0) + 1 = 1
Height(F) = -1 + 1 = 0
Height(C) = max(-1, 0) + 1 = 1
Height(A) = max(1, 1) + 1 = 2

Height of Tree = 2

Total Levels = Height + 1 = 3

13. Mirror of a Tree

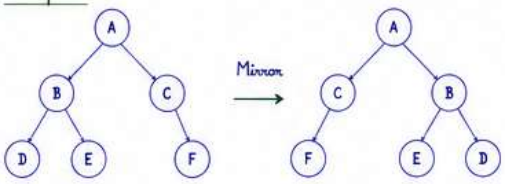
Algorithm (Recursive) :

1. If root is NULL, return.
2. Swap(root → left, root → right)
3. Mirror(root → left)
4. Mirror(root → right)

Time Complexity :

- ★ Best Case : O(1)
- ★ Average Case : O(n)
- ★ Worst Case : O(n)
- ★ Space (Recursion) : O(h)

Example :



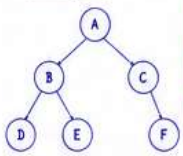
Original Tree — Mirror Tree

14. Count of Leaf Nodes

Algorithm (Recursive) :

1. If root is NULL, return 0.
2. If root → left is NULL and root → right is NULL, return 1.
3. leftCount = CountLeaves(root → left)
4. rightCount = CountLeaves(root → right)
5. Return (leftCount + rightCount)

Example :



Leaf Nodes = D, E, F

Total Leaf Nodes = 3

Time Complexity :

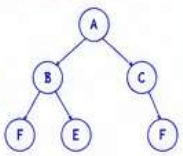
- ★ Best Case : O(1)
- ★ Average Case : O(n)
- ★ Worst Case : O(n)
- ★ Space (Recursion) : O(h)

15. Count of Non-Leaf (Internal) Nodes

Algorithm (Recursive) :

1. If root is NULL, return 0.
2. If root is leaf node, return 0.
3. leftCount = CountInternal(root → left)
4. rightCount = CountInternal(root → right)
5. Return (leftCount + rightCount + 1)

Example :



Internal Nodes = A, B, C

Total Internal Nodes = 3

Time Complexity :

- ★ Best Case : O(1)
- ★ Average Case : O(n)
- ★ Worst Case : O(n)
- ★ Space (Recursion) : O(h)

16. Summary Table of Operations with Complexity

Operation	Technique	Best Case	Average Case	Worst Case	Space Complexity (Recursion)	Space Complexity (Queue / Stack)
Tree Traversal (Inorder / Preorder / Postorder)	Recursive	O(n)	O(n)	O(n)	O(h)	O(1)
Level Order Traversal (BFS)	Queue	O(n)	O(n)	O(n)	O(1)	O(ω)
Searching in Tree	DFS / BFS	O(1)	O(n)	O(n)	O(h)	O(ω)
Insertion (Binary Tree)	Level Order	O(1)	O(n)	O(n)	O(1)	O(ω)
Deletion (Binary Tree)	Level Order	O(1)	O(n)	O(n)	O(1)	O(ω)
Height of Tree	Recursive	O(1)	O(n)	O(n)	O(h)	O(1)
Count of Nodes	Recursive	O(1)	O(n)	O(n)	O(h)	O(1)
Sum of Nodes	Recursive	O(1)	O(n)	O(n)	O(h)	O(1)
Mirror of Tree	Recursive	O(1)	O(n)	O(n)	O(h)	O(1)

Where,

- ★ n = Number of nodes in the tree
- ★ h = Height of the tree
- ★ ω = Maximum width of the tree

Notes :

- ★ In balanced tree, $h \approx \log n$
- ★ In skewed tree, $h = n - 1$
- ★ BFS uses extra space O(ω) due to queue for storing level nodes.

Important Points (Must Remember)

- ★ Traversal in trees helps to access all nodes.
- ★ DFS uses Recursion or Stack.
- ★ BFS uses Queue.
- ★ Inorder Traversal of BST gives sorted order.
- ★ Height of empty tree = -1.
- ★ Height of single node tree = 0.
- ★ Number of Nodes = $2^{h+1} - 1$ (Full Binary Tree)
- ★ Level of root is 0.
- ★ Leaf Node has no children.
- ★ Internal Node has at least one child.
- ★ Recursive algorithms are simple and elegant.
- ★ Complexity depends on Height (h) or Width (ω).

Apna College Bihar

17. Diameter of a Tree

Definition :

Diameter of a tree is the number of edges on the longest path between any two nodes.

Algorithm (Recursive) :

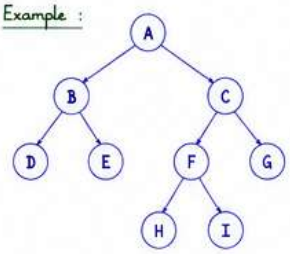
1. Find height of left and right subtrees.
2. Diameter passing through root = leftHeight + rightHeight + 2
3. Diameter = max (leftDiameter , rightDiameter , leftHeight + rightHeight + 2)
4. Return Diameter.

Time Complexity :

- ★ Best Case : O(n)
- ★ Average Case : O(n)
- ★ Worst Case : O(n)
- ★ Space (Recursion) : O(h)

where h = Height of tree

Example :



Longest path = D - B - A - C - F - H

Number of edges = 5

Diameter = 5

18. Lowest Common Ancestor (LCA)

Definition :

LCA of two nodes n1 and n2 is the lowest node in the tree that has both n1 and n2 as descendants (a node can be a descendant of itself).

Algorithm (Recursive) :

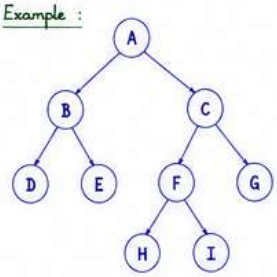
1. If root is NULL, return NULL.
2. If root == n1 or root == n2, return root.
3. Search in left subtree and right subtree.
4. If both sides return non-NULL, root is LCA.
5. Else return the non-NULL side.

Time Complexity :

- ★ Best Case : O(n)
- ★ Average Case : O(n)
- ★ Worst Case : O(n)
- ★ Space (Recursion) : O(h)

where h = Height of tree

Example :



LCA of D and E = B
LCA of D and I = A
LCA of H and I = F
LCA of E and G = A

19. Check if a Tree is Identical to Another Tree

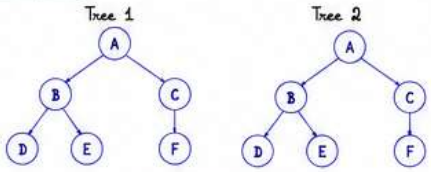
Algorithm (Recursive) :

1. If both trees are NULL, return true.
2. If one is NULL and other is not, return false.
3. If root1 → data != root2 → data, return false.
4. Check left subtrees are identical.
5. Check right subtrees are identical.
6. Return true if both left and right are true.

Time Complexity :

- ★ Best Case : O(n)
- ★ Average Case : O(n)
- ★ Worst Case : O(n)
- ★ Space (Recursion) : O(h)

Example :



Result : Identical

Note :

n = Number of nodes
h = Height of tree

20. Convert a Tree to its Mirror

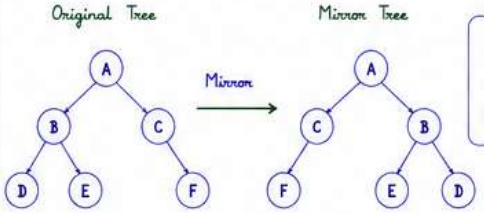
Algorithm (Recursive) :

1. If root is NULL, return.
2. Swap left and right child of root.
3. Recursively convert left subtree.
4. Recursively convert right subtree.

Time Complexity :

- ★ Best Case : O(n)
- ★ Average Case : O(n)
- ★ Worst Case : O(n)
- ★ Space (Recursion) : O(h)

Example :



Where :

n = Number of nodes
h = Height of tree

21. Summary Table of Tree Operations

Operation	Technique / Method	Time Complexity			Space Complexity	
		Best Case	Average Case	Worst Case	Recursion	Queue / Stack
Tree Traversal (DFS)	Recursive	O(n)	O(n)	O(n)	O(h)	O(1)
Level Order Traversal	Queue (BFS)	O(n)	O(n)	O(n)	O(1)	O(ω)
Searching in Tree	DFS / BFS	O(1)	O(n)	O(n)	O(h)	O(ω)
Insertion (Binary Tree)	Level Order	O(1)	O(n)	O(n)	O(1)	O(ω)
Deletion (Binary Tree)	Level Order	O(1)	O(n)	O(n)	O(1)	O(ω)
Height of Tree	Recursive	O(1)	O(n)	O(n)	O(h)	O(1)
Count of Nodes	Recursive	O(1)	O(n)	O(n)	O(h)	O(1)
Sum of Nodes	Recursive	O(1)	O(n)	O(n)	O(h)	O(1)
Mirror of Tree	Recursive	O(1)	O(n)	O(n)	O(h)	O(1)
Diameter of Tree	Recursive	O(n)	O(n)	O(n)	O(h)	O(1)
LCA of Two Nodes	Recursive	O(n)	O(n)	O(n)	O(h)	O(1)
Check Identical Trees	Recursive	O(n)	O(n)	O(n)	O(h)	O(1)

Where :

- ★ n = Number of nodes in the tree
- ★ h = Height of the tree
- ★ ω = Maximum width of the tree

Note :

- ★ In balanced tree, h ≈ log n
- ★ In skewed tree, h = n - 1
- ★ BFS uses extra space O(ω) due to queue for storing nodes of a level.

Important Points (Must Remember)

- ★ Traversal visits every node exactly once.
- ★ DFS uses Recursion or Stack.
- ★ BFS uses Queue.
- ★ Height of empty tree = -1.
- ★ Height of single node tree = 0.
- ★ Level of root = 0.
- ★ Number of nodes in full binary tree = $2^{h+1} - 1$.
- ★ Diameter is the longest path between any two nodes.

Applications of Tree Operations

- ★ File Systems
- ★ Database Indexing
- ★ Expression Evaluation
- ★ Compilers
- ★ Decision Trees
- ★ Artificial Intelligence
- ★ Network Routing
- ★ Search Engines
- ★ Syntax Trees
- ★ Game Tree Analysis

Applications of Binary Trees

Apna College Bihar

Introduction :

A Binary Tree is a non-linear data structure in which each node has at most two children, called Left child and Right child.

Binary trees are used in many real-life applications due to their hierarchical nature and efficiency.

Basic Properties of Binary Tree :

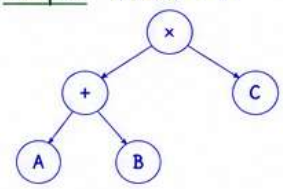
- ★ Each node has at most two children.
- ★ Left and right subtrees are disjoint.
- ★ The order of subtrees matters.
- ★ Recursive structure (subtrees are also trees).

1. Expression Trees

An expression tree represents an arithmetic or logical expression.

- ★ Leaf nodes → operands
- ★ Internal nodes → operators

Example : (A + B) × C



Traversals :

- ★ Inorder : A + B × C (Infix form)
- ★ Preorder : × + A B C (Prefix form)
- ★ Postorder : A B + C × (Postfix form)

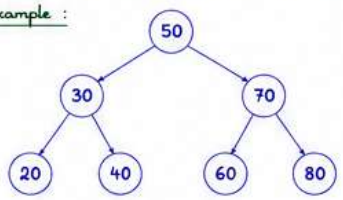
Applications :

Expression evaluation, Compilers, Conversions (infix ↔ prefix ↔ postfix), Calculator programs.

2. Binary Search Tree (BST)

In BST, for every node : Left Subtree < Root < Right Subtree

Example :



Applications :

- ★ Fast searching
- ★ Data storage
- ★ Dictionary implementation
- ★ Symbol tables
- ★ Database indexing

Average Time Complexity :

Operation	Average
Search	O(log n)
Insertion	O(log n)
Deletion	O(log n)

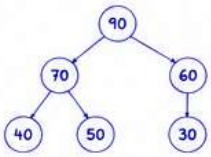
Note : In the worst case (skewed tree), time complexity may become O(n).

3. Binary Heap

A Binary Heap is a complete binary tree that satisfies the heap property.

Max Heap : Parent ≥ Children

Min Heap : Parent ≤ Children



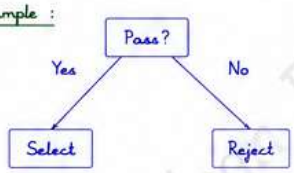
Applications :

- ★ Priority Queue
- ★ Heap Sort
- ★ Scheduling
- ★ Finding min/max
- ★ Graph algorithms

4. Decision Trees

A decision tree represents a sequence of decisions.

Example :



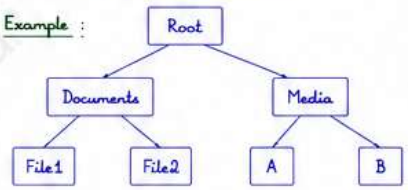
Applications :

- ★ Artificial Intelligence
- ★ Machine Learning
- ★ Classification
- ★ Decision making
- ★ Expert systems

5. File System Hierarchy

Binary tree is used to represent hierarchical structure of files and directories.

Example :



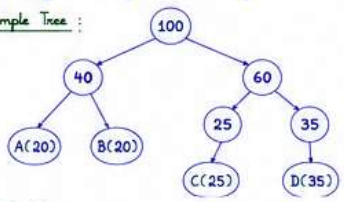
Applications :

- ★ Directory organization
- ★ File searching
- ★ Hierarchical data management

6. Huffman Coding

Huffman tree is used for lossless data compression. Frequently occurring characters get shorter codes.

Example Tree :



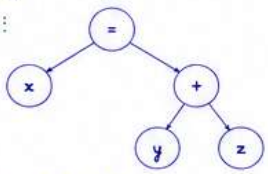
Applications :

- ★ Data compression
- ★ File compression
- ★ Text compression
- ★ Communication systems

7. Syntax Trees

Syntax tree represents the syntactic structure of a program or expression.

Example :



Represents : x = y + z

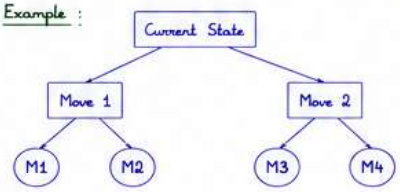
Applications :

- ★ Compiler design
- ★ Parsing
- ★ Syntax analysis
- ★ Code generation

8. Game Trees

Game tree represents possible moves in a game.

Example :



Applications :

- ★ Chess
- ★ Tic-Tac-Toe
- ★ Checkers
- ★ AI game playing

9. Advantages of Binary Trees

- ★ Represents hierarchical data efficiently.
- ★ Supports recursive processing.
- ★ Useful for searching and sorting.
- ★ Can represent expressions naturally.
- ★ Foundation for advanced structures like BSTs and Heaps.

10. Major Applications Summary

Application	Purpose
Expression Trees	Expression representation / evaluation
BST	Searching and ordered data
Binary Heap	Priority queues and Heap Sort
Decision Trees	Decision making and classification
Huffman Trees	Data compression
Syntax Trees	Compiler parsing and analysis
Game Trees	Game AI and possible moves
File Hierarchy	Organizing hierarchical data

11. Important Exam Points

- ★ Expression trees represent arithmetic / logical expressions.
- ★ BSTs are used for efficient searching and ordered data.
- ★ Binary heaps are used in priority queues and Heap Sort.
- ★ Decision trees are used for classification and decision-making.
- ★ Huffman trees are used for lossless data compression.
- ★ Syntax trees are important in compiler design.
- ★ Game trees are used in game-playing AI.
- ★ Binary trees are widely used wherever hierarchical or recursive data is needed.

Apna College Bihar

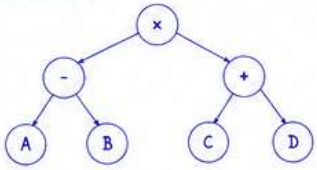
12. More Applications (In Detail)

(a) Expression Trees

- ★ Used to represent arithmetic / logical expressions.
- ★ Helps in evaluation and conversion.

Example : Infix → (A - B) × (C + D)

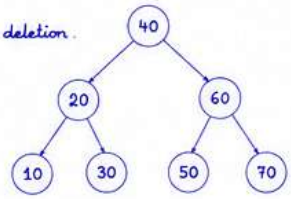
Expression Tree :



(b) Binary Search Trees (BST)

- ★ Maintains ordered data.
- ★ Efficient search, insertion and deletion.

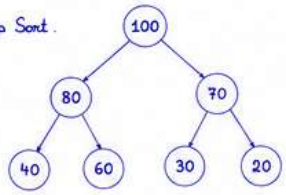
Property : Left < Root < Right



(c) Binary Heap

- ★ Complete binary tree.
- ★ Used in Priority Queue and Heap Sort.

Max Heap Example :



(d) Decision Trees

- ★ Represents decisions and their possible results.
- ★ Used in AI and machine learning.

Example :

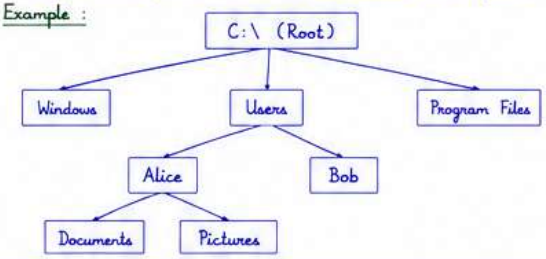


13. Real Life Examples

(a) File System Hierarchy

- ★ Files and directories are stored in hierarchical order.
- ★ Every directory can contain subdirectories and files.

Example :



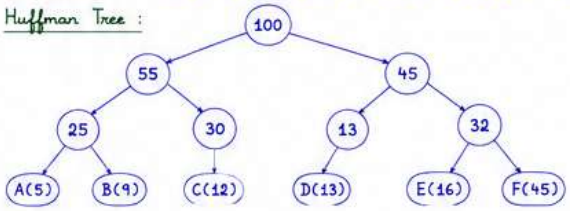
(b) Huffman Coding (Data Compression)

- ★ Characters with more frequency get shorter codes.
- ★ Binary tree is built using frequencies.

Example :

Frequencies → A:5, B:9, C:12, D:13, E:16, F:45

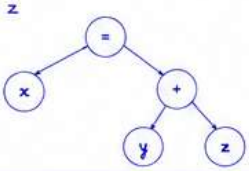
Huffman Tree :



(c) Syntax Trees

- ★ Used in compilers to represent structure of expressions.
- ★ Helps in parsing and syntax analysis.

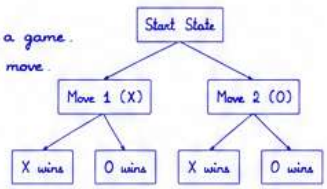
Example : Expression → x = y + z



(d) Game Trees

- ★ Represents all possible moves in a game.
- ★ Used in AI for finding the best move.

Example (Tic-Tac-Toe) :



14. Advantages of Using Binary Trees

- ★ Efficient representation of hierarchical data.
- ★ Supports recursive algorithms naturally.
- ★ Useful for searching, sorting and indexing.
- ★ Can represent complex expressions easily.
- ★ Foundation for advanced data structures like BST, Heap, AVL Tree.
- ★ Widely used in real-life applications.

15. Summary Table of Applications

Application	Binary Tree Used	Purpose	Example Use
Expression Trees	Expression Tree	Represent and evaluate expressions	Calculator, Compilers
Binary Search Trees	BST	Store ordered data and search efficiently	Dictionary, Database Indexing
Binary Heap	Heap (Max/Min)	Priority Queue and Heap Sort	Scheduling, Heapsort
Decision Trees	Decision Tree	Decision making and classification	AI, Machine Learning
File System Hierarchy	General Binary Tree	Represent directories and files	OS File System
Huffman Coding	Huffman Tree	Data compression (Lossless)	Zip, Text compression
Syntax Trees	Syntax Tree	Parsing and syntax analysis	Compiler Design
Game Trees	Game Tree	Explore possible moves	Chess, Tic-Tac-Toe

16. Important Points (Must Remember)

- ✓ Binary trees are used wherever hierarchical or recursive data is needed.
- ✓ Expression trees help in evaluation and conversion of expressions.
- ✓ BST provides fast search, insertion and deletion in ordered data.
- ✓ Binary heaps are complete trees used for priority queues.
- ✓ Decision trees are widely used in classification and decision making.
- ✓ File system hierarchy organizes files and directories.
- ✓ Huffman coding reduces storage by assigning variable length codes.
- ✓ Syntax trees are essential in compiler design for parsing.
- ✓ Game trees help AI to choose the best possible move.
- ✓ Binary trees form the base for many advanced data structures.

Apna College Bihar

B Tree and B+ Tree: Definitions, Algorithms and Analysis

Introduction :

B Tree and B+ Tree are balanced multi-way search trees. They are mainly used when a large amount of data must be stored and searched efficiently, especially in databases and file systems.

Unlike a binary search tree, a node in B Tree or B+ Tree can contain multiple keys and multiple children.

Key Points :

- ★ Multi-way search tree.
- ★ Self balancing.
- ★ All leaf nodes at the same level.
- ★ Used in databases, file systems, disk based indexing.
- ★ Efficient for large data.

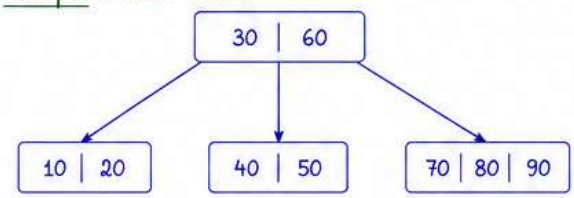
1. B Tree

Definition :

A B Tree is a self-balancing multi-way search tree in which each node can contain multiple keys and children.

All leaf nodes remain at the same level, so the tree remains balanced.

Example (Order = 4) :



2. Properties of B Tree

For a B Tree of order m :

- ★ A node can have at most m children.
- ★ A node can contain at most m - 1 keys.
- ★ Except the root, every internal node generally has at least $\lceil m/2 \rceil$ children.
- ★ Keys inside a node are stored in sorted order.
- ★ All leaf nodes are at the same level.
- ★ The tree remains balanced after insertion and deletion.

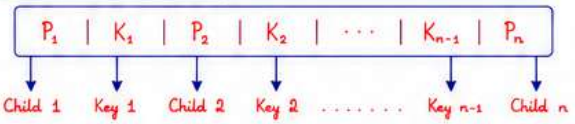
Note :

If a node has k keys, it has (k + 1) children.

Except root, every internal node has at least $\lceil m/2 \rceil$ children and at most m children.

3. Structure of B Tree (General View)

General Node Structure :



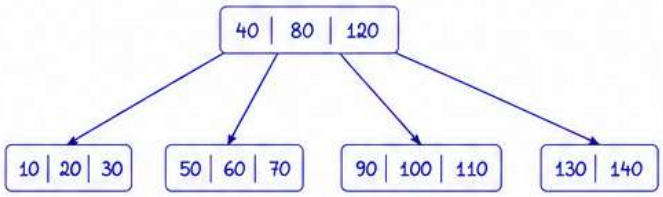
Where,

- ★ K_i are keys in sorted order ($K_1 < K_2 < \ldots < K_{n-1}$)
- ★ P_i are pointers to the child nodes.
- ★ A node with k keys has (k + 1) children.

Constraints (Order = m)

- ★ 1 ≤ number of keys ≤ m - 1
- ★ $\lceil m/2 \rceil$ ≤ number of children ≤ m (for non-root internal nodes)
- ★ Root has minimum 1 key and 2 children (if not a leaf).
- ★ All leaves are at the same level.
- ★ Keys in a node are sorted.

4. Another Example (Order = 5)



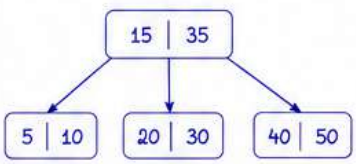
Advantages of B Tree :

- ★ Always balanced.
- ★ Reduces height of the tree (logarithmic height).
- ★ Efficient searching, insertion and deletion.
- ★ Suitable for large datasets.
- ★ Minimizes disk access.
- ★ Used in external memory storage.

Summary of B Tree (Quick View)

- ★ Balanced multi-way search tree.
- ★ Each node can have multiple keys and children.
- ★ Keys in a node are sorted.
- ★ All leaf nodes are at the same level.
- ★ Supports efficient search, insertion and deletion.

Example (Order = 3)



Important Uses :

- ★ Database systems
- ★ File systems
- ★ External memory storage
- ★ Disk based indexing

B Tree and B+ Tree: Definitions, Algorithms and Analysis

Apna College Bihar

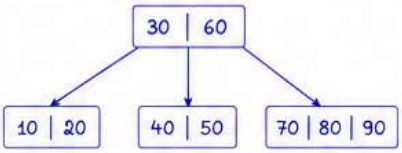
5. Searching in B Tree

Algorithm :

1. Start from the root.
2. Compare the search key with keys in the node.
3. If key is found, return FOUND.
4. Otherwise, choose the appropriate child.
5. Repeat until key is found or leaf is reached.
6. If key is not present, return NOT FOUND.

Complexity :
Search = $O(\log_m n) = O(h)$
where h = height of the tree

Example (Order = 4)
Search key : 50



Path followed : 30|60 → 40|50
Key 50 is found.

6. Insertion in B Tree

Basic Steps :

1. Search the correct leaf node.
2. Insert the new key in sorted order.
3. If node does not overflow, stop.
4. If node overflows, split the node.
5. Move the middle key up to parent.
6. Continue splitting upward if needed.
7. If root splits, create a new root.

Important Point :
Node splitting keeps the tree balanced.

7. Example of Insertion (Order = 4, Max 3 keys per node)

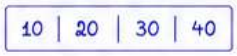
Insert 40

Step 1 : Before insertion



→

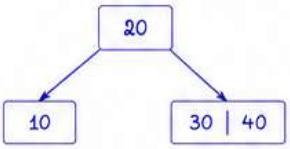
Step 2 : Insert 40



Overflow occurs

→

Step 3 : Split and promote



Note :
Middle key (20) is moved up to parent. Tree remains balanced.

8. Deletion in B Tree

Basic Steps :

1. Find the key.
2. If key is in leaf, remove it.
3. If key is in internal node, replace using predecessor or successor.
4. If a node has too few keys, perform Borrowing or Merging.
5. Continue fixing until all properties are satisfied.

9. Borrowing in B Tree

If a node has less keys, it may borrow a key from a sibling through parent.

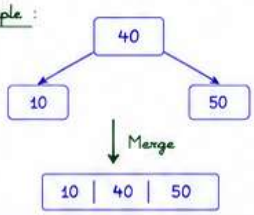
Example :



10. Merging in B Tree

If borrowing is not possible, two nodes are merged.

Example :



11. Complexity of B Tree

Operation	Complexity
Search	O(log n)
Insertion	O(log n)
Deletion	O(log n)
Space	O(n)

Note :
The base of logarithm depends on the order (m) of the tree.

12. B+ Tree

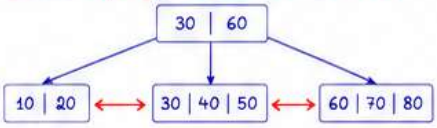
Definition :
A B+ Tree is a balanced multi-way search tree in which :

- Internal nodes contain keys used for navigation.
- Actual data/records are stored only in leaf nodes.
- Leaf nodes are linked using pointers.

13. Properties of B+ Tree

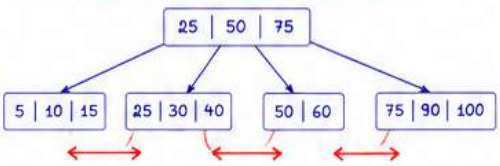
- All records are stored in leaf nodes.
- Internal nodes contain only search keys.
- All leaves are at the same level.
- Leaf nodes are linked.
- Supports efficient sequential traversal.
- Remains balanced after insertion and deletion.

14. Example of B+ Tree (Order = 4)



Key Point :
All data are in leaf nodes and leaves are linked. Hence, range search becomes very efficient.

15. Another Example (Order = 5)



16. B+ Tree Searching

Steps :

1. Start from root.
2. Compare key with internal node keys.
3. Follow appropriate child pointer.
4. Continue until leaf node is reached.
5. Search the key in leaf node.

Complexity :
Search = $O(\log n) = O(h)$

17. Insertion in B+ Tree

Steps :

1. Find the appropriate leaf.
2. Insert the key in sorted order.
3. If overflow occurs, split the leaf.
4. Copy the separator key into parent.
5. If parent overflows, split it.
6. Continue upward when necessary.

18. Deletion in B+ Tree

Steps :

1. Find the key in the leaf.
2. Delete the key.
3. If node still satisfies minimum occupancy, stop.
4. Otherwise borrow from a sibling.
5. If borrowing is impossible, merge nodes.
6. Update parent separator keys if required.

19. Why B+ Tree is Preferred in Databases

- Actual data are in leaf nodes.
- Leaves are linked.
- For range query (e.g., 100 to 500):
  - → Find 100.
  - → Move through linked leaves 

  - → Continue until 500.
  - → This makes range queries and sequential access very efficient.

Apna College Bihar

Apna College Bihar

Unit 6.0 : Graph

Basic Terminologies and Representations

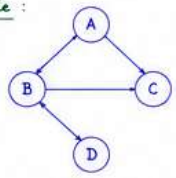
1. Introduction

Definition :
A Graph is a non-linear data structure consisting of a set of vertices (nodes) and edges that connect pairs of vertices.

It is represented as : G = (V, E)

V = Set of vertices
E = Set of edges

Example :



Vertices : A, B, C, D
Edges : (A,B), (A,C), (B,C), (B,D)

2. Vertex

A vertex is a fundamental element of a graph.
It is also called a Node.

Example :



Here, A is a vertex.

3. Edge

An edge is a connection between two vertices.

Example :



Edge is represented as : (A, B)

4. Adjacent Vertices

Two vertices are called Adjacent Vertices if an edge directly connects them.

Example :



A and B are adjacent.

5. Incident Edge

An edge is said to be Incident on a vertex if that vertex is one of the endpoints of the edge.

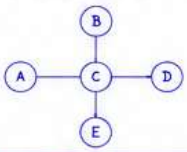
For 

The edge (A,B) is incident on both A and B.

6. Degree of a Vertex

The Degree of a vertex is the number of edges connected to it.

Example :



Degree (C) = 4

7. Directed Graph (Digraph)

A Directed Graph contains edges having a specific direction.

Example :



Edge is represented as : (A,B)
Direction is from A to B.

8. Undirected Graph

An Undirected Graph contains edges without direction.

Example :

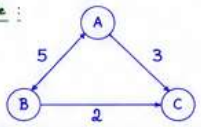


The connection can be travelled in both directions.

9. Weighted Graph

A Weighted Graph assigns a numerical value to each edge.

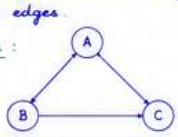
Example :



10. Unweighted Graph

An Unweighted Graph does not assign numerical weights to its edges.

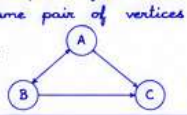
Example :



11. Simple Graph

A Simple Graph has :
- No self-loops
- No multiple edges between the same pair of vertices

Example :



12. Multigraph

A Multigraph can contain multiple edges between the same pair of vertices.

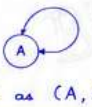
Example :



13. Self-Loop

A Self-Loop is an edge that starts and ends at the same vertex.

Example :



Represented as (A, A)

14. Path

A Path is a sequence of vertices in which consecutive vertices are connected by edges.

Example :

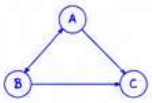


Path : A → B → C → D
Length of path = 3 edges

15. Cycle

A Cycle is a path that starts and ends at the same vertex.

Example :

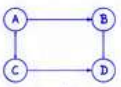


Cycle : A → B → C → A

16. Connected Graph

A graph is Connected if there is a path between every pair of vertices.

Example :



Every vertex can be reached from another vertex.

17. Disconnected Graph

A graph is Disconnected if at least one pair of vertices has no path between them.

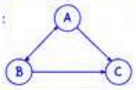
Example :

18. Complete Graph

A Complete Graph is a graph in which every vertex is directly connected to every other vertex.

For n vertices, number of edges in undirected complete graph = $\frac{n(n-1)}{2}$

Example (n = 3) :

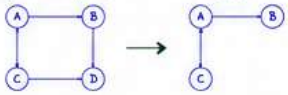


19. Subgraph

A Subgraph is a graph formed using some vertices and edges of another graph.

Example :

Original Graph G → Subgraph



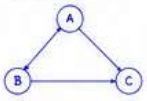
20. Graph Representation

Graphs are mainly represented using :
1. Adjacency Matrix
2. Adjacency List

21. Adjacency Matrix

Represents a graph using a two-dimensional matrix.

Example Graph :



Adjacency Matrix :

*	A	B	C
A	0	1	1
B	1	0	1
C	1	1	0

Space Complexity : $O(V^2)$

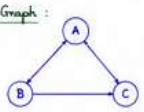
Advantages :
- Easy to implement.
- Edge existence check is fast.
- Suitable for dense graphs.

Disadvantages :
- Requires large memory for sparse graphs.
- Wastes space when there are few edges.

22. Adjacency List

Stores list of neighboring vertices for each vertex.

Example Graph :



Adjacency List Representation :

A → B → C
B → A → C
C → A → B

Space Complexity : $O(V + E)$

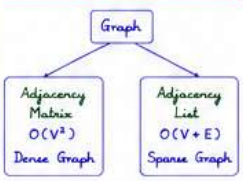
Advantages :
- Memory efficient for sparse graphs.
- Easy to traverse neighbors.
- Suitable for large sparse graphs.

Disadvantages :
- Checking a particular edge existence may be slower.

23. Adjacency Matrix vs Adjacency List

Feature	Adjacency Matrix	Adjacency List
Storage	$O(V^2)$	$O(V + E)$
Edge Checking	$O(1)$	$O(\text{degree})$
Best For	Dense Graph	Sparse Graph
Implementation	Simple	Relatively efficient
Neighbor Traversal	$O(V)$	$O(\text{degree})$

24. Graph Representation Summary



★ Important Exam Points

- ★ Graph consists of vertices and edges.
- ★ Graph is represented as G = (V, E).
- ★ Directed Graph has directional edges.
- ★ Undirected Graph has edges without direction. Weighted Graph has values on edges.
- ★ Unweighted Graph has no weights.
- ★ Path is a sequence of vertices.
- ★ Cycle starts and ends at same vertex.
- ★ Connected Graph has path between every pair of vertices.
- ★ Adjacency Matrix requires $O(V^2)$ space.
- ★ Adjacency List requires $O(V + E)$ space.
- ★ Adjacency Matrix is good for dense graphs.
- ★ Adjacency List is good for sparse graphs.

Graph Search and Traversal Algorithms

Apna College Bihar

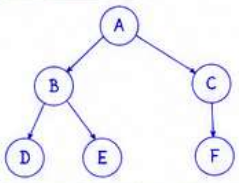
Introduction :

Graph Traversal is the process of visiting all or selected vertices of a graph systematically.

The two fundamental graph traversal algorithms are :

- ★ Breadth First Search (BFS)
- ★ Depth First Search (DFS)

Example Graph :



Vertices : A, B, C, D, E, F

Edges : (A,B), (A,C), (B,D), (B,E), (C,F)

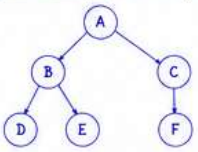
1. Breadth First Search (BFS)

Definition :

BFS is a graph traversal algorithm that visits vertices level by level.

Data Structure Used : Queue (FIFO)

BFS Example (Starting from A) :



BFS Traversal Order :

A → B → C → D → E → F

BFS Algorithm :

1. Create an empty Queue.
2. Mark start vertex as visited.
3. Insert start vertex into Queue.
4. While Queue is not empty :
   a. Remove a vertex from Queue.
   b. Visit the vertex.
   c. For every adjacent vertex :
      If it is not visited :
      - Mark it visited.
      - Insert it into Queue.
5. Stop.

2. BFS Using Queue

Queue Operations :

Step 1 : Insert A

Front → [A] ← Rear

Step 2 : Remove A
Add B, C

Front → [B | C] ← Rear

Step 3 : Remove B
Add D, E

Front → [C | D | E] ← Rear

Step 4 : Remove C
Add F

Front → [D | E | F] ← Rear

Step 5 : Remove D

Front → [E | F] ← Rear

Step 6 : Remove E

Front → [F] ← Rear

Step 7 : Remove F

Front → [ ] ← Rear (Empty)

Thus BFS = A → B → C → D → E → F

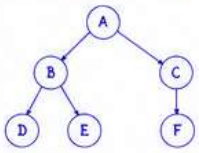
3. Depth First Search (DFS)

Definition :

DFS explores a graph by going as deep as possible along one path before backtracking.

Data Structure Used : Stack / Recursion

DFS Example (Starting from A) :



One possible DFS Traversal Order :

A → B → D → E → C → F

DFS Algorithm (Recursive) :

1. Mark start vertex as visited.
2. Visit start vertex.
3. For every adjacent vertex :
   If it is not visited :
   Apply DFS to that vertex.
4. Stop.

4. DFS Using Stack

Stack Operations :

Step 1 : Push A

Top → [A]

Step 2 : Pop A
Push B, C

Top → [C | B]

Step 3 : Pop B
Push D, E

Top → [C | D]

Step 4 : Pop D (Visit D)

Top → [C]

Step 5 : Pop E (Visit E)

Top → [C]

Step 6 : Pop C
Push F

Top → [F]

Step 7 : Pop F (Visit F)

Top → (Empty)

Thus DFS = A → B → D → E → C → F

5. BFS vs DFS

Feature	BFS	DFS
Full Form	Breadth First Search	Depth First Search
Approach	Level by level	Depth first
Data Structure	Queue	Stack / Recursion
Shortest path in unweighted graph	Yes	Not guaranteed
Memory Usage	Can be high	Usually lower in wide graphs
Best For	Level-based search, shortest path	Path exploration, backtracking

6. Time and Space Complexity

Time Complexity (Adjacency List) :

BFS = O(V + E)
DFS = O(V + E)

Space Complexity :

BFS = O(V)
DFS = O(V)

Where,

- V = Number of vertices
- E = Number of edges

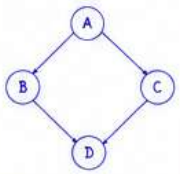
7. Applications of BFS

- ★ Finding shortest paths in unweighted graphs.
- ★ Level-order exploration.
- ★ Finding connected components.
- ★ Network broadcasting.
- ★ Social network analysis.
- ★ Web crawling.
- ★ Finding vertices at a given distance.

8. Applications of DFS

- ★ Detecting cycles.
- ★ Finding connected components.
- ★ Topological sorting.
- ★ Path finding.
- ★ Maze solving.
- ★ Backtracking.
- ★ Detecting bridges and articulation points.

9. BFS Shortest Path (Unweighted Graph)



Shortest path from A to D using BFS :

A → B → D (2 edges)

A → C → D (2 edges)

BFS gives minimum number of edges.

10. Disconnected Graph Traversal

If a graph is disconnected, run BFS/DFS from unvisited vertices as well.

Example :



Traversal steps :

1. BFS/DFS from A (visits A, B)
2. Then BFS/DFS from C (visits C, D)

11. Important Difference

BFS

Start
↓
Visit nearest vertices
↓
Move to next level
↓
Continue

DFS

Start
↓
Go deep
↓
Reach dead end
↓
Backtrack
↓
Explore another path

12. Summary of Graph Representation

Representation	Space Complexity	Best For
Adjacency Matrix	$O(V^2)$	Dense Graph
Adjacency List	O(V + E)	Sparse Graph

Note :

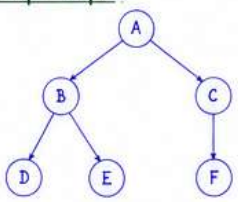
- ★ BFS uses Queue.
- ★ DFS uses Stack or Recursion.
- ★ Both BFS and DFS have time complexity O(V + E).
- ★ BFS finds shortest path in unweighted graph.
- ★ DFS is useful for cycle detection and topological sorting.

Graph Search and Traversal Algorithms (Contd.)

Apna College Bihar

13. DFS vs BFS Visualization

Example Graph :



BFS (from A) :

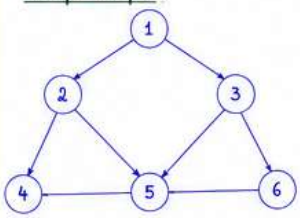
A → B → C → D → E → F

DFS (from A) :

A → B → D → E → C → F

14. Another Example

Example Graph :



BFS (from 1) :

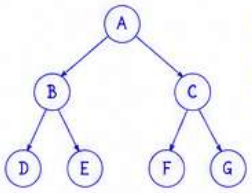
1 → 2 → 3 → 4 → 5 → 6

DFS (from 1) :

1 → 2 → 4 → 5 → 3 → 6

15. BFS Level Order Traversal

Example Graph :



Levels :

Level 0 : A

Level 1 : B, C

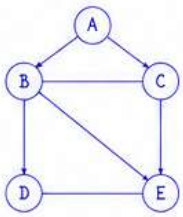
Level 2 : D, E, F, G

BFS Order :

A → B → C → D → E → F → G

16. DFS Path Exploration

Example Graph :



One possible DFS from A :

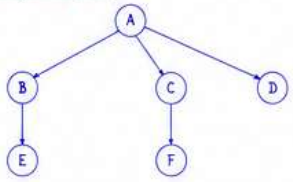
A → B → D → E → C

(Order may change based on adjacency consideration)

17. BFS Shortest Path Example

Find shortest path from A to F.

Example Graph :



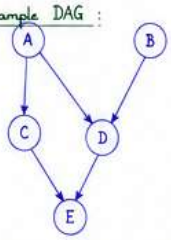
Shortest path from A to F using BFS :

A → C → F (2 edges)

18. Topological Sorting using DFS

Topological sorting is possible in a Directed Acyclic Graph (DAG).

Example DAG :



One Possible DFS Order (starting from A) :

A → C → D → E → B

Topological Order (Reverse of finishing time) :

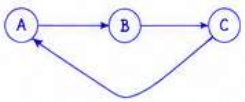
B → A → C → D → E

Note : Topological order may vary.

19. Cycle Detection using DFS

If during DFS we visit a vertex that is already in the current recursion stack, then a cycle exists.

Example (Graph with Cycle) :



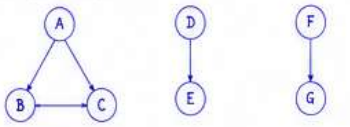
DFS Visit Order (starting from A) :

A (in stack)
↓
B (in stack)
↓
C (in stack)
↓
A (already in stack) ⇒ Cycle found

20. Connected Components

For disconnected graphs, run BFS or DFS from every unvisited vertex.

Example Graph :



Component 1 (A, B, C)

Component 2 (D, E)

Component 3 (F, G)

Process :

1. Start BFS/DFS from A → visit all in component 1.
2. Next unvisited vertex D → visit component 2.
3. Next unvisited vertex F → visit component 3.

21. Comparison Table

Feature	BFS	DFS
Full Form	Breadth First Search	Depth First Search
Approach	Level by level	Go as deep as possible
Data Structure	Queue (FIFO)	Stack (LIFO) or Recursion
Shortest Path in Unweighted Graph	Yes	Not guaranteed
Memory Usage	Higher (in wide graphs)	Lower (in wide graphs)
Best For	Shortest path, Level order search, Connectivity	Cycle detection, Topological sorting, Path exploration

22. Time and Space Complexity

For a graph represented using Adjacency List.

Time Complexity :

BFS = O(V + E)

DFS = O(V + E)

Space Complexity :

BFS = O(V)

DFS = O(V)

(Auxiliary Space)

Where :

V = Number of vertices

E = Number of edges

23. BFS and DFS Summary

BFS (Queue)

- Visit nearest vertices first.
- Explore level by level.
- Find shortest path in unweighted graph.
- Used in breadth-wise problems.
- Good for finding minimum number of steps.

DFS (Stack / Recursion)

- Go deep, then backtrack.
- Used in depth-wise problems.
- Not optimal for shortest path.
- Used in cycle detection, topological sorting, backtracking, etc.

24. Important Exam Points

- ✓ BFS uses Queue (FIFO).
- ✓ DFS uses Stack or Recursion.
- ✓ BFS visits vertices level by level.
- ✓ DFS goes deep before backtracking.
- ✓ BFS finds shortest path in unweighted graph.
- ✓ Time complexity of BFS and DFS is O(V + E).
- ✓ Space complexity is O(V).
- ✓ BFS is good for shortest path and level order.
- ✓ DFS is good for cycle detection, topological sorting and path exploration.
- ✓ For disconnected graph, run BFS/DFS from every unvisited vertex.

Complexity Analysis of Graph Algorithms

Apna College Bihar

Introduction :

Complexity Analysis tells us how much time and memory an algorithm requires as the size of the input increases.

V = Number of vertices (nodes)
E = Number of edges

Graph Representations :

- ★ Adjacency Matrix
- ★ Adjacency List

1. Complexity of Graph Representation

A. Adjacency Matrix

Matrix size = V × V

- ★ Space Complexity : $O(V^2)$
- ★ Edge Checking : $O(1)$
- ★ Finding all neighbors of a vertex : $O(V)$

Explanation :

We may need to scan an entire row to find all neighbors.

B. Adjacency List

- ★ Space Complexity : $O(V + E)$
- ★ Finding all neighbors of a vertex : $O(d)$ (d = degree of vertex)
- ★ Edge Checking : $O(d)$ (Generally)

Explanation :

We store only existing edges, so space is more efficient for sparse graphs.

2. BFS Complexity (Using Adjacency List)

Breadth First Search uses a Queue.

Time Complexity : $O(V + E)$

Why ?

- ★ Each vertex is visited at most once. → $O(V)$
- ★ Each edge is examined a limited number of times → $O(E)$

Therefore, $O(V) + O(E) = O(V + E)$

Space Complexity : $O(V)$

Queue and visited array can have up to $O(V)$ elements.

3. DFS Complexity (Using Adjacency List)

Depth First Search uses Stack or Recursion.

Time Complexity : $O(V + E)$

Space Complexity : $O(V)$

Why ?

- ★ Each vertex is visited at most once.
- ★ Each edge is examined a limited number of times.

4. BFS and DFS using Adjacency Matrix

When BFS or DFS uses an adjacency matrix, every vertex may need to scan an entire row.

Time Complexity : $O(V^2)$

Space Complexity : $O(V^2)$

Where matrix of size V × V is stored.

5. Comparison of BFS and DFS

Feature	BFS	DFS
Full Form	Breadth First Search	Depth First Search
Data Structure	Queue	Stack / Recursion
Time - Adjacency List	$O(V + E)$	$O(V + E)$
Time - Matrix	$O(V^2)$	$O(V^2)$
Auxiliary Space	$O(V)$	$O(V)$
Shortest path in unweighted graph	Yes	Not guaranteed
Cycle Detection	Yes	Yes
Topological Sorting	No	Yes
Level-wise Traversal	Yes	No

6. Complexity of Common Graph Operations

Operation	Adjacency Matrix	Adjacency List
Space	$O(V^2)$	$O(V + E)$
Check Edge (u, v)	$O(1)$	$O(\text{degree of } u)$
Find All Neighbors of a Vertex	$O(V)$	$O(\text{degree of vertex})$
BFS	$O(V^2)$	$O(V + E)$
DFS	$O(V^2)$	$O(V + E)$
Add Edge (u, v)	$O(1)$	$O(1)$ *
Remove Edge (u, v)	$O(1)$	$O(\text{degree of } u)$

* Assuming insertion at appropriate position in adjacency list is $O(1)$.

7. Complexity of Connected Components

To find all connected components in a graph, run BFS or DFS from every unvisited vertex.

Using Adjacency List : $O(V + E)$

Because every vertex and every edge is processed at most once.

8. Shortest Path Using BFS

In an unweighted graph, BFS finds the shortest path (minimum number of edges) from a source.

Using Adjacency List : $O(V + E)$

9. Topological Sorting

Topological sorting is done using :

- ★ DFS
- ★ Kahn's Algorithm (using BFS)

Applicable to DAG.

Using Adjacency List : $O(V + E)$

10. Cycle Detection

Cycle detection using BFS or DFS can be performed in :

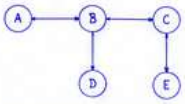
Using Adjacency List : $O(V + E)$

11. Sparse vs Dense Graph

Sparse Graph

- ★ Few edges.
- ★ $E << V^2$

Example :

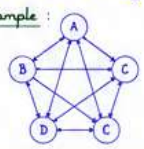


Use Adjacency List
Space = $O(V + E)$

Dense Graph

- ★ Many edges.
- ★ $E \approx V(V - 1\frac{1}{2})$

Example :



Use Adjacency Matrix
Space = $O(V^2)$

12. Big-O Summary

A. Using Adjacency Matrix

- ★ Space : $O(V^2)$
- ★ Edge Checking : $O(1)$
- ★ BFS : $O(V^2)$
- ★ DFS : $O(V^2)$

B. Using Adjacency List

- ★ Space : $O(V + E)$
- ★ BFS : $O(V + E)$
- ★ DFS : $O(V + E)$
- ★ Edge Checking : $O(\text{degree})$

13. Important Exam Points

- ★ Adjacency Matrix requires $O(V^2)$ space.
- ★ Adjacency List requires $O(V + E)$ space.
- ★ BFS using adjacency list = $O(V + E)$.
- ★ DFS using adjacency list = $O(V + E)$.
- ★ BFS / DFS using adjacency matrix = $O(V^2)$.
- ★ BFS auxiliary space = $O(V)$.
- ★ DFS auxiliary space = $O(V)$.
- ★ Adjacency List is better for sparse graphs.
- ★ Adjacency Matrix is useful for fast edge checking ($O(1)$).
- ★ BFS gives shortest path in unweighted graphs.
- ★ Topological sorting and cycle detection can be performed in $O(V + E)$.